SKILLS FOR REAL ENGINEERS · BILINGUAL EDITION

Matt Pocock Skills
双语工程实践手册

27 个正式技能 · 英文原文在上 · 简体中文翻译在下

非官方学习版 · v1.3.1 · 固定提交 b0618bc4
编译日期 2026-10-08

Contents 目录

目录与 PDF 书签包含阅读导引、27 个技能章节、主要小节和附录。

About This Edition 关于本版 12

Scope 收录范围 12

What Changed Since 1.2.3 自 1.2.3 以来的变化 12

Recommended Reading Order 推荐阅读顺序 13

Bilingual Layout 双语版式 13

Installation 安装 13

Version 版本 14

Ask Matt 问问 Matt 15

The main flow: idea → ship 主流程：想法 → 交付 15

On-ramps 匝道 18

Codebase health 代码库健康 19

Vocabulary underneath 底层词汇 19

Phase boundaries 阶段边界 19

Standalone 独立技能 20

Precondition 前置条件 22

Phase boundaries 阶段边界 (PHASE-BOUNDARIES.md) 22

The five options 五个选项 23

The tree 决策树 23

Primary and secondary sources 一手来源与二手来源 25

These are judgement calls 这些都是判断题 25

Diagnosing Bugs 诊断 Bug 29

Redact 脱敏 29

Phase 1: Build a feedback loop 阶段 1：构建反馈循环 . . . 29

Phase 2: Reproduce + minimise 阶段 2：复现 + 最小化 . . . 32

Phase 3: Hypothesise 阶段 3：提出假设 . . . 33

Phase 4: Instrument 阶段 4：插桩 . . . 34

Phase 5: Fix + regression test 阶段 5：修复 + 回归测试 . . . 34

Phase 6: Cleanup 阶段 6：清理 . . . 35

Grill With Docs 文档伴随式深度访谈 . . . 39

Triage 分诊 . . . 43

Reference docs 参考文档 . . . 43

Roles 角色 . . . 44

Invocation 调用 . . . 45

Show what needs attention 展示需要关注的内容 . . . 45

Triage a specific issue or PR 分诊某个具体的 issue 或 PR . . . 46

Quick state override 快速覆盖状态 . . . 48

Needs-info template Needs-info 模板 . . . 48

Resuming a previous session 恢复之前的会话 . . . 48

Writing Agent Briefs 撰写智能体简报 (AGENT-BRIEF.md) . . . 49

Principles 原则 . . . 49

Template 模板 . . . 51

Examples 示例 . . . 52

Out-of-Scope Knowledge Base 范围外知识库 (OUT-OF-SCOPE.md) . . . 58

Directory structure 目录结构 . . . 58

File format 文件格式 . . . 58

When to check .out-of-scope/ 何时检查 .out-of-scope/ . . . 60

When to write to .out-of-scope/ 何时写入 .out-of-scope/ . . . 61

Updating or removing out-of-scope files 更新或删除范围外文件 . . . 62

Improve Codebase Architecture 改进代码库架构 . . . 65

Process 流程 . . . 65

HTML Report Format HTML 报告格式 (HTML-REPORT.md) . . . 69

Scaffold 骨架 . . . 70

Header 页头 . . . 71

Candidate card 候选项卡片 . . . 71

Diagram patterns 图示模式 . . . 72

Style guidance 样式指导 . . . 73

Top recommendation section 首要推荐一节 . . . 74

Tone 语气 . . . 74

Setup Matt Pocock's Skills 配置 Matt Pocock 的技能 . . . 79

Process 流程 . . . 79

Domain Docs 领域文档 (domain.md) . . . 85

Before exploring, read these 探索之前，先读这些 . . . 85

File structure 文件结构 . . . 85

Use the glossary's vocabulary 使用术语表的词汇 . . . 86

Flag ADR conflicts 标出与 ADR 的冲突 . . . 87

Issue tracker: GitHub Issue 跟踪器：GitHub (issue-tracker-github.md) . . . 87

Conventions 约定 . . . 87

Pull requests as a triage surface 把 Pull Request 作为分诊入口 . . . 88

When a skill says "publish to the issue tracker" 当某个技能说“发布到 issue 跟踪器”时 . . . 88

When a skill says "fetch the relevant ticket" 当某个技能说“获取相关票据”时 . . . 89

Wayfinding operations 寻路操作 . . . 89

Issue tracker: GitLab Issue 跟踪器：GitLab (issue-tracker-gitlab.md) . . . 90

Conventions 约定 90

Merge requests as a triage surface 把合并请求作为分诊入口 91

When a skill says "publish to the issue tracker" 当某个技能说“发布到 issue 跟踪器”时 91

When a skill says "fetch the relevant ticket" 当某个技能说“获取相关票据”时 92

Wayfinding operations 寻路操作 92

Issue tracker: Local Markdown Issue 跟踪器：本地 Markdown (issue-tracker-local.md) 93

Conventions 约定 93

When a skill says "publish to the issue tracker" 当某个技能说“发布到 issue 跟踪器”时 94

When a skill says "fetch the relevant ticket" 当某个技能说“获取相关票据”时 94

Wayfinding operations 寻路操作 94

Triage Labels 分诊标签 (triage-labels.md) 95

Test-Driven Development 测试驱动开发 99

What a good test is 什么是好测试 99

Seams: where tests go 接缝：测试放在哪里 99

Anti-patterns 反模式 100

Rules of the loop 循环的规则 101

When to Mock 何时使用 mock (mocking.md) 101

Designing for Mockability 为可 mock 性而设计 102

Good and Bad Tests 好测试与坏测试 (tests.md) 103

Good Tests 好测试 104

Bad Tests 坏测试 104

To Spec 转为规范 109

Process 流程 109

Problem Statement 问题陈述 110

Solution 解决方案 110

User Stories 用户故事 . . . 110
Implementation Decisions 实现决策 . . . 110
Testing Decisions 测试决策 . . . 111
Out of Scope 范围之外 . . . 112
Further Notes 补充说明 . . . 112

To Tickets 转为票据 . . . 115

Process 流程 . . . 115
: : <票据标题> . . . 118
Parent 父 issue . . . 119
What to build 要构建什么 . . . 119
Acceptance criteria 验收标准 . . . 119
Blocked by 被阻塞于 . . . 119

Wayfinder 寻路者 . . . 123

Plan, don't do 只规划，不动手 . . . 123
Refer by name 用名称指代 . . . 124
The Map 地图 . . . 124
Ticket Types 票据类型 . . . 126
Fog of war 战争迷雾 . . . 127
Out of scope 范围之外 . . . 128
Invocation 调用 . . . 129

Implement 实现 . . . 135

Implement Spec 实现规范 . . . 139

Steps 步骤 . . . 139

Prototype 原型 . . . 145

Pick a branch 选择分支 . . . 145

Rules that apply to both 两个分支共同适用的规则 . . . 146

Logic Prototype 逻辑原型 (LOGIC.md) . . . 147

When this is the right shape 何时适合用这种形态 . . . 147

Process 流程 . . . 148

Anti-patterns 反模式 . . . 150

UI Prototype UI 原型 (UI.md) . . . 151

When this is the right shape 何时适合用这种形态 . . . 151

Two sub-shapes: strongly prefer sub-shape A 两种子形态：强烈优先选择子形态 A . . . 152

Process 流程 . . . 153

Anti-patterns 反模式 . . . 156

Research 调研 . . . 159

Domain Modeling 领域建模 . . . 163

File structure 文件结构 . . . 163

During the session 会话进行中 . . . 164

ADR Format ADR 格式 (ADR-FORMAT.md) . . . 166

Template 模板 . . . 166

Optional sections 可选小节 . . . 166

Numbering 编号 . . . 167

When to offer an ADR 何时提议 ADR . . . 167

GLOSSARY.md Format GLOSSARY.md 格式 (GLOSSARY-FORMAT.md) . . . 168

Structure 结构 . . . 169

Rules 规则 . . . 169

Single vs multi-context repos 单上下文与多上下文仓库 . . . 170

Codebase Design 代码库设计 . . . 175

Glossary 术语表 . . . 175

Deep vs shallow 深与浅 . . . 176
Principles 原则 . . . 178
Designing for testability 为可测试性而设计 . . . 178
Relationships 关系 . . . 179
Rejected framings 被否定的框架 . . . 179
Going deeper 更进一步 . . . 180
Deepening 加深 (DEEPENING.md) . . . 180
Dependency categories 依赖类别 . . . 180
Seam discipline 接缝纪律 . . . 181
Testing strategy: replace, don't layer 测试策略：替换，而非叠加 . . . 182
Design It Twice 设计两次 (DESIGN-IT-TWICE.md) . . . 182
Process 流程 . . . 183

Code Review 代码审查 . . . 187
Process 流程 . . . 187
Why two axes 为什么要分两个维度 . . . 192

PR PR 正文 . . . 195
Sections 各部分说明 . . . 196
Credits 致谢 (CREDITS.md) . . . 200

Retro 复盘 . . . 203
Steps 步骤 . . . 203
Reference 参考 . . . 205

Wizard 向导 . . . 209
Process 流程 . . . 210

Grill Me 追问我 . . . 215

Grilling 深度追问 219

Handoff 交接 223

Teach 教学 227

Teaching Workspace 教学工作区 227

Philosophy 理念 228

Lessons 课程 229

Assets 资产 230

The Mission 使命 230

Zone Of Proximal Development 最近发展区 231

Knowledge 知识 231

Skills 技能 232

Acquiring Wisdom 获取智慧 232

Reference Documents 参考文档 233

NOTES.md NOTES.md 234

GLOSSARY.md Format GLOSSARY.md 格式 (GLOSSARY-FORMAT.md) 234

Structure 结构 235

Rules 规则 235

Learning Record Format 学习记录格式 (LEARNING-RECORD-FORMAT.md) 236

Template 模板 237

Optional sections 可选小节 237

Numbering 编号 237

When to write a learning record 何时写学习记录 238

Supersession 取代 238

MISSION.md Format MISSION.md 格式 (MISSION-FORMAT.md) 239

Template 模板 239

Rules 规则 . . . 240
RESOURCES.md Format RESOURCES.md 格式 (RESOURCES-FORMAT.md) . . . 240
Structure 结构 . . . 241
Rules 规则 . . . 241
To Questionnaire 转为问卷 . . . 245
Document structure 文档结构 . . . 246
<问卷标题> . . . 246
Context 背景 . . . 246
How to answer 如何作答 . . . 246
<主题标题> . . . 246
Anything else? 还有其他吗？ . . . 247
Wait What 等等，什么？ . . . 251
Writing for Agents 为智能体写作 . . . 255
Context pointers 上下文指针 . . . 255
The two loads 两种负载 . . . 256
Information hierarchy 信息层级 . . . 256
Steps and completion criteria 步骤与完成标准 . . . 258
When to split 何时拆分 . . . 258
Leading words 引导词 . . . 259
Pruning 修剪 . . . 260
Skill mechanics 技能机制 (SKILL-MECHANICS.md) . . . 261
Invocation 调用方式 . . . 261
Splitting by invocation 按调用方式拆分 . . . 262
Router skills 路由技能 . . . 263
Reference and License 参考与许可证 . . . 267

Skill Index 技能索引 267

Fixed Source Links 固定源码链接 269

License 许可证 269

START HERE · 从这里开始

About This Edition
关于本版

Scope, reading order, installation, and reproducibility.

了解收录范围、阅读顺序、安装方式与固定版本。

This book is a fixed, bilingual learning edition of the 27 skills officially shipped by the mattpocock/skills plugin: 20 Engineering skills and 7 Productivity skills, exactly as listed in `.claude-plugin/plugin.json`. In-progress, miscellaneous, and deprecated material is intentionally excluded.

本书是 mattpocock/skills 插件正式发布的 27 个技能的固定版本双语学习版：20 个 Engineering 技能、7 个 Productivity 技能，与 `.claude-plugin/plugin.json` 中的清单完全一致。开发中、杂项和已废弃的内容均被明确排除。

Scope 收录范围

Each chapter reproduces one complete `SKILL.md` from the pinned source commit, followed by every sibling Markdown reference file the skill ships with (23 in total). The English source appears first, followed immediately by a block-matched Simplified Chinese translation.

每章完整复现固定源码 commit 中的一份 `SKILL.md`，随后附上该技能自带的全部同级 Markdown 参考文件（共 23 份）。英文原文在上，逐块对应的简体中文翻译紧随其后。

What Changed Since 1.2.3 自 1.2.3 以来的变化

The previous edition was pinned to version 1.2.3 at commit 84fdeffd (built 2026-08-10). This edition adds and removes the following skills:

上一版固定在版本 1.2.3、commit 84fdeffd（构建于 2026-08-10）。本版新增和移除了以下技能：

- Added `implement-spec`: implements the result of `/to-spec` and `/to-tickets` in code.

  新增 `implement-spec`：把 `/to-spec` 与 `/to-tickets` 的产出落实为代码。

- Added `pr`: a template and visual menu for writing a PR body, with its own `CREDITS.md`.

  新增 `pr`：撰写 PR 正文的模板与可视化手段清单，附带独立的 `CREDITS.md`。

- Added `retro`: a retrospective on a coding session, placed at the end of the main flow.

  新增 `retro`：对一次编码会话进行复盘，位于主流程末尾。

- Removed `resolving-merge-conflicts`, which the upstream repository deleted.

  移除 `resolving-merge-conflicts`，上游仓库已将其删除。

Across the remaining skills, the `CONTEXT.md` convention was renamed to `GLOSSARY.md` (so `domain-modeling` now ships `GLOSSARY-FORMAT.md`), and `ask-matt`, `wayfinder`, `setup-matt-pocock-skills`, `diagnosing-bugs`, `triage`, and `code-review` received the largest rewrites.

在其余技能中，`CONTEXT.md` 约定已更名为 `GLOSSARY.md`（因此 `domain-modeling` 现在附带 `GLOSSARY-FORMAT.md`）；`ask-matt`、`wayfinder`、`setup-matt-pocock-skills`、`diagnosing-bugs`、`triage` 和 `code-review` 改写幅度最大。

Recommended Reading Order 推荐阅读顺序

1. Start with Ask Matt for the map of the whole system.

   先读 Ask Matt，掌握整套体系的地图。

2. Read Setup Matt Pocock's Skills before applying the engineering flows to a repository.

   将工程流程应用到仓库前，阅读 Setup Matt Pocock's Skills。

3. Follow the main delivery flow: Grill With Docs → To Spec → To Tickets → Implement or Implement Spec → PR → Retro.

   沿主交付流程阅读：Grill With Docs → To Spec → To Tickets → Implement 或 Implement Spec → PR → Retro。

4. Study the discipline layer: TDD, Diagnosing Bugs, Code Review, Domain Modeling, and Codebase Design.

   学习底层纪律：TDD、Diagnosing Bugs、Code Review、Domain Modeling 和 Codebase Design。

5. Use the remaining chapters as standalone tools when their trigger fits.

   其余章节作为独立工具，在触发条件匹配时使用。

Bilingual Layout 双语版式

English source is always above its Chinese translation. Code, commands, identifiers, paths, URLs, labels, and protocol fields remain unchanged where translation would alter their behavior.

英文原文始终位于中文翻译上方。代码、命令、标识符、路径、URL、label 和协议字段，在翻译会改变其行为时保持不变。

Installation 安装

Claude Code installs the self-updating plugin from the official marketplace; any other agent can copy editable files with the skills installer. Pick one per agent, then run `/setup-matt-pocock-skills` once per repository.

Claude Code 从官方市场安装可自动更新的插件；其他智能体可以用 skills 安装器复制可编辑的文件。每个智能体只选一种方式，然后在每个仓库中运行一次 `/setup-matt-pocock-skills`。

```
claude plugin install mattpocock-skills@claude-plugins-official
npx skills@latest add mattpocock/skills
```


```
claude plugin install mattpocock-skills@claude-plugins-official
npx skills@latest add mattpocock/skills
```


Version 版本

This edition is pinned to version 1.3.1 (package version at `v1.3.1-44-gb0618bc`) and commit b0618bc436ad893b3c5e84e55fba86586d34a404, built on 2026-10-08. Every chapter links to that immutable source revision.

本版固定为版本 1.3.1（包版本，对应 `v1.3.1-44-gb0618bc`）、commit b0618bc436ad893b3c5e84e55fba86586d34a404，构建日期为 2026-10-08。每一章均链接到该不可变源码版本。

CHAPTER 01 · SKILL

Ask Matt
问问 Matt

Ask which skill or flow fits your situation. A router over the skills in this repo.

询问哪个技能或流程适合你当前的情况。这是本仓库所有技能之上的路由器。

Source 原文 · v1.3.1 · b0618bc4

You don't remember every skill, so ask.

你记不住每一个技能，所以来问。

Before stating what a skill does or recommending a step be skipped, read that skill's SKILL.md: the summaries here are for orientation only.

在陈述某个技能的作用、或建议跳过某一步之前，先读那个技能的 SKILL.md：这里的摘要只用于帮你辨认方向。

A **flow** is a path through the skills. Most paths run along one **main flow**, and two **on-ramps** merge onto it. Everything else is standalone, or a vocabulary layer that runs underneath.

流程（flow）是一条贯穿各技能的路径。大多数路径都沿着一条**主流程**前进，另有两条**匝道**汇入其中。其余的要么是独立技能，要么是在底层运行的词汇层。

The main flow: idea → ship 主流程：想法 → 交付

The route most work travels. You have an idea and want it built.

大多数工作走的路线。你有一个想法，想把它做出来。

1. `/grill-with-docs` sharpens the idea by interview. Start here whenever you are **working in a working directory**: it's stateful, retaining what it learns in `GLOSSARY.md` and ADRs. (No working directory? Use `/grill-me` instead, covered under Standalone. Both run the same `/grilling` primitive; `grill-with-docs` is the one that leaves a paper trail, which makes it the better of the two whenever a repo is there to leave it in.)

   `/grill-with-docs` 通过访谈把想法打磨清晰。只要你**在一个工作目录里工作**，就从这里开始：它是有状态的，会把学到的东西保存在 `GLOSSARY.md` 和 ADR（架构决策记录）中。（没有工作目录？改用 `/grill-me`，见“独立技能”一节。两者运行的是同一个 `/grilling` 原语；`grill-with-docs` 会留下书面记录，所以只要有仓库可供留存，它就是两者中更好的那个。）

2. Branch: can you settle every question in conversation? If a question needs a runnable answer (state, business logic, a UI you have to see), detour through a prototype, bridged by /handoff in both directions (a prototype lives in its own directory, which is exactly what /handoff is for; see Phase boundaries):

   分支：能否在对话中解决所有问题？ 如果某个问题需要一个可运行的答案（状态、业务逻辑、必须亲眼看到的 UI），就绕道做一个原型，两个方向都用 /handoff 衔接（原型位于自己的目录中，这恰恰是 /handoff 的用途；见“阶段边界”）：

- /handoff out, then open a fresh session against that file,

  先 /handoff 出去，然后针对那个文件打开一个全新会话，

- /prototype to answer the question with throwaway code,

  用 /prototype 以一次性代码回答这个问题，

- /handoff back what you learned, and reference it from the original idea thread.

  再把学到的东西 /handoff 回来，并在最初的想法线程中引用它。

3. Branch: is this a multi-session build?

   分支：这是一次跨多个会话的构建吗？

- Yes → /to-spec (turn the thread into a spec), then /to-tickets to split it into tracer-bullet tickets, each declaring its blocking edges. Then work the tickets one of two ways:

  是 → /to-spec（把线程转成规范），然后用 /to-tickets 把它拆分成曳光弹票据，每张票据都声明自己的**阻塞边**。然后用以下两种方式之一处理这些票据：

- /implement per ticket, /clearing context between each one. On a local tracker that's one file per ticket under .scratch/<feature>/issues/, worked blockers-first by hand; on a real tracker the edges become native blocking links, so any ticket whose blockers are done can be grabbed. Each ticket is self-contained, so the last one's context is disposable.

  每张票据各跑一次 /implement，**每张之间都 /clear 上下文**。在本地跟踪器上，每张票据是 .scratch/<feature>/issues/ 下的一个文件，由你手动按“先处理阻塞项”的顺序推进；在真正的跟踪器上，这些边会变成原生的阻塞链接，因此任何阻塞项已完成的票据都可以被领取。每张票据都是自包含的，所以上一张的上下文可以丢弃。

- /implement-spec for the whole spec in one run. It reads the tickets as a task graph, runs implementer subagents across the ready frontier in parallel, and lands everything on one integration branch. Reach for it when you'd rather orchestrate the build than drive each ticket yourself.

  用 /implement-spec 一次性跑完整个规范。它把票据读成一张**任务图**，在就绪的**前沿**上并行运行实现者子智能体，并把所有成果落到同一个**集成分支**上。当你更愿意编排整个构建、而不是亲自驱动每张票据时，就用它。

- No → /implement right here, in the same context window.

  否 → 就在这里、在同一个上下文窗口中运行 /implement。

Either way, the code gets built by driving /tdd (one red-green slice at a time) and closes out with /code-review, a two-axis review (Standards + Spec) of the diff. /implement runs both per ticket; /implement-spec's implementers each drive /tdd, and it runs one /code-review over the integration branch. Reach for /tdd on its own when you just want to build a concrete behaviour test-first without a full spec, and /code-review on its own whenever you want to review a branch or PR against a fixed point.

无论哪种方式，代码都是通过驱动 /tdd（一次一个红-绿切片）构建出来的，并以 /code-review 收尾--对 diff 做双轴评审（Standards + Spec）。/implement 对每张票据都运行这两者；/implement-spec 的每个实现者各自驱动 /tdd，并且它会对集成分支运行一次 /code-review。如果你只是想以测试先行的方式构建某个具体行为、而不需要完整规范，就单独使用 /tdd；任何时候想对照某个固定点评审一个分支或 PR，就单独使用 /code-review。

When the work goes up as a pull request, /pr shapes the body: the smallest visual that shows the change, before/after evidence that it works, and a one-way or two-way door call. It's model-invoked, so the agent reaches for it whenever it writes a PR.

当工作以 pull request 的形式提交时，/pr 负责组织 PR 正文：能展示改动的最小可视化、证明其可行的前后对比证据，以及“单向门还是双向门”的判断。它由模型调用，所以智能体每次写 PR 时都会主动用它。

4. /retro closes the loop. After a build, and especially one that went sideways, it looks back over the session and suggests changes to the agent's environment, not the code: navigation pointers, automated checks, the coding standards /code-review enforces, steering files, tooling. Mechanical mistakes become deterministic checks; judgement calls become coding standards. The next build then starts from a better environment.

   /retro 闭合这个循环。一次构建之后，尤其是出了岔子的构建之后，它会回顾整个会话，并就智能体的**环境**（而不是代码）提出改进建议：导航指引、自动化检查、/code-review 所执行的编码标准、引导文件、工具。机械性错误变成确定性检查；判断性问题变成编码标准。于是下一次构建就从一个更好的环境起步。

Context hygiene 上下文卫生

Keep steps 1-3 in one unbroken context window (don't compact or clear until after /to-tickets) so the grilling, spec, and tickets all build on the same thinking. Each /implement then starts fresh, working from the ticket. Run /retro in the session it's looking back on, before you clear; after clearing, point it at that session's log instead.

让第 1-3 步保持在**一个不间断的上下文窗口**中（在 /to-tickets 之前不要 compact 或 clear），这样深度追问、规范和票据都建立在同一套思考之上。之后每次 /implement 都从全新状态开始，以票据为依据工作。在 /retro 所回顾的那个会话中、清空之前运行它；如果已经清空，就把它指向那个会话的日志。

The limit on this is the smart zone: the window (~150k tokens on state-of-the-art models) within which the model still reasons sharply. If a session approaches it before /to-tickets, don't push on degraded; /compact at the nearest phase boundary and carry on (see Phase boundaries).

这件事的上限是 smart zone（智能区）：模型仍能敏锐推理的窗口范围（在最先进的模型上约为 150k token）。如果某个会话在 /to-tickets 之前就接近这个上限，不要在能力退化的状态下硬撑；在最近的阶段边界处 /compact，然后继续（见“阶段边界”）。

On-ramps 匝道

A starting situation that generates work, then merges onto the main flow.

一种会产生工作、随后汇入主流程的起始情境。

- Bugs and requests piling up → /triage. It moves issues through triage roles and produces agent-ready issues, which /implement later picks up.

  bug 和需求堆积如山 → /triage。它让 issue 依次经过各个分诊角色，产出可交给智能体的 issue，之后由 /implement 接手。

Triage is only for issues you didn't create: bug reports, incoming feature requests, anything that arrives raw. Tickets that /to-tickets produced are already agent-ready, so don't triage them.

分诊只适用于**不是你自己创建的** issue：bug 报告、新来的功能请求，任何原始状态进来的东西。/to-tickets 产出的票据已经可以交给智能体，所以**不要对它们分诊**。

- Something's broken → /diagnosing-bugs. For the hard ones: the bug that resists a first glance, the intermittent flake, the regression that crept in between two known-good states. It refuses to theorise until it has a tight feedback loop (one command that already goes red on this bug), then fixes with a regression test. Once the fix is in, run /retro in the same session to ask what would have prevented the bug; where the real finding is that there's no good seam to lock it down, that's a job for /improve-codebase-architecture.

  有东西坏了 → /diagnosing-bugs。专治难缠的：第一眼看不出来的 bug、时有时无的偶发故障、在两个已知正常状态之间悄悄混进来的回归。在拥有一个**紧凑的反馈循环**（一条已经能在这个 bug 上变红的命令）之前，它拒绝做任何推测，然后用一个回归测试完成修复。修复落地后，在同一个会话中运行 /retro，问一问什么本可以预防这个 bug；如果真正的发现是没有合适的接缝可以把它锁住，那就是 /improve-codebase-architecture 的活了。

- A huge, foggy effort: a greenfield project or a huge feature build, too big for one session → /wayfinder, the most cognitively demanding flow here. When the way from here to the destination isn't visible yet, it charts a shared map of decision tickets on the issue tracker and resolves them one at a time, producing decisions, not deliverables, until the fog is pushed back and the way is clear. Where /grill-with-docs sharpens an idea you can hold in one session, wayfinder is for the idea you can't, and it's slower and denser, so save it for exactly that, never a well-scoped feature.

  一项庞大而模糊的工作：一个全新项目或一次巨大的功能构建，一个会话装不下 → /wayfinder，这里认知负担最重的流程。当从此处到终点的路还看不清时，它会在 issue 跟踪器上绘制一张由**决策票据**组成的**共享地图**，并逐一解决它们，产出的是**决策，而不是交付物**，直到迷雾被推开、道路变得清晰。/grill-with-docs 打磨的是你能在一个会话内把握的想法，而寻路者针对的是你把握不住的想法；它更慢、更密集，所以只把它留给这种情况，绝不要用在范围清晰的功能上。

When the map clears, it hands off, it doesn't build: merge onto the main flow at /to-spec, which collapses the map's linked decisions into a buildable plan, then /to-tickets and /implement as usual. Looping the map straight into /implement skips that collapse and throws the linked detail away, so go straight to /implement only when the effort turned out genuinely small.

当地图清晰之后，**它负责交接，而不负责构建**：在 /to-spec 处汇入主流程，由它把地图上相互关联的决策收拢成一份可构建的计划，然后照常 /to-tickets 和 /implement。把地图直接接到 /implement 会跳过这次收拢，并丢掉那些关联细节，所以只有当这项工作最终被证明确实很小时，才直接进入 /implement。

Codebase health 代码库健康

Not feature work, just upkeep.

不是功能开发，只是日常维护。

- `/improve-codebase-architecture` runs whenever you have a spare moment to keep the codebase good for agents to operate in. It surfaces **deepening opportunities**; picking one _generates an idea_ you can take into the main flow at `/grill-with-docs`. It's the survey that finds the candidates; `/codebase-design` (below) is the bench you design the chosen one on.

  `/improve-codebase-architecture` 只要你有空闲就可以运行，让代码库保持适合智能体在其中工作的状态。它会找出**加深机会**；选中其中一个就会产生一个想法，你可以把它带到主流程的 `/grill-with-docs` 中。它是找出候选项的勘察；`/codebase-design`（见下文）则是你设计选中项的工作台。

Vocabulary underneath 底层词汇

Two model-invoked references that run beneath the other skills, each the single source of truth for its vocabulary. Reach for them directly when the **words**, not the process, are the problem; or let the skills above pull them in.

两份由模型调用的参考资料，运行在其他技能的下方，各自是其词汇的唯一事实来源。当问题出在**用词**而不是流程上时，直接使用它们；或者让上面的技能把它们拉进来。

- `/domain-modeling`: sharpen the project's domain language: challenge a fuzzy term, resolve an overloaded word ("account" doing three jobs), record a hard-to-reverse decision as an ADR. It's the active discipline `/grill-with-docs` drives to keep `GLOSSARY.md` a clean glossary.

  `/domain-modeling`：打磨项目的领域语言：质疑一个含糊的术语，化解一个被过度使用的词（一个“account”身兼三职），把一个难以逆转的决策记录为 ADR。它是 `/grill-with-docs` 所驱动的主动纪律，用来让 `GLOSSARY.md` 保持为一份干净的术语表。

- `/codebase-design` is the deep-module vocabulary (module, interface, depth, seam, adapter, leverage, locality) for designing a module's shape: a lot of behaviour behind a small interface at a clean seam. `/tdd` and `/improve-codebase-architecture` both speak it.

  `/codebase-design` 是深模块词汇（module、interface、depth、seam、adapter、leverage、locality），用于设计一个模块的形状：在一个干净的接缝处，用一个小接口承载大量行为。`/tdd` 和 `/improve-codebase-architecture` 都使用这套词汇。

Phase boundaries 阶段边界

A **phase** is a chunk of work inside a session: the grilling, the implementation, the QA. At the **boundary** between two of them you have five options, and picking between them is the fuzziest decision in this whole map:

阶段是会话中的一段工作：深度追问、实现、QA。在两个阶段之间的**边界**上，你有五个选项，而在它们之间做选择，是整张地图中最模糊的决定：

- Continue: stay put. Costs nothing, loses nothing.

  Continue（继续）：原地不动。不花任何代价，也不丢任何东西。

- /clear: empty the window, when nothing here matters to what's next.

  /clear：清空窗口，适用于此处的一切都与下一步无关的情况。

- /handoff writes a portable markdown file. Narrow: only for a new harness, a new directory, a colleague, or forking a side task mid-phase. What it buys is portability.

  /handoff 写出一个可移植的 markdown 文件。用途很窄：只用于**新的 harness**、**新的目录**、**同事**，或在**阶段中途**分叉出一个旁支任务。它换来的是可移植性。

- Subagent: send a tightly-scoped task to its own window and get a report back.

  子智能体：把一个范围严格限定的任务发到它自己的窗口中，然后拿回一份报告。

- /compact compresses this context and seeds a fresh session with it. The default, at the bottom of the tree rather than the first reach.

  /compact 压缩当前上下文，并用它为一个全新会话打底。它是**默认选项**，位于决策树的底部，而不是第一个伸手去拿的。

Read PHASE-BOUNDARIES.md for the ordered tree: the five questions, the reasoning behind each branch, and why the primary-source cost makes Continue the one to rule out first. Make the decision at a boundary; mid-phase, continue or split the rest into subagents.

阅读 PHASE-BOUNDARIES.md 了解有序的决策树：五个问题、每个分支背后的理由，以及为什么一手来源的代价让 Continue 成为首先要排除的选项。**在**边界上做这个决定；阶段中途，要么继续，要么把剩下的工作拆分给子智能体。

Standalone 独立技能

Off the main flow entirely.

完全不在主流程上。

- /grill-me: the same relentless interview as /grill-with-docs, but stateless: it saves nothing locally and builds no GLOSSARY.md. Reach for it when you are not working in a working directory (sharpening a plan, a design, a piece of writing, anything with no repo under it). If you are in a working directory, use /grill-with-docs instead: it runs the same interview and leaves a paper trail, so it is strictly the better one.

  /grill-me：与 /grill-with-docs 一样毫不留情的访谈，但它是**无状态的**：不在本地保存任何东西，也不构建 GLOSSARY.md。当你**不在工作目录中工作**时使用它（打磨一个计划、一个设计、一篇文章，任何底下没有仓库的东西）。如果你在工作目录中，就改用 /grill-with-docs：它运行同样的访谈并留下书面记录，所以严格来说它更好。

- /grilling is the interview primitive itself: rounds, the frontier, facts are the agent's job and decisions are yours. /grill-me and /grill-with-docs are the two named ways in, and /triage, /wayfinder and /improve-codebase-architecture all run it internally. Reach for it directly only when you want the interview with no wrapper around it.

  /grilling 就是访谈原语本身：轮次、前沿，事实归智能体查，决策归你做。/grill-me 和 /grill-with-docs 是两个有名字的入口，而 /triage、/wayfinder 和 /improve-codebase-architecture 都在内部运行它。只有当你想要一个没有任何包装的访谈时，才直接使用它。

- /prototype is a small, throwaway program that answers one design question: does this state model feel right, or what should this UI look like. Throwaway is a constraint on how the code is written, not a promise to destroy it: the answer folds into the real code, and the prototype itself is kept as a primary source on a prototype/<name> branch out of main, pointed at from the implementation issue. It's the detour in step 2 of the main flow, but reach for it any time a design question is hard to settle on paper.

  /prototype 是一个小型的一次性程序，用来回答一个设计问题：这个状态模型感觉对不对，或者这个 UI 应该长什么样。“一次性”是对代码写法的约束，而不是销毁它的承诺：答案会融入真正的代码，而原型本身作为**一手来源**保存在一个从 main 拉出的 prototype/<name> 分支上，并由实现 issue 指向它。它是主流程第 2 步中的那段绕路，但任何时候只要一个设计问题难以在纸面上定下来，都可以用它。

- /research: delegate reading legwork to a background agent: it investigates a question against primary sources, then leaves a cited Markdown file in the repo. Keep working while it reads. The file it produces is something to take into the main flow at /grill-with-docs, since research feeds the thinking rather than replacing it.

  /research：把阅读的跑腿活委派给一个**后台智能体**：它对照**一手来源**调查一个问题，然后在仓库中留下一份带引用的 Markdown 文件。它读资料的同时你可以继续工作。它产出的文件可以带进主流程的 /grill-with-docs，因为研究是为思考提供养料，而不是取代思考。

- /to-questionnaire comes in when the thing blocking you isn't in your head or the codebase but in someone else's, and it writes them a questionnaire to fill in. It's the inverse of /grill-me: instead of interviewing you about the subject, it interviews you about the send (who it's going to, what you need back) and aims the questions at the gap. What comes back is material for /grill-with-docs or /to-spec.

  /to-questionnaire 用在挡住你的东西既不在你脑子里、也不在代码库里，而是在**别人**脑子里的时候，它会为对方写一份问卷来填写。它是 /grill-me 的反面：它不是就主题访谈你，而是就**发送**访谈你（发给谁、你需要拿回什么），并把问题瞄准信息缺口。收回来的内容可作为 /grill-with-docs 或 /to-spec 的素材。

- /wizard is for the steps only a human can take: provisioning infrastructure, setting up credentials or CI secrets, clicking through an unfamiliar third-party dashboard, running a one-off migration or cutover. It generates an interactive bash script that opens each URL, captures each value, and writes it into .env and GitHub secrets, so the procedure stops being something you re-explain to an agent every time. Model-invoked, so the agent reaches for it the moment it hits a wall only you can pass. If the agent could just do it itself, it should; this is for where a human is genuinely in the loop.

  /wizard 用于只有**人**才能完成的步骤：配置基础设施、设置凭据或 CI secrets、在一个陌生的第三方控制台里一路点击、执行一次性的迁移或切换。它会生成一个交互式 bash 脚本，打开每个 URL、捕获每个值，并把它们写进 .env 和 GitHub secrets，这样这套流程就不再是你每次都要向智能体重新解释一遍的东西。它由模型调用，所以智能体一碰到只有你才能越过的墙，就会立刻用它。如果智能体自己就能做，那就让它自己做；这个技能是给真正需要人参与的环节准备的。

- `/wait-what` is the corrective for a message that didn't land. Use it mid-conversation, inside any other skill, and the agent re-pitches what it just said with the context you were missing, in plain English, using the `GLOSSARY.md` vocabulary. It works after the fact; `/grill-with-docs` is the upfront cure, because a shared language agreed early is what stops the jargon arriving at all.

  `/wait-what` 用来纠正一条没被理解的消息。在对话中途、在任何其他技能内部使用它，智能体就会把刚才说的内容重新讲一遍，补上你缺失的上下文，用平实的英语，并使用 `GLOSSARY.md` 中的词汇。它是事后补救；`/grill-with-docs` 才是事前的根治办法，因为早早约定好的共享语言，才能从根本上阻止术语黑话的出现。

- `/teach`: learn a concept over multiple sessions, using the current directory as a stateful workspace.

  `/teach`：用多个会话学习一个概念，把当前目录当作有状态的工作区。

- `/writing-for-agents` is the reference for writing documents agents consume: skills, AGENTS.md, pointed-at docs.

  `/writing-for-agents` 是撰写供智能体使用的文档的参考：技能、AGENTS.md、被指向的文档。

Precondition 前置条件

`/setup-matt-pocock-skills`: run before your first engineering flow to configure the issue tracker, triage labels, and doc layout the other skills assume. Custom issue trackers also work.

`/setup-matt-pocock-skills`：在你第一次运行工程类流程之前执行，用来配置其他技能所假定的 issue 跟踪器、分诊标签和文档布局。自定义的 issue 跟踪器同样可用。

REFERENCE 参考文档 · PHASE-BOUNDARIES.md

Phase boundaries 阶段边界

Source 原文

A **phase** is a chunk of work inside a session: the grilling, the implementation, the QA. The definition is fuzzy on purpose: a phase ends when you think "ok, we're done with that".

阶段是会话中的一段工作：深度追问、实现、QA。这个定义是故意模糊的：当你觉得“好，这部分做完了”时，一个阶段就结束了。

The **phase boundary** is the gap between two phases, and it is the only place this decision belongs. Mid-phase there is no decision to make: continue, or split the work that's left into subagents. Compacting mid-phase makes the agent lose the thread.

阶段边界是两个阶段之间的间隙，也是这个决定唯一该出现的地方。在阶段中途没有什么需要决定的：要么继续，要么把剩下的工作拆分给子智能体。在阶段中途 compact 会让智能体丢掉思路。

The five options 五个选项

Option	What it does
Continue	Stay in the session. No context switch at all.
/clear	Empty the context window and start from nothing.
/handoff	Write a portable markdown file and seed a session anywhere with it.
Subagent	Send the task to its own context window and get a report back.
/compact	Compress this context and seed a fresh session with the summary.

选项	作用
Continue（继续）	留在当前会话中。完全不切换上下文。
/clear	清空上下文窗口，从零开始。
/handoff	写出一个可移植的 markdown 文件，并用它在任何地方为一个会话打底。
子智能体	把任务发到它自己的上下文窗口中，然后拿回一份报告。
/compact	压缩当前上下文，并用摘要为一个全新会话打底。

The tree 决策树

Work top to bottom at the boundary. The first yes wins.

在边界上从上往下逐项判断。第一个回答是的胜出。

1. Can you continue in this session? Two things make the answer yes: the next phase needs this phase as a primary source, or you have enough smart zone left (~150k tokens) for the next phase to fit. Grilling → implementation is the standard yes: the implementation wants the reasoning verbatim, not a summary of it. Continue costs nothing and loses nothing, so rule it out before anything else.

1. **你能在这个会话中继续吗？** 有两种情况会让答案为“是”：下一个阶段需要把这个阶段作为**一手来源**，或者你剩下的 smart zone（智能区）（约 150k token）足够容纳下一个阶段。深度追问 → 实现是标准的“是”：实现需要的是原原本本的推理过程，而不是它的摘要。继续不花任何代价，也不丢任何东西，所以先把它排除掉，再考虑其他选项。

2. Is the context irrelevant to what comes next? Is everything in this session (the exploration, the decisions, the dead ends) disposable? If so, /clear. It is the cheapest move on the board: it takes no time and hands back the whole window. /clear also isn't terminal: the old session stays resumable.

2. **当前上下文与接下来要做的事无关吗？** 这个会话里的一切（探索、决策、死胡同）都可以丢弃吗？如果是，就 /clear。这是棋盘上最便宜的一步：不花任何时间，还能把整个窗口还给你。/clear 也不是终点：旧会话仍然可以恢复。

The cost of getting this wrong is one-way. Clear a relevant context and you lose the why behind what you built, and no amount of reading the diff back gets it returned.

判断错误的代价是单向的。如果清掉了一个相关的上下文，你就会丢失所构建内容背后的**原因**，而且无论把 diff 重读多少遍都找不回来。

3. Do you need to hand off? `/handoff` is narrow. You need it only when you are:

3. **你需要交接吗？** `/handoff` 的用途很窄。只有在以下情况下才需要它：

- swapping to a new harness (Claude → Codex),

  换到一个**新的 harness**（Claude → Codex），

- moving to a new directory or repo,

  转到一个**新的目录**或仓库，

- sending the work to a colleague,

  把工作交给一位**同事**，

- or forking a side task you found mid-phase without derailing what you're doing.

  或者在**阶段中途**发现了一个旁支任务，想把它分叉出去而不打乱手头的事。

That list is the whole clause. What `/handoff` buys is portability: a file that travels. If nothing is travelling, you don't need it.

这份清单就是全部条件。`/handoff` 换来的是**可移植性**：一个能带走的文件。如果没有东西需要带走，你就不需要它。

4. Can the task be done AFK? Is it scoped tightly enough to run with you away from the keyboard, no steering? Then send it to a subagent and leave this session untouched. Automated review is the standard case: the agent reads the diff and reports, and you aren't needed while it does.

4. **这个任务能 AFK 完成吗？** 它的范围是否严格到你离开键盘、无需任何引导也能运行？那就把它发给一个**子智能体**，并让当前会话保持原样。自动化评审是标准案例：智能体读 diff 并报告，在它工作期间不需要你。

5. Otherwise, `/compact`. Relevant context, same harness, same directory, and you need to stay in the loop: this is where the tree lands, and it lands here often. Pass it an instruction (`/compact we're going to QA this area`) so the summary keeps what the next phase needs.

5. **否则，**`/compact`。上下文相关、同一个 harness、同一个目录，而且你需要继续参与其中：这就是决策树的落点，而且经常落在这里。给它传一条指令（`/compact we're going to QA this area`），让摘要保留下一个阶段需要的内容。

`/compact` is the default, not the first reach. It sits at the bottom because the four questions above it are all cheaper or more precise. The failure mode when people start here is a fresh session that is confidently wrong about a decision the summary flattened.

`/compact` **是默认选项，而不是第一个伸手去拿的**。它位于底部，是因为它上面的四个问题都更便宜或更精确。人们一上来就用它时的失败模式是：得到一个全新会话，却对某个被摘要压扁了的决策自信地理解错了。

Primary and secondary sources 一手来源与二手来源

Every move except **Continue** turns a **primary source** into a **secondary source**: the session as it happened, replaced by a summary of it. The trade is always the same shape:

除了 **Continue** 之外，每一步都会把**一手来源**变成**二手来源**：原原本本发生过的会话，被替换成它的摘要。这种取舍的形态始终相同：

Source	Information	Noise	Room to move
Primary (Continue)	Full	Lots	Little
Secondary (`/compact`, `/handoff`)	Lossy	Less	Lots

来源	信息	噪声	腾挪空间
一手（Continue）	完整	很多	很少
二手（`/compact`、`/handoff`）	有损	较少	很多

This is why question 1 comes first. You only pay the lossiness when staying costs more than it saves.

这就是为什么问题 1 排在第一位。只有当留下来的代价大于收益时，你才去承担这种损耗。

These are judgement calls 这些都是判断题

The questions are not objective: each has taste in it, and the same boundary can go two ways on two days. The value is in asking them **in order**, at the boundary rather than in the middle of the work.

这些问题并不客观：每个问题里都带着个人品味，同一个边界在不同的日子可能走向两个方向。价值在于**按顺序**提出它们，而且是在边界上，而不是在工作进行到一半时。

CHAPTER 02 · SKILL

Diagnosing Bugs
诊断 Bug

Diagnosis loop for hard bugs and performance regressions. Use when the user says "diagnose"/"debug this", or reports something broken/throwing/failing/slow.

针对疑难 bug 和性能回退的诊断循环。当用户说 "diagnose"/"debug this"，或报告某处坏了/抛异常/失败/变慢时使用。

Source 原文 · v1.3.1 · b0618bc4

A discipline for hard bugs. Skip phases only when explicitly justified.

一套对付疑难 bug 的纪律。只有在有明确理由时才可跳过某个阶段。

When exploring the codebase, read `GLOSSARY.md` (if it exists) to get a clear mental model of the relevant modules, and check ADRs in the area you're touching.

探索代码库时，先读 `GLOSSARY.md`（如果存在），对相关模块建立清晰的心智模型，并查看你所触及区域的 ADR（架构决策记录）。

Redact 脱敏

This skill has you show commands, outputs and captured artifacts. **Redact every secret first:** write `<REDACTED>` in its place. Build loops against env vars, so the credential stays in the environment rather than in what you show. Captured artifacts carry auth headers: quote only the lines that carry the signal.

本技能会让你展示命令、输出和捕获的产物。**先把每一个机密信息脱敏**：用 `<REDACTED>` 替换它。基于环境变量来构建循环，让凭据留在环境里，而不是出现在你展示的内容中。捕获的产物会带有认证头：只引用承载信号的那几行。

If the redacted output is not enough to diagnose the bug, say so and ask the user.

如果脱敏后的输出不足以诊断这个 bug，就直说，并询问用户。

Phase 1: Build a feedback loop 阶段 1：构建反馈循环

This is the skill. Everything else is mechanical. If you have a **tight** pass/fail signal for the bug (one that goes red on _this_ bug), you will find the cause; bisection, hypothesis-testing, and instrumentation all just consume it. If you don't have one, no amount of staring at code will save you.

这才是本技能的核心。 其余一切都是机械操作。如果你对这个 bug 有一个**紧凑**（tight）的通过/失败信号（一个会在 _这个_ bug 上变红的信号），你就一定能找到原因；二分、假设检验和插桩都只是在消耗这个信号。如果你没有，盯着代码看多久都救不了你。

Spend disproportionate effort here. Be aggressive. Be creative. Refuse to give up.

在这里投入不成比例的精力。**要激进。要有创意。绝不放弃。**

Ways to construct one, in roughly this order 构建方式，大致按以下顺序

1. Failing test at whatever seam reaches the bug: unit, integration, e2e.

   失败的测试，放在任何能触及该 bug 的接缝处：单元、集成、e2e。

2. Curl / HTTP script against a running dev server.

   Curl / HTTP **脚本**，针对正在运行的开发服务器。

3. CLI invocation with a fixture input, diffing stdout against a known-good snapshot.

   CLI **调用**，使用 fixture 输入，把 stdout 与已知正确的快照做 diff。

4. Headless browser script (Playwright / Puppeteer) that drives the UI and asserts on DOM/console/network.

   无头浏览器脚本（Playwright / Puppeteer），驱动 UI 并对 DOM/console/network 做断言。

5. Replay a captured trace. Save a real network request / payload / event log to disk; replay it through the code path in isolation.

   重放捕获的 trace。 把真实的网络请求 / payload / 事件日志存到磁盘；在隔离环境中让它重放经过该代码路径。

6. Throwaway harness. Spin up a minimal subset of the system (one service, mocked deps) that exercises the bug code path with a single function call.

   一次性测试装置。 启动系统的一个最小子集（一个服务，依赖用 mock（模拟）替代），用一次函数调用就能触发 bug 所在的代码路径。

7. Property / fuzz loop. If the bug is "sometimes wrong output", run 1000 random inputs and look for the failure mode.

   属性 / 模糊测试循环。 如果 bug 是“有时输出不对”，就跑 1000 个随机输入，寻找失败模式。

8. Bisection harness. If the bug appeared between two known states (commit, dataset, version), automate "boot at state X, check, repeat" so you can `git bisect run` it.

   二分装置。 如果 bug 出现在两个已知状态之间（提交、数据集、版本），就把“在状态 X 启动、检查、重复”自动化，这样就能用 `git bisect run` 跑它。

9. Differential loop. Run the same input through old-version vs new-version (or two configs) and diff outputs.

   差分循环。 让同一个输入分别跑过旧版本与新版本（或两套配置），然后对输出做 diff。

10. HITL bash script. Last resort. If a human must click, drive _them_ with `scripts/hitl-loop.template.sh` so the loop is still structured. Captured output feeds back to you.

    HITL bash **脚本。** 最后手段。如果必须由人来点击，就用 `scripts/hitl-loop.template.sh` 驱动 _他们_，让循环依然是结构化的。捕获的输出会反馈给你。

Build the right feedback loop, and the bug is 90% fixed.

构建出正确的反馈循环，bug 就已经修好了 90%。

Tighten the loop 收紧循环

Treat the loop as a product. Once you have _a_ loop, tighten it:

把循环当作一个产品来对待。一旦有了 _一个_ 循环，就去**收紧**它：

- Can I make it faster? (Cache setup, skip unrelated init, narrow the test scope.)

  能让它更快吗？（缓存准备工作，跳过无关的初始化，缩小测试范围。）

- Can I make the signal sharper? (Assert on the specific symptom, not "didn't crash".)

  能让信号更锐利吗？（对具体症状做断言，而不是“没崩溃”。）

- Can I make it more deterministic? (Pin time, seed RNG, isolate filesystem, freeze network.)

  能让它更确定吗？（固定时间，给随机数生成器设种子，隔离文件系统，冻结网络。）

A 30-second flaky loop is barely better than no loop; a 2-second deterministic one is tight, a debugging superpower.

一个 30 秒且时好时坏的循环，几乎不比没有循环强；一个 2 秒且确定的循环才是紧凑的，是调试的超能力。

Non-deterministic bugs 非确定性 bug

The goal is not a clean repro but a higher reproduction rate. Loop the trigger 100×, parallelise, add stress, narrow timing windows, inject sleeps. A 50%-flake bug is debuggable; 1% is not, so keep raising the rate until it's debuggable.

目标不是一个干净的复现，而是**更高的复现率**。把触发条件循环 100 次、并行化、施加压力、收窄时间窗口、注入 sleep。50% 概率出现的 bug 可以调试；1% 的不行，所以要不断提高复现率，直到它可以调试为止。

When you genuinely cannot build a loop 当你确实无法构建循环时

Stop and say so explicitly. List what you tried. Ask the user for: (a) access to whatever environment reproduces it, (b) a redacted captured artifact (HAR file, log dump, core dump, screen recording with timestamps), or (c) permission to add temporary production instrumentation. Do not proceed to hypothesise without a loop.

停下来，明确说出来。列出你尝试过的方法。向用户请求：(a) 能够复现它的任何环境的访问权限，(b) 一份脱敏后的捕获产物（HAR 文件、日志转储、core dump、带时间戳的录屏），或 (c) 添加临时生产环境插桩的许可。没有循环时，**不要**进入提出假设的环节。

Completion criterion: a tight loop that goes red 完成标准：一个会变红的紧凑循环

Phase 1 is done when the loop is **tight** and **red-capable**: you can name **one command** (a script path, a test invocation, a curl) that you have **already run at least once** (show the invocation and its output, redacted), and that is:

当循环**紧凑**且**能变红**时，阶段 1 即告完成：你能说出**一条命令**（一个脚本路径、一次测试调用、一个 curl），你**已经至少运行过一次**（展示调用及其输出，并脱敏），并且它满足：

- [ ] **Red-capable**: it drives the actual bug code path and asserts the **user's exact symptom**, so it can go red on this bug and green once fixed. Not "runs without erroring"; it must be able to _catch this specific bug_.

  [ ] **能变红**：它驱动真实的 bug 代码路径，并对**用户所说的确切症状**做断言，因此能在这个 bug 上变红，修复后变绿。不能只是“运行时不报错”；它必须能 _抓住这个具体的 bug_。

- [ ] **Deterministic**: same verdict every run (flaky bugs: a pinned, high reproduction rate, per above).

  [ ] **确定**：每次运行结论相同（对时好时坏的 bug：如上所述，是一个固定下来的高复现率）。

- [ ] **Fast**: seconds, not minutes.

  [ ] **快速**：以秒计，而不是以分钟计。

- [ ] **Agent-runnable**: you can run it unattended; a human in the loop only via `scripts/hitl-loop.template.sh`.

  [ ] **智能体可运行**：你能在无人值守的情况下运行它；需要人参与时，只能通过 `scripts/hitl-loop.template.sh`。

If you catch yourself reading code to build a theory before this command exists, **stop: jumping straight to a hypothesis is the exact failure this skill prevents.** No red-capable command, no Phase 2.

如果你发现自己在这条命令存在之前就去读代码构建理论，**停下：直接跳到假设，正是本技能要防止的那种失败。**没有能变红的命令，就没有阶段 2。

Phase 2: Reproduce + minimise 阶段 2：复现 + 最小化

Run the loop. Watch it go red as the bug appears.

运行循环。看着它随着 bug 出现而变红。

Confirm:

确认：

- [ ] The loop produces the failure mode the **user** described, not a different failure that happens to be nearby. Wrong bug = wrong fix.

  [ ] 循环产生的是**用户**描述的那种失败模式，而不是碰巧就在附近的另一种失败。bug 错了 = 修复错了。

- [ ] The failure is reproducible across multiple runs (or, for non-deterministic bugs, reproducible at a high enough rate to debug against).

  [ ] 失败在多次运行中都能复现（或者，对非确定性 bug 而言，能以足够高、足以调试的概率复现）。

- [ ] You have captured the exact symptom (error message, wrong output, slow timing) so later phases can verify the fix actually addresses it.

  [ ] 你已捕获确切的症状（错误信息、错误输出、缓慢的耗时），以便后续阶段验证修复确实解决了它。

Minimise 最小化

Once it's red, shrink the repro to the **smallest scenario that still goes red.** Cut inputs, callers, config, data, and steps **one at a time**, re-running the loop after each cut, and keep only what's load-bearing for the failure.

一旦变红，就把复现缩小到**仍会变红的最小场景**。**一次一个**地删减输入、调用方、配置、数据和步骤，每删一次就重新运行循环，只保留对这次失败起承重作用的部分。

Why bother: a minimal repro shrinks the hypothesis space in Phase 3 (fewer moving parts left to suspect) and becomes the clean regression test in Phase 5.

为什么要费这个劲：最小复现能缩小阶段 3 的假设空间（剩下需要怀疑的活动部件更少），并会成为阶段 5 中干净的回归测试。

Done when **every remaining element is load-bearing**: removing any one of them makes the loop go green.

当**剩下的每个元素都起承重作用**时即告完成：去掉其中任何一个，循环都会变绿。

Do not proceed until you have reproduced **and** minimised.

在复现**并且**最小化之前，不要继续往下走。

Phase 3: Hypothesise 阶段 3：提出假设

Generate **3-5 ranked hypotheses** before testing any of them. Single-hypothesis generation anchors on the first plausible idea.

在检验任何一个假设之前，先生成 **3-5 个排好序的假设**。只生成单个假设，会锚定在第一个看似合理的想法上。

Each hypothesis must be **falsifiable**: state the prediction it makes.

每个假设都必须是**可证伪的**：说明它做出的预测。

> Format: "If <X> is the cause, then <changing Y> will make the bug disappear / <changing Z> will make it worse."

> 格式：“如果 <X> 是原因，那么 <改变 Y> 会让 bug 消失 / <改变 Z> 会让它更严重。”

If you cannot state the prediction, the hypothesis is a vibe: discard or sharpen it.

如果你说不出预测，这个假设就只是一种感觉：丢弃它，或把它磨锋利。

Show the ranked list to the user before testing. They often have domain knowledge that re-ranks instantly ("we just deployed a change to #3"), or know hypotheses they've already ruled out. Cheap checkpoint, big time saver. Don't block on it; proceed with your ranking if the user is AFK.

在检验之前，把排好序的列表展示给用户。 他们往往拥有能立即重新排序的领域知识（“我们刚给 #3 部署了一个改动”），或者知道哪些假设他们已经排除了。廉价的检查点，巨大的时间节省。不要因此阻塞；如果用户 AFK，就按你自己的排序继续。

Phase 4: Instrument 阶段 4：插桩

Each probe must map to a specific prediction from Phase 3. **Change one variable at a time.**

每个探针都必须对应阶段 3 中的某个具体预测。**一次只改变一个变量。**

Tool preference:

工具偏好：

1. **Debugger / REPL inspection** if the env supports it. One breakpoint beats ten logs.

   调试器 / REPL 检查，如果环境支持的话。一个断点胜过十条日志。

2. **Targeted logs** at the boundaries that distinguish hypotheses.

   定向日志，放在能区分各个假设的边界处。

3. Never "log everything and grep".

   绝不“把所有东西都打日志然后 grep”。

Tag every debug log with a unique prefix, e.g. `[DEBUG-a4f2]`. Cleanup at the end becomes a single grep. Untagged logs survive; tagged logs die.

给每条调试日志打标签，使用唯一前缀，例如 `[DEBUG-a4f2]`。最后的清理就变成一次 grep。没打标签的日志会留下来；打了标签的日志会被清除。

Perf branch. For performance regressions, logs are usually wrong. Instead: establish a baseline measurement (timing harness, `performance.now()`, profiler, query plan), then bisect. Measure first, fix second.

性能分支。 对于性能回退，日志通常是错误的工具。改为：先建立基线测量（计时装置、`performance.now()`、性能剖析器、查询计划），然后二分。先测量，再修复。

Phase 5: Fix + regression test 阶段 5：修复 + 回归测试

Write the regression test **before the fix**, but only if there is a **correct seam** for it.

在修复之前编写回归测试，但前提是存在一个**正确的接缝**来放它。

A correct seam is one where the test exercises the **real bug pattern** as it occurs at the call site. If the only available seam is too shallow (single-caller test when the bug needs multiple callers, unit test that can't replicate the chain that triggered the bug), a regression test there gives false confidence.

正确的接缝是指，测试能在其中按 bug 在调用点实际发生的样子，演练**真实的 bug 模式**。如果唯一可用的接缝太浅（bug 需要多个调用方，测试却只有单个调用方；单元测试无法复现触发该 bug 的调用链），在那里写回归测试只会带来虚假的信心。

If no correct seam exists, that itself is the finding. Note it. The codebase architecture is preventing the bug from being locked down. Flag this for the next phase.

如果不存在正确的接缝，这本身就是发现。 记下来。代码库的架构正在阻止这个 bug 被锁定。把这一点标记出来，留给下一阶段。

If a correct seam exists:

如果存在正确的接缝：

1. Turn the minimised repro into a failing test at that seam.

   把最小化后的复现变成该接缝处的一个失败测试。

2. Watch it fail. If you forced the red by mutating code or a fixture, `diff` against a pristine copy to prove the mutation landed before you trust it.

   看着它失败。如果你是通过修改代码或 fixture 强行让它变红的，就用 `diff` 与一份原始副本对比，在信任它之前证明修改确实生效了。

3. Apply the fix.

   应用修复。

4. Watch it pass.

   看着它通过。

5. Re-run the Phase 1 feedback loop against the original (un-minimised) scenario.

   针对原始（未最小化的）场景，重新运行阶段 1 的反馈循环。

Phase 6: Cleanup 阶段 6：清理

Required before declaring done:

宣布完成之前必须做到：

- [ ] Original repro no longer reproduces (re-run the Phase 1 loop)

  [ ] 原始复现不再复现（重新运行阶段 1 的循环）

- [ ] Regression test passes (or absence of seam is documented)

  [ ] 回归测试通过（或已记录缺少接缝这一情况）

- [ ] All `[DEBUG-...]` instrumentation removed (`grep` the prefix)

  [ ] 所有 `[DEBUG-...]` 插桩都已移除（`grep` 该前缀）

- [ ] Throwaway prototypes deleted (or moved to a clearly-marked debug location)

  [ ] 一次性原型已删除（或移到明确标记的调试位置）

- [ ] The hypothesis that turned out correct is stated in the commit / PR message, so the next debugger learns

  [ ] 最终被证实正确的那个假设已写进 commit / PR 信息，让下一个调试者能从中学习

CHAPTER 03 · SKILL

Grill With Docs
文档伴随式深度访谈

A relentless interview to sharpen a plan or design, which also creates docs (ADR's and glossary) as we go.

一场毫不留情的访谈，用来打磨计划或设计，并在过程中同步产出文档（ADR 和术语表）。

Source 原文 · v1.3.1 · b0618bc4

Call the Skill tool twice, for "grilling" and "domain-modeling".

调用 Skill 工具两次，分别用于 "grilling" 和 "domain-modeling"。

CHAPTER 04 · SKILL

Triage
分诊

Move issues and external PRs through a state machine of triage roles, categorise, verify, grill if needed, and write agent-ready briefs.

让 issue 和外部 PR 在一个由分诊角色构成的状态机中流转：归类、验证、必要时深度追问，并撰写可直接交给智能体执行的简报。

Source 原文 · v1.3.1 · b0618bc4

Move issues on the project issue tracker through a small state machine of triage roles.

让项目 issue 跟踪器上的 issue 在一个由分诊角色构成的小型状态机中流转。

If this repo treats external pull requests as a request surface (see the issue-tracker config), triage covers them too: **a PR is an issue with attached code**, using the same roles, same states, and same machine, with a few deltas marked "for a PR" below. Resolve a bare `#42` to an issue or PR per the tracker config.

如果这个仓库把外部 pull request 也视为一种需求入口（见 issue 跟踪器配置），那么分诊同样覆盖它们：**PR 就是附带了代码的 issue**，使用相同的角色、相同的状态和相同的状态机，只在下文标注“对于 PR”之处有少量差异。按照跟踪器配置，把单独的 `#42` 解析为 issue 或 PR。

Every comment or issue posted to the issue tracker during triage **must** start with this disclaimer:

分诊期间发布到 issue 跟踪器上的每一条评论或 issue **必须**以这段免责声明开头：

```
> *This was generated by AI during triage.*
```

```
> *This was generated by AI during triage.*
```

Reference docs 参考文档

- AGENT-BRIEF.md: how to write durable agent briefs

  AGENT-BRIEF.md：如何撰写经得起时间考验的智能体简报

- OUT-OF-SCOPE.md: how the `.out-of-scope/` knowledge base works

  OUT-OF-SCOPE.md：`.out-of-scope/` 知识库如何运作

Roles 角色

Two category roles:

两个**类别**角色：

- `bug`: something is broken

  `bug`：某处坏了

- `enhancement`: new feature or improvement

  `enhancement`：新功能或改进

Five state roles:

五个**状态**角色：

- `needs-triage`: maintainer needs to evaluate

  `needs-triage`：需要维护者评估

- `needs-info`: waiting on reporter for more information

  `needs-info`：等待报告者提供更多信息

- `ready-for-agent`: fully specified, ready for an AFK agent

  `ready-for-agent`：已完全明确，可交给 AFK 智能体

- `ready-for-human`: needs human implementation

  `ready-for-human`：需要人工实现

- `wontfix`: will not be actioned

  `wontfix`：不会处理

For a PR, the same states read against the attached code: `ready-for-agent` means a brief is attached and an agent should take the next step on the diff; `ready-for-human` means it's ready for a human to merge.

对于 PR，同样的状态是针对所附代码来解读的：`ready-for-agent` 表示已附上简报，应由智能体在该 diff 上执行下一步；`ready-for-human` 表示已可由人来合并。

Every triaged issue should carry exactly one category role and one state role. If state roles conflict, flag it and ask the maintainer before doing anything else.

每个已分诊的 issue 都应恰好带有一个类别角色和一个状态角色。如果状态角色相互冲突，先指出问题并询问维护者，再做其他任何事。

These are canonical role names. The actual label strings used in the issue tracker may differ. The mapping should have been provided to you. If not, tell the user to run `/setup-matt-pocock-skills`.

这些是规范的角色名称。issue 跟踪器中实际使用的标签字符串可能不同。这份映射本应已提供给你。如果没有，告诉用户运行 `/setup-matt-pocock-skills`。

State transitions: an unlabeled issue normally goes to `needs-triage` first; from there it moves to `needs-info`, `ready-for-agent`, `ready-for-human`, or `wontfix`. `needs-info` returns to `needs-triage` once the reporter replies. The maintainer can override at any time; flag transitions that look unusual and ask before proceeding.

状态流转：未打标签的 issue 通常先进入 `needs-triage`；从那里再转入 `needs-info`、`ready-for-agent`、`ready-for-human` 或 `wontfix`。报告者回复后，`needs-info` 回到 `needs-triage`。维护者可以随时覆盖；对看起来不寻常的流转要指出来，并在继续之前先询问。

Invocation 调用

The maintainer invokes `/triage` and describes what they want in natural language. Interpret the request and act. Examples:

维护者调用 `/triage`，并用自然语言描述想做什么。理解请求并据此行动。示例：

- "Show me anything that needs my attention"

  “给我看看所有需要我关注的东西”

- "Let's look at #42" (issue or PR)

  “我们来看看 #42”（issue 或 PR）

- "Move #42 to ready-for-agent"

  “把 #42 移到 ready-for-agent”

- "What's ready for agents to pick up?"

  “有哪些已经可以交给智能体去领取了？”

Show what needs attention 展示需要关注的内容

Query the issue tracker and present three buckets, oldest first:

查询 issue 跟踪器，按从旧到新的顺序展示三个分组：

1. Unlabeled: never triaged.

   未打标签：从未分诊过。

2. `needs-triage`: evaluation in progress.

   `needs-triage`：评估进行中。

3. `needs-info` with reporter activity since the last triage notes: needs re-evaluation.

   自上次分诊记录以来报告者有新动态的 `needs-info`：需要重新评估。

When PRs are in scope, include external PRs in these buckets and tag each line `[PR]` or `[issue]`. Discovery surfaces only external PRs (the tracker config defines who counts as external), so a collaborator's in-flight PR is not triage work. This filter is discovery-only; an explicitly named PR is always triaged regardless of author.

当 PR 在范围内时，把外部 PR 也纳入这些分组，并给每一行打上 `[PR]` 或 `[issue]` 标记。发现阶段只呈现外部 PR（由跟踪器配置定义谁算外部），因此协作者正在进行中的 PR 不属于分诊工作。这个过滤只作用于发现阶段；被明确点名的 PR，无论作者是谁，都一律分诊。

Show counts and a one-line summary per item. Let the maintainer pick.

展示数量，并为每一项给出一行摘要。让维护者自己挑选。

Triage a specific issue or PR 分诊某个具体的 issue 或 PR

1. **Gather context.** Read the full issue or PR (body, comments, labels, author, dates; for a PR, the diff too). Parse any prior triage notes so you don't re-ask resolved questions. Explore the codebase using the project's domain glossary, respecting ADRs in the area. Run two checks against the codebase: (a) **redundancy**: search for an existing implementation of the requested behavior by domain concept (not just the request's wording), and report where you looked. If found, it's an already-implemented `wontfix` (step 5). (b) **prior rejection**: read `.out-of-scope/*.md` and surface any that resembles this request.

   收集上下文。 完整阅读该 issue 或 PR（正文、评论、标签、作者、日期；如果是 PR，还要看 diff）。解析此前的分诊记录，避免重复询问已解决的问题。借助项目的领域术语表探索代码库，并遵守该区域的 ADR（架构决策记录）。针对代码库做两项检查：(a) **冗余**：按领域概念（而不只是请求的字面措辞）搜索所请求行为是否已有实现，并报告你查找过的位置。如果找到了，就属于“已实现”的 `wontfix`（第 5 步）。(b) **此前已拒绝**：阅读 `.out-of-scope/*.md`，把与本请求相似的条目呈现出来。

2. **Recommend.** Tell the maintainer your category and state recommendation with reasoning, plus a brief codebase summary relevant to the request (including whether it's already implemented). Wait for direction.

   给出建议。 告诉维护者你对类别和状态的建议及理由，再附上与该请求相关的简短代码库概述（包括它是否已经实现）。等待指示。

3. **Verify the claim.** Before any grilling, check that the claim holds up. For a bug, reproduce it from the reporter's steps. For a PR, confirm the diff does what it claims: check it out, run the relevant tests or commands. Report what happened: confirmed (with code path), failed, or insufficient detail (a strong `needs-info` signal). A confirmed verification makes a much stronger agent brief.

   验证说法。 在任何深度追问之前，先确认这个说法站得住脚。对于 bug，按报告者的步骤复现。对于 PR，确认 diff 确实做到了它声称的事：把它检出，运行相关测试或命令。报告结果：已确认（附代码路径）、失败，或细节不足（这是强烈的 `needs-info` 信号）。经过确认的验证会让智能体简报有力得多。

4. Grill (if needed). If the request needs fleshing out, call the Skill tool twice, for "grilling" and "domain-modeling", and grill it into shape a round of questions at a time, sharpening domain terms and updating `GLOSSARY.md`/ADRs inline as decisions land.

   深度追问（如有需要）。 如果请求还需要充实，就调用两次 Skill 工具，分别调用 "grilling" 和 "domain-modeling"，一次一轮问题地把它追问成形，同时打磨领域术语，并在决策落定时就地更新 `GLOSSARY.md`/ADR。

5. Apply the outcome:

   落实结果：

- `ready-for-agent`: post an agent brief comment (AGENT-BRIEF.md).

  `ready-for-agent`：发布一条智能体简报评论（AGENT-BRIEF.md）。

- `ready-for-human`: same structure as an agent brief, but note why it can't be delegated (judgment calls, external access, design decisions, manual testing).

  `ready-for-human`：结构与智能体简报相同，但要说明为什么不能委派出去（需要判断取舍、需要外部访问权限、涉及设计决策、需要人工测试）。

- `needs-info`: post triage notes (template below).

  `needs-info`：发布分诊记录（模板见下文）。

- For `wontfix`, close the issue, with the comment depending on why:

  对于 `wontfix`，关闭该 issue，评论内容取决于原因：

- Already implemented: the change already exists in the codebase. Point to where it lives; do not write to `.out-of-scope/` (that KB is for rejected requests, not built ones).

  已实现：该改动已存在于代码库中。指出它所在的位置；**不要**写入 `.out-of-scope/`（那个知识库是给被拒绝的请求用的，不是给已经做好的）。

- Rejected (bug): give a polite explanation, then close.

  已拒绝（bug）：礼貌地给出解释，然后关闭。

- Rejected (enhancement): write to `.out-of-scope/`, link to it from a comment, then close (OUT-OF-SCOPE.md).

  已拒绝（enhancement）：写入 `.out-of-scope/`，在评论中链接到它，然后关闭（OUT-OF-SCOPE.md）。

- `needs-triage`: apply the role. Optional comment if there's partial progress.

  `needs-triage`：应用该角色。如果已有部分进展，可选择附一条评论。

Quick state override 快速覆盖状态

If the maintainer says "move #42 to ready-for-agent", trust them and apply the role directly. Confirm what you're about to do (role changes, comment, close), then act. Skip grilling. If moving to `ready-for-agent` without a grilling session, ask whether they want to write an agent brief.

如果维护者说“把 #42 移到 ready-for-agent”，就信任他们，直接应用该角色。先确认你将要做什么（角色变更、评论、关闭），然后执行。跳过深度追问。如果在没有深度追问环节的情况下移到 `ready-for-agent`，询问他们是否要写一份智能体简报。

Needs-info template Needs-info 模板

```
## Triage Notes

**What we've established so far:**

- point 1
- point 2

**What we still need from you (@reporter):**

- question 1
- question 2
```


```
## Triage Notes

**What we've established so far:**

- point 1
- point 2

**What we still need from you (@reporter):**

- question 1
- question 2
```


Capture everything resolved during grilling under "established so far" so the work isn't lost. Questions must be specific and actionable, not "please provide more info".

把深度追问中已解决的所有内容都记录在“established so far”下，这样工作成果不会丢失。问题必须具体且可操作，不能是“请提供更多信息”。

Resuming a previous session 恢复之前的会话

If prior triage notes exist on the issue or PR, read them, check whether the reporter has answered any outstanding questions, and present an updated picture before continuing. Don't re-ask resolved questions.

如果该 issue 或 PR 上已有此前的分诊记录，先阅读它们，检查报告者是否回答了尚未解决的问题，并在继续之前呈现一份更新后的全貌。不要重复询问已解决的问题。

REFERENCE 参考文档 · AGENT-BRIEF.md

Writing Agent Briefs 撰写智能体简报

Source 原文

An agent brief is a structured comment posted on a GitHub issue or PR when it moves to `ready-for-agent`. It is the authoritative specification that an AFK agent will work from. The original body and discussion are context: the agent brief is the contract.

智能体简报是一条结构化评论，在 GitHub issue 或 PR 转入 `ready-for-agent` 时发布在其上。它是 AFK 智能体据以工作的权威规范。原始正文和讨论只是上下文：智能体简报才是契约。

The brief states what the agent should do, which stretches to both surfaces: for an issue, that's building the change from nothing; for a PR, it's what's left to do to the existing diff: finish it, close gaps, address review points. Same principles either way; the PR example below shows the difference.

简报说明**智能体应该做什么**，这一点同时适用于两种对象：对于 issue，是从零开始构建改动；对于 PR，是针对现有 diff 还剩下什么要做：把它完成、补上缺口、处理评审意见。两种情况遵循相同的原则；下面的 PR 示例展示了其中的差别。

Principles 原则

Durability over precision 持久性优先于精确性

The issue may sit in `ready-for-agent` for days or weeks. The codebase will change in the meantime. Write the brief so it stays useful even as files are renamed, moved, or refactored.

issue 可能会在 `ready-for-agent` 中停留几天甚至几周。在此期间代码库会发生变化。撰写简报时，要让它在文件被重命名、移动或重构之后依然有用。

- Do describe interfaces, types, and behavioral contracts

  要描述接口、类型和行为契约

- Do name specific types, function signatures, or config shapes that the agent should look for or modify

  要点名智能体应该查找或修改的具体类型、函数签名或配置结构

- Don't reference file paths: they go stale

  不要引用文件路径：它们会过时

- Don't reference line numbers

  不要引用行号

- Don't assume the current implementation structure will remain the same

  不要假定当前的实现结构会保持不变

Behavioral, not procedural 描述行为，而非步骤

Describe what the system should do, not how to implement it. The agent will explore the codebase fresh and make its own implementation decisions.

描述系统应该做**什么**，而不是**如何**实现。智能体会重新探索代码库，并自行做出实现决策。

- Good: "The `SkillConfig` type should accept an optional `schedule` field of type `CronExpression`"

  好：“`SkillConfig` 类型应接受一个可选的 `schedule` 字段，类型为 `CronExpression`”

- Bad: "Open src/types/skill.ts and add a schedule field on line 42"

  差：“打开 src/types/skill.ts，在第 42 行添加一个 schedule 字段”

- Good: "When a user runs `/triage` with no arguments, they should see a summary of issues needing attention"

  好：“当用户不带参数运行 `/triage` 时，应看到一份需要关注的 issue 摘要”

- Bad: "Add a switch statement in the main handler function"

  差：“在主处理函数里添加一个 switch 语句”

Complete acceptance criteria 完整的验收标准

The agent needs to know when it's done. Every agent brief must have concrete, testable acceptance criteria. Each criterion should be independently verifiable.

智能体需要知道什么时候算完成。每份智能体简报都必须有具体、可测试的验收标准。每条标准都应能独立验证。

- Good: "Running `gh issue list --label needs-triage` returns issues that have been through initial classification"

  好：“运行 `gh issue list --label needs-triage` 会返回已经过初步分类的 issue”

- Bad: "Triage should work correctly"

  差：“分诊应该正常工作”

Explicit scope boundaries 明确的范围边界

State what is out of scope. This prevents the agent from gold-plating or making assumptions about adjacent features.

写明哪些不在范围内。这可以防止智能体画蛇添足，或对相邻功能做出臆测。

Template 模板

```
## Agent Brief

**Category:** bug / enhancement
**Summary:** one-line description of what needs to happen

**Current behavior:**
Describe what happens now. For bugs, this is the broken behavior.
For enhancements, this is the status quo the feature builds on.

**Desired behavior:**
Describe what should happen after the agent's work is complete.
Be specific about edge cases and error conditions.

**Key interfaces:**
- `TypeName`: what needs to change and why
- `functionName()` return type: what it currently returns vs what it should return
- Config shape: any new configuration options needed

**Acceptance criteria:**
- [ ] Specific, testable criterion 1
- [ ] Specific, testable criterion 2
- [ ] Specific, testable criterion 3

**Out of scope:**
- Thing that should NOT be changed or addressed in this issue
- Adjacent feature that might seem related but is separate
```


```
## Agent Brief

**Category:** bug / enhancement
**Summary:** one-line description of what needs to happen

**Current behavior:**
Describe what happens now. For bugs, this is the broken behavior.
For enhancements, this is the status quo the feature builds on.

**Desired behavior:**
Describe what should happen after the agent's work is complete.
Be specific about edge cases and error conditions.

**Key interfaces:**
- `TypeName`: what needs to change and why
- `functionName()` return type: what it currently returns vs what it should return
- Config shape: any new configuration options needed

**Acceptance criteria:**
- [ ] Specific, testable criterion 1
- [ ] Specific, testable criterion 2
- [ ] Specific, testable criterion 3

**Out of scope:**
- Thing that should NOT be changed or addressed in this issue
- Adjacent feature that might seem related but is separate
```

Examples 示例

Good agent brief (bug) 好的智能体简报（bug）

```
## Agent Brief

**Category:** bug
**Summary:** Skill description truncation drops mid-word, producing broken output

**Current behavior:**
When a skill description exceeds 1024 characters, it is truncated at exactly
1024 characters regardless of word boundaries. This produces descriptions
that end mid-word (e.g. "Use when the user wants to confi").

**Desired behavior:**
Truncation should break at the last word boundary before 1024 characters
and append "..." to indicate truncation.

**Key interfaces:**
- The `SkillMetadata` type's `description` field: no type change needed,
  but the validation/processing logic that populates it needs to respect
  word boundaries
- Any function that reads SKILL.md frontmatter and extracts the description

**Acceptance criteria:**
- [ ] Descriptions under 1024 chars are unchanged
- [ ] Descriptions over 1024 chars are truncated at the last word boundary
      before 1024 chars
- [ ] Truncated descriptions end with "..."
- [ ] The total length including "..." does not exceed 1024 chars

**Out of scope:**
- Changing the 1024 char limit itself
- Multi-line description support
```

```
## Agent Brief

**Category:** bug
**Summary:** Skill description truncation drops mid-word, producing broken output

**Current behavior:**
When a skill description exceeds 1024 characters, it is truncated at exactly
1024 characters regardless of word boundaries. This produces descriptions
that end mid-word (e.g. "Use when the user wants to confi").

**Desired behavior:**
Truncation should break at the last word boundary before 1024 characters
and append "..." to indicate truncation.

**Key interfaces:**
- The `SkillMetadata` type's `description` field: no type change needed,
  but the validation/processing logic that populates it needs to respect
  word boundaries
- Any function that reads SKILL.md frontmatter and extracts the description

**Acceptance criteria:**
- [ ] Descriptions under 1024 chars are unchanged
- [ ] Descriptions over 1024 chars are truncated at the last word boundary
      before 1024 chars
- [ ] Truncated descriptions end with "..."
- [ ] The total length including "..." does not exceed 1024 chars

**Out of scope:**
- Changing the 1024 char limit itself
```

```
- Multi-line description support
```

Good agent brief (enhancement) 好的智能体简报（enhancement）

```
## Agent Brief

**Category:** enhancement
**Summary:** Add `.out-of-scope/` directory support for tracking rejected feature requests

**Current behavior:**
When a feature request is rejected, the issue is closed with a `wontfix` label
and a comment. There is no persistent record of the decision or reasoning.
Future similar requests require the maintainer to recall or search for the
prior discussion.

**Desired behavior:**
Rejected feature requests should be documented in `.out-of-scope/<concept>.md`
files that capture the decision, reasoning, and links to all issues that
requested the feature. When triaging new issues, these files should be
checked for matches.

**Key interfaces:**
- Markdown file format in `.out-of-scope/`: each file should have a
  `# Concept Name` heading, a `**Decision:**` line, a `**Reason:**` line,
  and a `**Prior requests:**` list with issue links
- The triage workflow should read all `.out-of-scope/*.md` files early
  and match incoming issues against them by concept similarity

**Acceptance criteria:**
- [ ] Closing a feature as wontfix creates/updates a file in `.out-of-scope/`
- [ ] The file includes the decision, reasoning, and link to the closed issue
- [ ] If a matching `.out-of-scope/` file already exists, the new issue is
      appended to its "Prior requests" list rather than creating a duplicate
- [ ] During triage, existing `.out-of-scope/` files are checked and surfaced
      when a new issue matches a prior rejection

**Out of scope:**
- Automated matching (human confirms the match)
- Reopening previously rejected features
- Bug reports (only enhancement rejections go to `.out-of-scope/`)
```


```
## Agent Brief

**Category:** enhancement
**Summary:** Add `.out-of-scope/` directory support for tracking rejected feature requests

**Current behavior:**
When a feature request is rejected, the issue is closed with a `wontfix` label
and a comment. There is no persistent record of the decision or reasoning.
Future similar requests require the maintainer to recall or search for the
prior discussion.

**Desired behavior:**
Rejected feature requests should be documented in `.out-of-scope/<concept>.md`
files that capture the decision, reasoning, and links to all issues that
requested the feature. When triaging new issues, these files should be
checked for matches.

**Key interfaces:**
- Markdown file format in `.out-of-scope/`: each file should have a
  `# Concept Name` heading, a `**Decision:**` line, a `**Reason:**` line,
  and a `**Prior requests:**` list with issue links
- The triage workflow should read all `.out-of-scope/*.md` files early
  and match incoming issues against them by concept similarity

**Acceptance criteria:**
- [ ] Closing a feature as wontfix creates/updates a file in `.out-of-scope/`
- [ ] The file includes the decision, reasoning, and link to the closed issue
```

```
- [ ] If a matching `.out-of-scope/` file already exists, the new issue is
      appended to its "Prior requests" list rather than creating a duplicate
- [ ] During triage, existing `.out-of-scope/` files are checked and surfaced
      when a new issue matches a prior rejection

**Out of scope:**
- Automated matching (human confirms the match)
- Reopening previously rejected features
- Bug reports (only enhancement rejections go to `.out-of-scope/`)
```

Good agent brief (PR) 好的智能体简报（PR）

For a PR, "Current behavior" describes the state of the diff, and the brief asks the agent to finish or fix it rather than build from scratch.

对于 PR，“Current behavior”描述的是该 diff 的现状，简报要求智能体把它完成或修好，而不是从零开始构建。

```
## Agent Brief

**Category:** enhancement
**Summary:** Finish the contributor's `--json` output flag for `triage list`

**Current behavior:**
The PR adds a `--json` flag that serializes the issue list to JSON. The happy
path works and the diff matches the project's command structure. Two gaps
remain: errors are still printed as human text (not JSON), and the new flag has
no test coverage.

**Desired behavior:**
With `--json`, all output (including errors) is well-formed JSON on stdout,
and the command's exit codes are unchanged. The existing human-readable output
is untouched when the flag is absent.

**Key interfaces:**
- The command's error path should emit `{ "error": string }` under `--json`
  instead of the plain-text error
- Reuse the existing serializer the PR already added; don't introduce a second

**Acceptance criteria:**
- [ ] `triage list --json` emits valid JSON for both success and error cases
- [ ] Exit codes match the non-JSON command
- [ ] A test covers the `--json` success output and one error case
- [ ] Default (non-JSON) output is byte-for-byte unchanged

**Out of scope:**
- Adding `--json` to any other command
- Changing the JSON shape of the success payload the PR already defined
```


```
## Agent Brief

**Category:** enhancement
**Summary:** Finish the contributor's `--json` output flag for `triage list`

**Current behavior:**
The PR adds a `--json` flag that serializes the issue list to JSON. The happy
path works and the diff matches the project's command structure. Two gaps
remain: errors are still printed as human text (not JSON), and the new flag has
no test coverage.

**Desired behavior:**
With `--json`, all output (including errors) is well-formed JSON on stdout,
and the command's exit codes are unchanged. The existing human-readable output
is untouched when the flag is absent.

**Key interfaces:**
- The command's error path should emit `{ "error": string }` under `--json`
  instead of the plain-text error
- Reuse the existing serializer the PR already added; don't introduce a second

**Acceptance criteria:**
- [ ] `triage list --json` emits valid JSON for both success and error cases
- [ ] Exit codes match the non-JSON command
- [ ] A test covers the `--json` success output and one error case
- [ ] Default (non-JSON) output is byte-for-byte unchanged

**Out of scope:**
- Adding `--json` to any other command
- Changing the JSON shape of the success payload the PR already defined
```

Bad agent brief 差的智能体简报

```
## Agent Brief

**Summary:** Fix the triage bug

**What to do:**
The triage thing is broken. Look at the main file and fix it.
The function around line 150 has the issue.

**Files to change:**
- src/triage/handler.ts (line 150)
- src/types.ts (line 42)
```


```
## Agent Brief

**Summary:** Fix the triage bug

**What to do:**
The triage thing is broken. Look at the main file and fix it.
The function around line 150 has the issue.

**Files to change:**
- src/triage/handler.ts (line 150)
- src/types.ts (line 42)
```


This is bad because:

它之所以差，是因为：

- No category

  没有类别

- Vague description ("the triage thing is broken")

  描述含糊（“the triage thing is broken”）

- References file paths and line numbers that will go stale

  引用了会过时的文件路径和行号

- No acceptance criteria

  没有验收标准

- No scope boundaries

  没有范围边界

- No description of current vs desired behavior

  没有描述当前行为与期望行为

REFERENCE 参考文档 · OUT-OF-SCOPE.md

Out-of-Scope Knowledge Base 范围外知识库

Source 原文

The `.out-of-scope/` directory in a repo stores persistent records of rejected feature requests. It serves two purposes:

仓库中的 `.out-of-scope/` 目录用于持久保存被拒绝的功能请求记录。它有两个用途：

1. **Institutional memory**: why a feature was rejected, so the reasoning isn't lost when the issue is closed

   机构记忆：记录某个功能为什么被拒绝，这样 issue 关闭后理由也不会丢失

2. **Deduplication**: when a new issue comes in that matches a prior rejection, the skill can surface the previous decision instead of re-litigating it

   去重：当新进来的 issue 与此前的某次拒绝相匹配时，该技能可以呈现之前的决定，而不是重新争论一遍

Directory structure 目录结构

```
.out-of-scope/
├── dark-mode.md
├── plugin-system.md
└── graphql-api.md
```


```
.out-of-scope/
├── dark-mode.md
├── plugin-system.md
└── graphql-api.md
```


One file per **concept**, not per issue. Multiple issues requesting the same thing are grouped under one file.

每个**概念**一个文件，而不是每个 issue 一个文件。请求同一件事的多个 issue 归并到同一个文件下。

File format 文件格式

The file should be written in a relaxed, readable style, more like a short design document than a database entry. Use paragraphs, code samples, and examples to make the reasoning clear and useful to someone encountering it for the first time.

文件应以轻松、易读的风格撰写，更像一份简短的设计文档，而不是一条数据库记录。用段落、代码示例和例子把理由讲清楚，让第一次接触它的人也能看懂、用得上。

```
# Dark Mode

This project does not support dark mode or user-facing theming.

## Why this is out of scope

The rendering pipeline assumes a single color palette defined in
`ThemeConfig`. Supporting multiple themes would require:

- A theme context provider wrapping the entire component tree
- Per-component theme-aware style resolution
- A persistence layer for user theme preferences

This is a significant architectural change that doesn't align with the
project's focus on content authoring. Theming is a concern for downstream
consumers who embed or redistribute the output.
```


```
# Dark Mode

This project does not support dark mode or user-facing theming.

## Why this is out of scope

The rendering pipeline assumes a single color palette defined in
`ThemeConfig`. Supporting multiple themes would require:

- A theme context provider wrapping the entire component tree
- Per-component theme-aware style resolution
- A persistence layer for user theme preferences

This is a significant architectural change that doesn't align with the
project's focus on content authoring. Theming is a concern for downstream
consumers who embed or redistribute the output.
```


// The current ThemeConfig interface is not designed for runtime switching: interface ThemeConfig { colors: ColorPalette; // single palette, resolved at build time fonts: FontStack; }

// The current ThemeConfig interface is not designed for runtime switching: interface ThemeConfig { colors: ColorPalette; // single palette, resolved at build time fonts: FontStack; }

```
## Prior requests

- #42: "Add dark mode support"
- #87: "Night theme for accessibility"
- #134: "Dark theme option"
```


```
## Prior requests

- #42: "Add dark mode support"
- #87: "Night theme for accessibility"
- #134: "Dark theme option"
```

Naming the file 文件命名

Use a short, descriptive kebab-case name for the concept: `dark-mode.md`, `plugin-system.md`, `graphql-api.md`. The name should be recognizable enough that someone browsing the directory understands what was rejected without opening the file.

为概念取一个简短、有描述性的 kebab-case 名称：`dark-mode.md`、`plugin-system.md`、`graphql-api.md`。名称应足够好认，让浏览该目录的人不打开文件就能明白被拒绝的是什么。

Writing the reason 撰写理由

The reason should be substantive: not "we don't want this" but why. Good reasons reference:

理由应当有实质内容：不是“我们不想要这个”，而是为什么不要。好的理由会引用：

- Project scope or philosophy ("This project focuses on X; theming is a downstream concern")

  项目范围或理念（“本项目专注于 X；主题化是下游关心的事”）
- Technical constraints ("Supporting this would require Y, which conflicts with our Z architecture")

  技术约束（“支持这个需要 Y，这与我们的 Z 架构相冲突”）
- Strategic decisions ("We chose to use A instead of B because...")

  战略决策（“我们选择用 A 而不是 B，因为……”）

The reason should be durable. Avoid referencing temporary circumstances ("we're too busy right now"); those aren't real rejections, they're deferrals.

理由应当经得起时间考验。避免引用临时性的情况（“我们现在太忙了”）；那些不是真正的拒绝，而是推迟。

When to check `.out-of-scope/` 何时检查 `.out-of-scope/`

During triage (Step 1: Gather context), read all files in `.out-of-scope/`. When evaluating a new issue:

在分诊期间（第 1 步：收集上下文），阅读 `.out-of-scope/` 中的所有文件。评估新 issue 时：

- Check if the request matches an existing out-of-scope concept

  检查该请求是否与某个已有的范围外概念相匹配
- Matching is by concept similarity, not keyword: "night theme" matches `dark-mode.md`

  按概念相似度匹配，而非关键词：“night theme”匹配 `dark-mode.md`
- If there's a match, surface it to the maintainer: "This is similar to `.out-of-scope/dark-mode.md`. We rejected this before because [reason]. Do you still feel the same way?"

  如果有匹配，把它呈现给维护者：“这与 `.out-of-scope/dark-mode.md` 相似。我们之前拒绝它是因为 [理由]。你现在还这么看吗？”

The maintainer may:

维护者可能会：

- Confirm: the new issue gets added to the existing file's "Prior requests" list, then closed

  确认：新 issue 被添加到已有文件的“Prior requests”列表中，然后关闭

- Reconsider: the out-of-scope file gets deleted or updated, and the issue proceeds through normal triage

  重新考虑：删除或更新该范围外文件，issue 进入常规分诊流程

- Disagree: the issues are related but distinct, proceed with normal triage

  不同意：这些 issue 相关但并不相同，按常规分诊继续

When to write to `.out-of-scope/` 何时写入 `.out-of-scope/`

Only when an **enhancement** (not a bug) is rejected as `wontfix`. This applies to enhancement PRs exactly as it does to issues: a rejected PR is recorded here so the same request doesn't return as fresh code.

只有当一个 **enhancement**（而非 bug）以 `wontfix` 被拒绝时才写入。这对 enhancement 类 PR 同样适用，和 issue 完全一样：被拒绝的 PR 也记录在这里，免得同样的请求以新代码的形式卷土重来。

Do **not** write here when something is closed as `wontfix` because it's **already implemented**. That's a built feature, not a rejected one; recording it would poison the dedup checks with false rejections. Instead, the closing comment points to where the feature already lives.

当某项因**已经实现**而以 `wontfix` 关闭时，**不要**写到这里。那是已经做好的功能，不是被拒绝的功能；记录它会用虚假的拒绝污染去重检查。应当改为在关闭评论中指出该功能已经在哪里实现。

The flow:

流程：

1. Maintainer decides a feature request is out of scope

   维护者判定某个功能请求超出范围

2. Check if a matching `.out-of-scope/` file already exists

   检查是否已存在匹配的 `.out-of-scope/` 文件

3. If yes: append the new issue to the "Prior requests" list

   如果有：把新 issue 追加到“Prior requests”列表中

4. If no: create a new file with the concept name, decision, reason, and first prior request

   如果没有：新建一个文件，写明概念名称、决定、理由以及第一条历史请求

5. Post a comment on the issue explaining the decision and mentioning the `.out-of-scope/` file

   在 issue 上发布评论，解释该决定并提及对应的 `.out-of-scope/` 文件

6. Close the issue with the `wontfix` label

   以 `wontfix` 标签关闭该 issue

Updating or removing out-of-scope files 更新或删除范围外文件

If the maintainer changes their mind about a previously rejected concept:

如果维护者对此前拒绝的某个概念改变了主意：

- Delete the `.out-of-scope/` file

  删除该 `.out-of-scope/` 文件

- The skill does not need to reopen old issues; they're historical records

  该技能不需要重新打开旧 issue；它们是历史记录

- The new issue that triggered the reconsideration proceeds through normal triage

  引发这次重新考虑的新 issue 进入常规分诊流程

CHAPTER 05 · SKILL

Improve Codebase Architecture 改进代码库架构

Scan a codebase for deepening opportunities, present them as a visual HTML report, then grill through whichever one you pick.

扫描代码库，寻找加深的机会，以可视化 HTML 报告呈现，然后针对你选中的那一项进行深度追问。

Source 原文 · v1.3.1 · b0618bc4

Surface architectural friction and propose **deepening opportunities**: refactors that turn shallow modules into deep ones. The aim is testability and AI-navigability.

找出架构上的摩擦点，并提出**加深机会**：把浅模块变成深模块的重构。目标是可测试性，以及让 AI 易于在代码中导航。

This command is _informed_ by the project's domain model and built on a shared design vocabulary:

本命令以项目的领域模型为依据，并建立在一套共享的设计词汇之上：

- Call the Skill tool with "codebase-design" for the architecture vocabulary (**module**, **interface**, **depth**, **seam**, **adapter**, **leverage**, **locality**) and its principles (the deletion test, "the interface is the test surface", "one adapter = hypothetical seam, two = real"). Use these terms exactly in every suggestion, and don't drift into "component," "service," "API," or "boundary."

  用 "codebase-design" 调用 Skill 工具，获取架构词汇（**模块**（module）、**接口**（interface）、**深度**（depth）、**接缝**（seam）、**适配器**（adapter）、**杠杆**（leverage）、**局部性**（locality））及其原则（删除测试、“接口即测试面”、“一个适配器 = 假想的接缝，两个 = 真实的接缝”）。每条建议都要严格使用这些术语，不要滑向“组件”“服务”“API”或“边界”。

- The domain language in `GLOSSARY.md` gives names to good seams; ADRs in `docs/adr/` record decisions this command should not re-litigate.

  `GLOSSARY.md` 中的领域语言为好的接缝提供名称；`docs/adr/` 中的 ADR（架构决策记录）记录了本命令不应重新争论的决策。

Process 流程

1. Explore 1. 探索

Scope before you scan: YAGNI. Deepening a module pays off by making future changes to it easier, so put extra weight on the parts of the codebase that have recently changed. Decide where to look before you look:

先划范围再扫描：YAGNI。 加深一个模块的回报在于让它未来的改动更容易，所以要格外看重代码库中最近发生过变化的部分。动手查看之前，先决定去哪里看：

- If the user named a direction (a module, a subsystem, a pain point), take it, and skip the inference below.

  如果用户指定了方向（某个模块、某个子系统、某个痛点），就照着来，并跳过下面的推断步骤。

- Otherwise, walk back a good stretch of the commit history (`git log --oneline`) to find the codebase's hot spots, the files and areas that keep coming up, and let those paths pull your attention first. If the changes are scattered with no clear hot spot, widen the net.

  否则，往回翻一大段提交历史（`git log --oneline`），找出代码库的热点--那些反复出现的文件和区域--让这些路径优先吸引你的注意力。如果改动分散、没有明显热点，就扩大搜索范围。

Read the project's domain glossary (`GLOSSARY.md`) and any ADRs in the area you're touching first.

先阅读项目的领域术语表（`GLOSSARY.md`），以及你要触及的区域里的所有 ADR。

Then spawn a sub-agent to walk the codebase. Don't follow rigid heuristics; explore organically and note where you experience friction:

然后派出一个子智能体去遍历代码库。不要遵循僵化的启发式规则；有机地探索，并记下你在哪里感到摩擦：

- Where does understanding one concept require bouncing between many small modules?

  理解某一个概念时，需要在许多小模块之间来回跳转的地方在哪里？

- Where are modules **shallow**, with an interface nearly as complex as the implementation?

  哪些模块是**浅**的，接口几乎和实现一样复杂？

- Where have pure functions been extracted just for testability, but the real bugs hide in how they're called (no **locality**)?

  哪里为了可测试性把纯函数抽了出来，但真正的 bug 却藏在它们被调用的方式里（缺乏**局部性**）？

- Where do tightly-coupled modules leak across their seams?

  哪些紧耦合的模块跨越接缝发生了泄漏？

- Which parts of the codebase are untested, or hard to test through their current interface?

  代码库的哪些部分没有测试，或者很难通过当前接口进行测试？

Apply the **deletion test** to anything you suspect is shallow: would deleting it concentrate complexity, or just move it? A "yes, concentrates" is the signal you want.

对任何你怀疑是浅模块的东西应用**删除测试**：删除它会让复杂度集中起来，还是只是把复杂度挪到别处？“是的，会集中”就是你要找的信号。

2. Present candidates as an HTML report 2. 以 HTML 报告呈现候选项

Write a self-contained HTML file to the OS temp directory so nothing lands in the repo. Resolve the temp dir from `$TMPDIR`, falling back to `/tmp` (or `%TEMP%` on Windows), and write to `<tmpdir>/architecture-review-<timestamp>.html` so each run gets a fresh file. Open it for the user (`xdg-open <path>` on Linux, `open <path>` on macOS, `start <path>` on Windows) and tell them the absolute path.

把一个自包含的 HTML 文件写到操作系统的临时目录，这样不会有任何东西落进仓库。从 `$TMPDIR` 解析临时目录，回退到 `/tmp`（Windows 上为 `%TEMP%`），并写入 `<tmpdir>/architecture-review-<timestamp>.html`，让每次运行都得到一个新文件。为用户打开它（Linux 上用 `xdg-open <path>`，macOS 上用 `open <path>`，Windows 上用 `start <path>`），并告诉用户绝对路径。

The report uses Tailwind via CDN for layout and styling, and Mermaid via CDN for diagrams where a graph/flow/sequence reliably communicates the structure. Mix Mermaid with hand-crafted CSS/SVG visuals: use Mermaid when relationships are graph-shaped (call graphs, dependencies, sequences), and hand-built divs/SVG when you want something more editorial (mass diagrams, cross-sections, collapse animations). Each candidate gets a before/after visualisation. Be visual.

报告使用**通过 CDN 引入的 Tailwind** 做布局和样式，在图/流程/时序能可靠传达结构的地方使用**通过 CDN 引入的 Mermaid** 绘图。把 Mermaid 与手工打造的 CSS/SVG 视觉效果混合使用：关系呈图状（调用图、依赖、时序）时用 Mermaid；想要更有编辑设计感的效果（体量图、横截面、折叠动画）时用手工搭建的 div/SVG。每个候选项都要有一张**前后对比可视化图**。要视觉化。

For each candidate, render a card with:

为每个候选项渲染一张卡片，包含：

- Files: which files/modules are involved

  文件：涉及哪些文件/模块

- Problem: why the current architecture is causing friction

  问题：当前架构为什么造成摩擦

- Solution: plain English description of what would change

  方案：用平实的语言描述会改变什么

- Benefits: explained in terms of locality and leverage, and how tests would improve

  收益：从局部性和杠杆的角度解释，以及测试会如何改善

- Before / After diagram: side-by-side, custom-drawn, illustrating the shallowness and the deepening

  前后对比图：并排、定制绘制，展示浅的现状与加深后的样子

- Recommendation strength: one of `Strong`, `Worth exploring`, `Speculative`, rendered as a badge

  推荐强度：`Strong`、`Worth exploring`、`Speculative` 三者之一，渲染为徽章

End the report with a Top recommendation section: which candidate you'd tackle first and why.

报告以**首要推荐**一节结尾：你会先处理哪个候选项，以及为什么。

Use GLOSSARY.md vocabulary for the domain, and the /codebase-design vocabulary for the architecture. If GLOSSARY.md defines "Order," talk about "the Order intake module," not "the FooBarHandler," and not "the Order service."

领域方面使用 GLOSSARY.md 的词汇，架构方面使用 /codebase-design 的词汇。 如果 GLOSSARY.md 定义了“Order”，就说 “Order 接收模块”，而不是 “FooBarHandler”，也不是 “Order 服务”。

ADR conflicts: if a candidate contradicts an existing ADR, only surface it when the friction is real enough to warrant revisiting the ADR. Mark it clearly in the card (e.g. a warning callout: _"contradicts ADR-0007, but worth reopening because…"_). Don't list every theoretical refactor an ADR forbids.

ADR 冲突：如果某个候选项与现有 ADR 相矛盾，只有当摩擦真实到值得重新审视该 ADR 时才提出它。在卡片中清楚标注（例如一个警告提示框：“与 ADR-0007 相矛盾，但值得重新讨论，因为……”）。不要罗列每一个被 ADR 禁止的理论上的重构。

See HTML-REPORT.md for the full HTML scaffold, diagram patterns, and styling guidance.

完整的 HTML 骨架、图示模式和样式指导，参见 HTML-REPORT.md。

Do NOT propose interfaces yet. After the file is written, ask the user: "Which of these would you like to explore?"

暂时不要提出接口设计。文件写好后，问用户：“你想深入探讨其中哪一个？”

3. Grilling loop 3. 深度追问循环

Once the user picks a candidate, call the Skill tool with "grilling" to walk the decision tree with them: constraints, dependencies, the shape of the deepened module, what sits behind the seam, what tests survive.

用户选定一个候选项后，用 "grilling" 调用 Skill 工具，与用户一起走一遍决策树：约束、依赖、加深后模块的形态、接缝背后放什么、哪些测试能保留下来。

Side effects happen inline as decisions crystallize; call the Skill tool with "domain-modeling" to keep the domain model current as you go:

随着决策逐渐明确，副作用就地发生；用 "domain-modeling" 调用 Skill 工具，在推进过程中保持领域模型的时效：

- Naming a deepened module after a concept not in GLOSSARY.md? Add the term to GLOSSARY.md. Create the file lazily if it doesn't exist.

  要用一个不在 GLOSSARY.md 中的概念来命名加深后的模块？ 把这个术语加到 GLOSSARY.md。如果文件不存在，就在需要时再创建。

- Sharpening a fuzzy term during the conversation? Update GLOSSARY.md right there.

  在对话中把一个模糊的术语讲清楚了？ 当场更新 GLOSSARY.md。

- User rejects the candidate with a load-bearing reason? Offer an ADR, framed as: _"Want me to record this as an ADR so future architecture reviews don't re-suggest it?"_ Only offer when the reason would actually be needed by a future explorer to avoid re-suggesting the same thing; skip ephemeral reasons ("not worth it right now") and self-evident ones.

  用户以一个关键性理由否决了候选项？ 提议写一份 ADR，措辞是：“要不要我把这个记录为 ADR，免得以后的架构评审再次提出它？” 只有当未来的探索者确实需要这个理由来避免重复提出同一建议时才提议；跳过临时性理由（“现在不值得做”）和不言自明的理由。

- Want to explore alternative interfaces for the deepened module? Call the Skill tool with " codebase-design" and use its design-it-twice parallel sub-agent pattern.

  想为加深后的模块探索不同的接口方案？ 用 "codebase-design" 调用 Skill 工具，使用其中的 design-it-twice 并行子智能体模式。

REFERENCE 参考文档 · HTML-REPORT.md

HTML Report Format HTML 报告格式

Source 原文

The architectural review is rendered as a single self-contained HTML file in the OS temp directory. Tailwind and Mermaid both come from CDNs. Mermaid handles graph-shaped diagrams reliably; hand-built divs and inline SVG handle the more editorial visuals (mass diagrams, cross-sections). Mix the two: don't lean on Mermaid for everything, it'll start to look generic.

架构评审渲染为操作系统临时目录中的单个自包含 HTML 文件。Tailwind 和 Mermaid 都来自 CDN。Mermaid 可靠地处理图状的图示；手工搭建的 div 和内联 SVG 处理更有编辑设计感的视觉效果（体量图、横截面）。两者混用：不要什么都靠 Mermaid，否则看起来会千篇一律。

Scaffold 骨架

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>Architecture review for {{repo name}}</title>
    <script src="https://cdn.tailwindcss.com"></script>
    <script type="module">
      import mermaid from "https://cdn.jsdelivr.net/npm/mermaid@11/dist/mermaid.esm.min.mjs";
      mermaid.initialize({ startOnLoad: true, theme: "neutral", securityLevel: "loose" });
    </script>
    <style>
      /* small custom layer for things Tailwind doesn't cover cleanly:
         dashed seam lines, hand-drawn-feeling arrow heads, etc. */
      .seam { stroke-dasharray: 4 4; }
      .leak { stroke: #dc2626; }
      .deep { background: linear-gradient(135deg, #0f172a, #1e293b); }
    </style>
  </head>
  <body class="bg-stone-50 text-slate-900 font-sans">
    <main class="max-w-5xl mx-auto px-6 py-12 space-y-12">
      <header>...</header>
      <section id="candidates" class="space-y-10">...</section>
      <section id="top-recommendation">...</section>
    </main>
  </body>
</html>
```


```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>Architecture review for {{repo name}}</title>
    <script src="https://cdn.tailwindcss.com"></script>
    <script type="module">
      import mermaid from "https://cdn.jsdelivr.net/npm/mermaid@11/dist/mermaid.esm.min.mjs";
      mermaid.initialize({ startOnLoad: true, theme: "neutral", securityLevel: "loose" });
    </script>
    <style>
      /* small custom layer for things Tailwind doesn't cover cleanly:
         dashed seam lines, hand-drawn-feeling arrow heads, etc. */
      .seam { stroke-dasharray: 4 4; }
      .leak { stroke: #dc2626; }
      .deep { background: linear-gradient(135deg, #0f172a, #1e293b); }
    </style>
  </head>
  <body class="bg-stone-50 text-slate-900 font-sans">
    <main class="max-w-5xl mx-auto px-6 py-12 space-y-12">
      <header>...</header>
      <section id="candidates" class="space-y-10">...</section>
      <section id="top-recommendation">...</section>
    </main>
  </body>
</html>
```

Header 页头

Repo name, date, and a compact legend: solid box = module, dashed line = seam, red arrow = leakage, thick dark box = deep module. No introduction paragraph. Straight into the candidates.

仓库名、日期，以及一个紧凑的图例：实线框 = 模块，虚线 = 接缝，红色箭头 = 泄漏，粗重的深色框 = 深模块。不要引言段落。直接进入候选项。

Candidate card 候选项卡片

The diagrams carry the weight. Prose is sparse, plain, and uses the glossary terms (from the `/codebase-design` skill) without ceremony.

图示承担主要分量。文字要少、要平实，并且不加修饰地使用术语表中的术语（来自 `/codebase-design` 技能）。

Each candidate is one `<article>`:

每个候选项是一个 `<article>`：

- **Title**: short, names the deepening (e.g. "Collapse the Order intake pipeline").

  标题：简短，点出这次加深（例如 “Collapse the Order intake pipeline”）。

- **Badge row**: recommendation strength (`Strong` = emerald, `Worth exploring` = amber, `Speculative` = slate), plus a tag for the dependency category (`in-process`, `local-substitutable`, `ports & adapters`, `mock`).

  徽章行：推荐强度（`Strong` = emerald，`Worth exploring` = amber，`Speculative` = slate），外加一个表示依赖类别的标签（`in-process`、`local-substitutable`、`ports & adapters`、`mock`）。

- **Files**: monospaced list, `font-mono text-sm`.

  文件：等宽列表，`font-mono text-sm`。

- **Before / After diagram**: the centrepiece. Two columns, side by side. See patterns below.

  前后对比图：核心部分。两栏，并排。参见下面的模式。

- **Problem**: one sentence. What hurts.

  问题：一句话。痛在哪里。

- **Solution**: one sentence. What changes.

  方案：一句话。改变什么。

- **Wins**: bullets, ≤6 words each. e.g. "Tests hit one interface", "Pricing logic stops leaking", "Delete 4 shallow wrappers".

  收益：要点列表，每条 ≤6 个词。例如 “Tests hit one interface” “Pricing logic stops leaking” “Delete 4 shallow wrappers”。

- **ADR callout** (if applicable): one line in an amber-tinted box.

  ADR 提示框（如适用）：琥珀色底框里的一行字。

No paragraphs of explanation. If the diagram needs a paragraph to be understood, redraw the diagram.

不要写解释性段落。如果图示需要一段文字才能看懂，就重画图示。

Diagram patterns 图示模式

Pick the pattern that fits the candidate. Mix them. Don't make every diagram look the same. Variety is part of the point.

选择适合该候选项的模式。混着用。不要让每张图都长得一样。多样性本身就是目的之一。

Mermaid graph (the workhorse for dependencies / call flow) Mermaid 图（依赖 / 调用流程的主力）

Use a Mermaid `flowchart` or `graph` when the point is "X calls Y calls Z, and look at the mess." Wrap it in a Tailwind-styled card so it doesn't feel parachuted in. Style with classDef to colour leakage edges red and the deep module dark. Sequence diagrams work well for "before: 6 round-trips; after: 1."

当要表达的是 “X 调用 Y 调用 Z，看看这一团乱麻” 时，用 Mermaid 的 `flowchart` 或 `graph`。把它包在一个 Tailwind 样式的卡片里，免得显得像是空降进来的。用 classDef 把泄漏的边染成红色、把深模块染成深色。时序图很适合表达 “之前：6 次往返；之后：1 次” 。

```
<div class="rounded-lg border border-slate-200 bg-white p-4">
  <pre class="mermaid">
    flowchart LR
      A[OrderHandler] --> B[OrderValidator]
      B --> C[OrderRepo]
      C -.leak.-> D[PricingClient]
      classDef leak stroke:#dc2626,stroke-width:2px;
      class C,D leak
  </pre>
</div>
```


```
<div class="rounded-lg border border-slate-200 bg-white p-4">
  <pre class="mermaid">
    flowchart LR
      A[OrderHandler] --> B[OrderValidator]
      B --> C[OrderRepo]
      C -.leak.-> D[PricingClient]
      classDef leak stroke:#dc2626,stroke-width:2px;
      class C,D leak
  </pre>
</div>
```

Hand-built boxes-and-arrows (when Mermaid's layout fights you) 手工搭建的方框与箭头（当 Mermaid 的布局和你作对时）

Modules as `<div>`s with borders and labels. Arrows as inline SVG `<line>` or `<path>` elements positioned absolutely over a relative container. Reach for this when you want the "after" diagram to feel like one thick-bordered deep module with greyed-out internals, since Mermaid won't render that with the right weight.

模块用带边框和标签的 `<div>` 表示。箭头用内联 SVG 的 `<line>` 或 `<path>` 元素，绝对定位在一个相对定位的容器之上。当你希望“之后”的图看起来像一个粗边框的深模块、内部细节呈灰色淡出时，就用这种方式，因为 Mermaid 渲染不出恰当的分量感。

Cross-section (good for layered shallowness) 横截面（适合分层式的浅）

Stack horizontal bands (`h-12 border-l-4`) to show layers a call passes through. Before: 6 thin layers each doing nothing. After: 1 thick band labelled with the consolidated responsibility.

堆叠水平色带（`h-12 border-l-4`），展示一次调用要穿过的各个层。之前：6 个薄层，每层什么都不做。之后：1 条厚色带，标注合并后的职责。

Mass diagram (good for "interface as wide as implementation") 体量图（适合“接口和实现一样宽”）

Two rectangles per module: one for interface surface area, one for implementation. Before: interface rectangle is nearly as tall as the implementation rectangle (shallow). After: interface rectangle is short, implementation rectangle is tall (deep).

每个模块两个矩形：一个代表接口的表面积，一个代表实现。之前：接口矩形几乎和实现矩形一样高（浅）。之后：接口矩形很矮，实现矩形很高（深）。

Call-graph collapse 调用图折叠

Before: a tree of function calls rendered as nested boxes. After: the same tree collapsed into one box, with the now-internal calls shown faded inside it.

之前：一棵函数调用树，渲染为嵌套的方框。之后：同一棵树折叠成一个方框，现已成为内部调用的部分在框内以淡色显示。

Style guidance 样式指导

- Lean editorial, not corporate-dashboard. Generous whitespace. Serif optional for headings (`font-serif` works well with stone/slate).

  偏向编辑设计风格，而不是企业仪表盘风格。留白要充足。标题可选用衬线字体（`font-serif` 与 stone/slate 搭配效果很好）。

- Colour sparingly: one accent (emerald or indigo) plus red for leakage and amber for warnings.

  用色要克制：一个强调色（emerald 或 indigo），外加红色表示泄漏、琥珀色表示警告。

- Keep diagrams ~320px tall so before/after sits comfortably side by side without scrolling.

  图示高度保持在约 320px，这样前后对比能舒服地并排显示，无需滚动。

- Use `text-xs uppercase tracking-wider` for module labels inside diagrams, so they read as schematic, not as UI.

  图示内的模块标签用 `text-xs uppercase tracking-wider`，让它们读起来像示意图，而不是 UI。

- The only scripts are the Tailwind CDN and the Mermaid ESM import. The report is otherwise static: no app code, no interactivity beyond Mermaid's own rendering.

  唯一的脚本是 Tailwind CDN 和 Mermaid ESM 导入。除此之外报告是静态的：没有应用代码，除了 Mermaid 自身的渲染之外没有任何交互。

Top recommendation section 首要推荐一节

One larger card. Candidate name, one sentence on why, anchor link to its card. That's it.

一张更大的卡片。候选项名称、一句话说明原因、指向其卡片的锚点链接。仅此而已。

Tone 语气

Plain English, concise, but the architectural nouns and verbs come straight from the `/codebase-design` skill. Concision is not an excuse to drift.

平实、简洁，但架构上的名词和动词直接取自 `/codebase-design` 技能。简洁不是跑偏的借口。

Use exactly: module, interface, implementation, depth, deep, shallow, seam, adapter, leverage, locality.

严格使用： module、interface、implementation、depth、deep、shallow、seam、adapter、leverage、locality。

Never substitute: component, service, unit (for module) · API, signature (for interface) · boundary (for seam) · layer, wrapper (for module, when you mean module).

绝不替换为： component、service、unit（代替 module）· API、signature（代替 interface）· boundary（代替 seam）· layer、wrapper（代替 module，当你指的是 module 时）。

Phrasings that fit the style:

符合该风格的措辞：

- "Order intake module is shallow: interface nearly matches the implementation."

  “Order 接收模块是浅的：接口几乎与实现一致。”

- "Pricing leaks across the seam."

  “定价逻辑跨越接缝泄漏。”

- "Deepen: one interface, one place to test."

  “加深：一个接口，一处测试。”

- "Two adapters justify the seam: HTTP in prod, in-memory in tests."

  “两个适配器让这个接缝站得住：生产环境用 HTTP，测试中用内存实现。”

Wins bullets name the gain in glossary terms: "locality: bugs concentrate in one module", "leverage: one interface, N call sites", "interface shrinks; implementation absorbs the wrappers". Don't write "easier to maintain" or "cleaner code", because those terms aren't in the glossary and don't earn their place.

收益要点用术语表中的术语点明收获：“局部性：bug 集中在一个模块里”、“杠杆：一个接口，N 个调用点”、“接口收缩；实现吸收了那些包装层”。不要写 “更易维护” 或 “代码更整洁”，因为这些词不在术语表里，配不上它们占的位置。

No hedging, no throat-clearing, no "it's worth noting that…". If a sentence could be a bullet, make it a bullet. If a bullet could be cut, cut it. If a term isn't in the `/codebase-design` glossary, reach for one that is before inventing a new one.

不要含糊其辞，不要铺垫清嗓，不要 “值得注意的是……”。如果一句话能写成要点，就写成要点。如果一个要点能删，就删掉。如果某个术语不在 `/codebase-design` 术语表里，先找一个在表里的，再考虑发明新词。

CHAPTER 06 · SKILL

Setup Matt Pocock's Skills
配置 Matt Pocock 的技能

Configure this repo for the engineering skills: set up its issue tracker, triage label vocabulary, and domain doc layout. Run once before first use of the other engineering skills.

为工程类技能配置本仓库：设置其 issue 跟踪器、分诊标签词汇和领域文档布局。在首次使用其他工程类技能之前运行一次。

Source 原文 · v1.3.1 · b0618bc4

Scaffold the per-repo configuration that the engineering skills assume:

搭建工程类技能所依赖的每仓库配置：

- **Issue tracker**: where issues live (GitHub by default; local markdown is also supported out of the box)

  Issue 跟踪器：issue 存放在哪里（默认是 GitHub；也开箱即用地支持本地 markdown）

- **Triage labels**: the strings used for the five canonical triage roles

  分诊标签：五个标准分诊角色所使用的字符串

- **Domain docs**: where `GLOSSARY.md` and ADRs live, and the consumer rules for reading them

  领域文档：`GLOSSARY.md` 和 ADR（架构决策记录）存放在哪里，以及读取它们的使用规则

This is a prompt-driven skill, not a deterministic script. Explore, present what you found, confirm with the user, then write.

这是一个由提示驱动的技能，而不是确定性的脚本。先探索，展示你的发现，与用户确认，然后再写入。

Process 流程

1. Explore 1. 探索

Look at the current repo to understand its starting state. Read whatever exists; don't assume:

查看当前仓库，了解它的初始状态。读取现有的一切；不要假设：

- `git remote -v` and `.git/config`: is this a GitHub repo? Which one?

  `git remote -v` 和 `.git/config`：这是一个 GitHub 仓库吗？是哪一个？

- `AGENTS.md` and `CLAUDE.md` at the repo root: does either exist? Is there already an `## Agent skills` section in either?

  仓库根目录下的 `AGENTS.md` 和 `CLAUDE.md`：二者是否有一个存在？其中是否已经有 `## Agent skills` 小节？

- `GLOSSARY.md` and `GLOSSARY-MAP.md` at the repo root

  仓库根目录下的 `GLOSSARY.md` 和 `GLOSSARY-MAP.md`

- `docs/adr/` and any `src/*/docs/adr/` directories

  `docs/adr/` 以及任何 `src/*/docs/adr/` 目录

- `docs/agents/`: does this skill's prior output already exist?

  `docs/agents/`：本技能之前的输出是否已经存在？

- `.scratch/`: a sign that a local-markdown issue tracker convention is already in use

  `.scratch/`：表明本地 markdown issue 跟踪器约定已在使用中的信号

- Is the `triage` skill installed? (a `triage` skill folder alongside this one, or `triage` in your available skills.) This decides whether Section B runs at all.

  `triage` 技能是否已安装？（与本技能并列的 `triage` 技能文件夹，或你可用技能中的 `triage`。）这决定了 B 部分是否运行。

- Monorepo signals: a `pnpm-workspace.yaml`, a `workspaces` field in `package.json`, or a populated `packages/*` with its own `src/`. These are present only in a genuinely large multi-package repo; their absence means single-context, which is almost every repo.

  Monorepo 信号：`pnpm-workspace.yaml`、`package.json` 中的 `workspaces` 字段，或一个填充了内容、各自带有 `src/` 的 `packages/*`。只有真正大型的多包仓库才会有这些；没有这些就意味着单上下文，几乎所有仓库都是如此。

2. Present findings and ask 2. 展示发现并提问

Summarise what's present and what's missing. Then take the sections in order. One section, one answer, then the next.

总结已有的和缺失的内容。然后按顺序逐个处理各部分。一个部分，一个回答，然后进入下一个。

Lead each section with the recommended answer so the user can accept it in a word. Give a one-line explainer only when the choice genuinely branches; skip the section entirely when exploration already settled it (Section B when `triage` isn't installed, Section C when there's no monorepo).

每个部分都先给出推荐答案，让用户一个词就能接受。只有当选择真正出现分支时才给一行说明；如果探索已经确定了答案，就整个跳过该部分（`triage` 未安装时跳过 B 部分，没有 monorepo 时跳过 C 部分）。

Section A: Issue tracker.

A 部分：Issue 跟踪器。

> Explainer: The "issue tracker" is where issues live for this repo. Skills like `to-tickets`, `triage`, and `to-spec` read from and write to it. They need to know whether to call `gh issue create`, write a markdown file under `.scratch/`, or follow some other workflow you describe. Pick the place you actually track work for this repo.

> 说明：“issue 跟踪器”是本仓库存放 issue 的地方。`to-tickets`、`triage` 和 `to-spec` 等技能会从中读取并向其写入。它们需要知道是该调用 `gh issue create`、在 `.scratch/` 下写一个 markdown 文件，还是遵循你描述的其他某种工作流。选择你实际为本仓库跟踪工作的地方。

Default posture: these skills were designed for GitHub. If a `git remote` points at GitHub, propose that. If a `git remote` points at GitLab (`gitlab.com` or a self-hosted host), propose GitLab. Otherwise (or if the user prefers), offer:

默认立场：这些技能是为 GitHub 设计的。如果某个 `git remote` 指向 GitHub，就推荐 GitHub。如果某个 `git remote` 指向 GitLab（`gitlab.com` 或自托管主机），就推荐 GitLab。否则（或者用户另有偏好），提供以下选项：

- GitHub: issues live in the repo's GitHub Issues (uses the `gh` CLI)

  GitHub：issue 存放在仓库的 GitHub Issues 中（使用 `gh` CLI）

- GitLab: issues live in the repo's GitLab Issues (uses the `glab` CLI)

  GitLab：issue 存放在仓库的 GitLab Issues 中（使用 `glab` CLI）

- Local markdown: issues live as files under `.scratch/<feature>/` in this repo (good for solo projects or repos without a remote)

  本地 markdown：issue 以文件形式存放在本仓库的 `.scratch/<feature>/` 下（适合个人项目或没有远程仓库的仓库）

- Other (Jira, Linear, etc.): ask the user to describe the workflow in one paragraph; the skill will record it as freeform prose

  其他（Jira、Linear 等）：请用户用一段话描述其工作流；技能会把它记录为自由格式的文字

Record the choice in `docs/agents/issue-tracker.md`. The GitHub and GitLab templates carry a "PRs as a request surface" flag, defaulted **off**. Leave it off and don't raise it: a user who wants external PRs in the triage queue can flip the flag in the file later.

把选择记录在 `docs/agents/issue-tracker.md` 中。GitHub 和 GitLab 模板带有一个“把 PR 作为请求入口”的标志，默认**关闭**。保持关闭，也不要提起它：想把外部 PR 纳入分诊队列的用户以后可以在文件里自行打开该标志。

Section B: Triage label vocabulary. Skip this section entirely if the `triage` skill isn't installed (exploration told you), since an uninstalled skill needs no labels.

B 部分：分诊标签词汇。 如果 `triage` 技能未安装（探索已经告诉你了），就整个跳过本部分，因为未安装的技能不需要标签。

If it is installed, ask exactly one question:

如果已安装，只问一个问题：

> Do you want to keep the default triage labels? (recommended: yes)

> 你想保留默认的分诊标签吗？（推荐：**是**）

The defaults are the five canonical roles, each label string equal to its name: needs-triage, needs-info, ready-for-agent, ready-for-human, wontfix. On yes, write them as-is. Only if the user says no, usually because their tracker already uses other names (e.g. bug:triage for needs-triage), collect the overrides so triage applies existing labels instead of creating duplicates.

默认值是五个标准角色，每个标签字符串都等于其名称：needs-triage、needs-info、ready-for-agent、ready-for-human、wontfix。回答是时，原样写入。只有当用户说不时（通常是因为他们的跟踪器已经在用别的名称，例如用 bug:triage 表示 needs-triage），才收集覆盖值，这样 triage 会应用已有的标签，而不是创建重复标签。

Section C: Domain docs. Default to single-context (one GLOSSARY.md + docs/adr/ at the repo root). This fits almost every repo; write it without asking.

C 部分：领域文档。默认采用单上下文（仓库根目录下一个 GLOSSARY.md + docs/adr/）。这适用于几乎所有仓库；不用询问，直接写入。

Offer multi-context (a root GLOSSARY-MAP.md pointing to per-context GLOSSARY.md files) only when exploration found monorepo signals. Then confirm which layout they want.

只有当探索发现了 monorepo 信号时，才提供多上下文选项（根目录下的 GLOSSARY-MAP.md 指向各上下文的 GLOSSARY.md 文件）。然后确认他们想要哪种布局。

3. Confirm and edit 3. 确认与编辑

Show the user a draft of:

向用户展示以下内容的草稿：

- The ## Agent skills block to add to whichever of CLAUDE.md / AGENTS.md is being edited (see step 4 for selection rules)

  要添加到正在编辑的 CLAUDE.md / AGENTS.md 之一中的 ## Agent skills 块（选择规则见第 4 步）

- The contents of docs/agents/issue-tracker.md, docs/agents/domain.md, and docs/agents/triage-labels.md (the last only when triage is installed)

  docs/agents/issue-tracker.md、docs/agents/domain.md 和 docs/agents/triage-labels.md 的内容（最后一个仅在 triage 已安装时）

Let them edit before writing.

在写入之前让他们编辑。

4. Write 4. 写入

Pick the file to edit:

选择要编辑的文件：

- If CLAUDE.md exists, edit it.

  如果 CLAUDE.md 存在，编辑它。

- Else if `AGENTS.md` exists, edit it.

  否则，如果 `AGENTS.md` 存在，编辑它。

- If neither exists, ask the user which one to create; don't pick for them.

  如果二者都不存在，询问用户要创建哪一个；不要替他们选择。

Never create `AGENTS.md` when `CLAUDE.md` already exists (or vice versa); always edit the one that's already there.

当 `CLAUDE.md` 已经存在时，绝不要创建 `AGENTS.md`（反之亦然）；始终编辑已经存在的那个。

If an `## Agent skills` block already exists in the chosen file, update its contents in-place rather than appending a duplicate. Don't overwrite user edits to the surrounding sections.

如果所选文件中已经存在 `## Agent skills` 块，就原地更新其内容，而不是追加一个重复的块。不要覆盖用户对周围小节的编辑。

The block:

该块：

```
## Agent skills

### Issue tracker

[one-line summary of where issues are tracked]. See `docs/agents/issue-tracker.md`.

### Triage labels

[one-line summary of the label vocabulary]. See `docs/agents/triage-labels.md`.

### Domain docs

[one-line summary of layout: "single-context" or "multi-context"]. See `docs/agents/domain.md`.
```

```
## Agent skills

### Issue tracker

[一行概述 issue 在哪里跟踪]。参见 `docs/agents/issue-tracker.md`。

### Triage labels

[一行概述标签词汇]。参见 `docs/agents/triage-labels.md`。

### Domain docs

[一行概述布局："single-context" 或 "multi-context"]。参见 `docs/agents/domain.md`。
```

Include the `### Triage labels` sub-block, and write `docs/agents/triage-labels.md`, only when `triage` is installed and Section B ran. When it isn't, both are omitted.

只有当 `triage` 已安装且 B 部分运行过时，才包含 `### Triage labels` 子块并写入 `docs/agents/triage-labels.md`。否则二者都省略。

When Section B ran on GitHub or GitLab, create each configured label the tracker lacks (`gh label create` / `glab label create`).

当 B 部分在 GitHub 或 GitLab 上运行过时，为跟踪器中缺少的每个已配置标签执行创建（`gh label create` / `glab label create`）。

Then write the docs files using the seed templates in this skill folder as a starting point:

然后以本技能文件夹中的种子模板为起点，写入 docs 文件：

- issue-tracker-github.md: GitHub issue tracker

  issue-tracker-github.md：GitHub issue 跟踪器
- issue-tracker-gitlab.md: GitLab issue tracker

  issue-tracker-gitlab.md：GitLab issue 跟踪器
- issue-tracker-local.md: local-markdown issue tracker

  issue-tracker-local.md：本地 markdown issue 跟踪器
- triage-labels.md: label mapping (only if `triage` is installed)

  triage-labels.md：标签映射（仅当 `triage` 已安装时）
- domain.md: domain doc consumer rules + layout

  domain.md：领域文档使用规则 + 布局

For "other" issue trackers, write `docs/agents/issue-tracker.md` from scratch using the user's description.

对于“其他”issue 跟踪器，根据用户的描述从头编写 `docs/agents/issue-tracker.md`。

5. Done 5. 完成

Tell the user the setup is complete and which engineering skills will now read from these files. Mention they can edit `docs/agents/*.md` directly later; re-running this skill is only necessary if they want to switch issue trackers or restart from scratch.

告诉用户配置已完成，以及哪些工程类技能现在会读取这些文件。提一句他们以后可以直接编辑 `docs/agents/*.md`；只有在想切换 issue 跟踪器或从头重来时，才需要重新运行本技能。

REFERENCE 参考文档 · domain.md

Domain Docs 领域文档

Source 原文

How the engineering skills should consume this repo's domain documentation when exploring the codebase.

工程类技能在探索代码库时应如何使用本仓库的领域文档。

Before exploring, read these 探索之前，先读这些

- `GLOSSARY.md` at the repo root, or

  仓库根目录下的 `GLOSSARY.md`，或者

- `GLOSSARY-MAP.md` at the repo root if it exists: it points at one `GLOSSARY.md` per context. Read each one relevant to the topic.

  仓库根目录下的 `GLOSSARY-MAP.md`（如果存在）：它为每个上下文指向一个 `GLOSSARY.md`。阅读与当前主题相关的每一个。

- `docs/adr/`: read ADRs that touch the area you're about to work in. In multi-context repos, also check `src/<context>/docs/adr/` for context-scoped decisions.

  `docs/adr/`：阅读涉及你即将动手区域的 ADR（架构决策记录）。在多上下文仓库中，还要查看 `src/<context>/docs/adr/`，了解限定于该上下文的决策。

If any of these files don't exist, **proceed silently**. Don't flag their absence; don't suggest creating them upfront. The `/domain-modeling` skill (reached via `/grill-with-docs` and `/improve-codebase-architecture`) creates them lazily when terms or decisions actually get resolved.

如果这些文件中有任何一个不存在，**静默继续**。不要指出它们缺失；不要建议预先创建它们。`/domain-modeling` 技能（经由 `/grill-with-docs` 和 `/improve-codebase-architecture` 触达）会在术语或决策真正被敲定时按需创建它们。

File structure 文件结构

Single-context repo (most repos):

单上下文仓库（大多数仓库）：

```
/
├── GLOSSARY.md
├── docs/adr/
│   ├── 0001-event-sourced-orders.md
│   └── 0002-postgres-for-write-model.md
└── src/
```


```
/
├── GLOSSARY.md
├── docs/adr/
│   ├── 0001-event-sourced-orders.md
│   └── 0002-postgres-for-write-model.md
└── src/
```


Multi-context repo (presence of `GLOSSARY-MAP.md` at the root):

多上下文仓库（根目录存在 `GLOSSARY-MAP.md`）：

```
/
├── GLOSSARY-MAP.md
├── docs/adr/                    ← system-wide decisions
└── src/
    ├── ordering/
    │   ├── GLOSSARY.md
    │   └── docs/adr/            ← context-specific decisions
    └── billing/
        ├── GLOSSARY.md
        └── docs/adr/
```

```
/
├── GLOSSARY-MAP.md
├── docs/adr/                    ← 系统级决策
└── src/
    ├── ordering/
    │   ├── GLOSSARY.md
    │   └── docs/adr/            ← 特定于上下文的决策
    └── billing/
        ├── GLOSSARY.md
        └── docs/adr/
```

Use the glossary's vocabulary 使用术语表的词汇

When your output names a domain concept (in an issue title, a refactor proposal, a hypothesis, a test name), use the term as defined in `GLOSSARY.md`. Don't drift to synonyms the glossary explicitly avoids.

当你的输出要命名某个领域概念时（在 issue 标题、重构提案、假设、测试名称中），使用 `GLOSSARY.md` 中定义的术语。不要漂移到术语表明确避免使用的同义词上。

If the concept you need isn't in the glossary yet, that's a signal: either you're inventing language the project doesn't use (reconsider) or there's a real gap (note it for `/domain-modeling`).

如果你需要的概念还不在术语表中，这是一个信号：要么你在发明项目并不使用的语言（重新考虑），要么确实存在一个空缺（为 `/domain-modeling` 记下来）。

Flag ADR conflicts 标出与 ADR 的冲突

If your output contradicts an existing ADR, surface it explicitly rather than silently overriding:

如果你的输出与某个现有 ADR 相矛盾，要明确指出，而不是悄悄覆盖：

> _Contradicts ADR-0007 (event-sourced orders), but worth reopening because…_

> _与 ADR-0007（event-sourced orders）相矛盾，但值得重新讨论，因为……_

REFERENCE 参考文档 · issue-tracker-github.md

Issue tracker: GitHub Issue 跟踪器：GitHub

Source 原文

Issues and specs for this repo live as GitHub issues. Use the `gh` CLI for all operations.

本仓库的 issue 和规范以 GitHub issue 的形式存放。所有操作都使用 `gh` CLI。

Conventions 约定

- Create an issue: `gh issue create --title "..." --body "..."`. Use a heredoc for multi-line bodies.

  创建 issue：`gh issue create --title "..." --body "..."`。多行正文用 heredoc。

- Read an issue: `gh issue view <number> --json number,title,body,labels,comments`.

  读取 issue：`gh issue view <number> --json number,title,body,labels,comments`。

- List issues: `gh issue list --state open --json number,title,body,labels,comments --jq '[.[] | {number, title, body, labels: [.labels[].name], comments: [.comments[].body]}]'` with appropriate `--label` and `--state` filters.

  列出 issue：`gh issue list --state open --json number,title,body,labels,comments --jq '[.[] | {number, title, body, labels: [.labels[].name], comments: [.comments[].body]}]'`，并配合适当的 `--label` 和 `--state` 过滤条件。

- Make an issue a sub-issue of a parent: `gh issue create --parent <parent> ...`, or `gh issue edit <parent> --add-sub-issue <child>` afterwards (`gh` 2.94+). Older `gh`: `gh api --method POST repos/<owner>/<repo>/issues/<parent>/sub_issues -F sub_issue_id=<child-db-id>` (database id, as in Blocking below). Without sub-issues, put `Part of #<parent>` at the top of the child body.

  把 issue 设为某个父 issue 的子 issue：`gh issue create --parent <parent> ...`，或事后执行 `gh issue edit <parent> --add-sub-issue <child>`（`gh` 2.94+）。较旧的 `gh`：`gh api --method POST repos/<owner>/<repo>/issues/<parent>/sub_issues -F sub_issue_id=<child-db-id>`（数据库 id，同下文阻塞中所述）。没有子 issue 功能时，在子 issue 正文顶部写上 `Part of #<parent>`。

- Comment on an issue: `gh issue comment <number> --body "..."`

  评论 issue：`gh issue comment <number> --body "..."`

- Apply / remove labels: `gh issue edit <number> --add-label "..." / --remove-label "..."`

  添加 / 移除标签：`gh issue edit <number> --add-label "..." / --remove-label "..."`

- Close: `gh issue close <number> --comment "..."`

  关闭：`gh issue close <number> --comment "..."`

Infer the repo from `git remote -v`; `gh` does this automatically when run inside a clone.

从 `git remote -v` 推断仓库；在克隆目录中运行时，`gh` 会自动完成这一步。

Pull requests as a triage surface 把 Pull Request 作为分诊入口

PRs as a request surface: no. _(Set to `yes` if this repo treats external PRs as feature requests; `/triage` reads this flag.)_

把 PR 作为请求入口：no。_（如果本仓库把外部 PR 视为功能请求，则设为 `yes`；`/triage` 会读取这个标志。）_

When set to `yes`, PRs run through the same labels and states as issues, using the `gh pr` equivalents:

设为 `yes` 时，PR 会和 issue 走同样的标签和状态，使用对应的 `gh pr` 命令：

- Read a PR: `gh pr view <number> --comments` and `gh pr diff <number>` for the diff.

  读取 PR：`gh pr view <number> --comments`，用 `gh pr diff <number>` 查看 diff。

- List external PRs for triage: `gh api --paginate 'repos/{owner}/{repo}/pulls?state=open' --jq '.[] | select(.author_association | IN("OWNER","MEMBER","COLLABORATOR") | not) | {number, title, author: .user.login, author_association, labels: [.labels[].name]}'`.

  列出待分诊的外部 PR：`gh api --paginate 'repos/{owner}/{repo}/pulls?state=open' --jq '.[] | select(.author_association | IN("OWNER","MEMBER","COLLABORATOR") | not) | {number, title, author: .user.login, author_association, labels: [.labels[].name]}'`。

- Comment / label / close: `gh pr comment`, `gh pr edit --add-label`/`--remove-label`, `gh pr close`.

  评论 / 标签 / 关闭：`gh pr comment`、`gh pr edit --add-label`/`--remove-label`、`gh pr close`。

GitHub shares one number space across issues and PRs, so a bare `#42` may be either: resolve with `gh pr view 42` and fall back to `gh issue view 42`.

GitHub 的 issue 和 PR 共用同一套编号，所以单独的 `#42` 可能是二者之一：先用 `gh pr view 42` 解析，失败再回退到 `gh issue view 42`。

When a skill says "publish to the issue tracker" 当某个技能说“发布到 issue 跟踪器”时

Create a GitHub issue.

创建一个 GitHub issue。

When a skill says "fetch the relevant ticket" 当某个技能说“获取相关票据”时

Read it as in **Read an issue** above.

按上文**读取** issue的方式读取。

Wayfinding operations 寻路操作

Used by `/wayfinder`. The **map** is a single issue with **child** issues as tickets.

供 `/wayfinder` 使用。**地图**是单个 issue，其**子** issue 即为票据。

- **Map:** a single issue labelled `wayfinder:map`, holding the Notes / Decisions-so-far / Fog body. `gh issue create --label wayfinder:map`.

  地图：一个带有 `wayfinder:map` 标签的 issue，承载 Notes / Decisions-so-far / Fog 正文。`gh issue create --label wayfinder:map`。

- **Child ticket:** an issue linked to the map as a GitHub sub-issue (see **Make an issue a sub-issue of a parent**). Where sub-issues aren't enabled, add the child to a task list in the map body and put `Part of #<map>` at the top of the child body. Labels: `wayfinder:<type>` (`research`/`prototype`/`grilling`/`task`). Once claimed, the ticket is assigned to the driving dev.

  子票据：一个以 GitHub 子 issue 形式链接到地图的 issue（见**把 issue 设为某个父 issue 的子 issue**）。未启用子 issue 功能时，把子票据加到地图正文的任务列表中，并在子票据正文顶部写上 `Part of #<map>`。标签：`wayfinder:<type>`（`research`/`prototype`/`grilling`/`task`）。一旦被认领，票据就指派给推进它的开发者。

- **Blocking:** GitHub's **native issue dependencies**, the canonical, UI-visible representation. Add an edge with `gh api --method POST repos/<owner>/<repo>/issues/<child>/dependencies/blocked_by -F issue_id=<blocker-db-id>`, where `<blocker-db-id>` is the blocker's numeric **database id** (`gh api repos/<owner>/<repo>/issues/<n> --jq .id`, _not_ the `#number` or `node_id`). GitHub reports `issue_dependencies_summary.blocked_by` (open blockers only, the live gate). Where dependencies aren't available, fall back to a `Blocked by: #<n>, #<n>` line at the top of the child body. A ticket is unblocked when every blocker is closed.

  阻塞：GitHub 的**原生 issue 依赖**，这是规范的、在 UI 中可见的表示方式。用 `gh api --method POST repos/<owner>/<repo>/issues/<child>/dependencies/blocked_by -F issue_id=<blocker-db-id>` 添加一条边，其中 `<blocker-db-id>` 是阻塞方的数字**数据库 id**（`gh api repos/<owner>/<repo>/issues/<n> --jq .id`，_不是_ `#number` 或 `node_id`）。GitHub 会报告 `issue_dependencies_summary.blocked_by`（只计仍打开的阻塞方，即实时的闸门）。无法使用依赖功能时，回退为在子票据正文顶部写一行 `Blocked by: #<n>, #<n>`。当所有阻塞方都已关闭时，该票据即解除阻塞。

- **Frontier query:** list the map's open children (`gh issue list --state open`, scoped to the map's sub-issues / task list), drop any with an open blocker (`issue_dependencies_summary.blocked_by > 0`, or an open issue in the `Blocked by` line) or an assignee; first in map order wins.

  前沿查询：列出地图下打开的子票据（`gh issue list --state open`，范围限定为地图的子 issue / 任务列表），剔除任何有打开阻塞方（`issue_dependencies_summary.blocked_by > 0`，或 `Blocked by` 行中有仍打开的 issue）或已有指派人的票据；按地图顺序排在最前者优先。

- Claim: `gh issue edit <n> --add-assignee @me`, the session's first write.

  认领：`gh issue edit <n> --add-assignee @me`，这是本次会话的第一个写操作。

- Resolve: `gh issue comment <n> --body "<answer>"`, then `gh issue close <n>`, then append a context pointer (gist + link) to the map's Decisions-so-far.

  解决：`gh issue comment <n> --body "<answer>"`，然后 `gh issue close <n>`，再在地图的 Decisions-so-far 中追加一个上下文指针（要点 + 链接）。

REFERENCE 参考文档 · issue-tracker-gitlab.md

Issue tracker: GitLab Issue 跟踪器：GitLab

Source 原文

Issues and specs for this repo live as GitLab issues. Use the `glab` CLI for all operations.

本仓库的 issue 和规范以 GitLab issue 的形式存放。所有操作都使用 `glab` CLI。

Conventions 约定

- Create an issue: `glab issue create --title "..." --description "..."`. Use a heredoc for multi-line descriptions. Pass `--description -` to open an editor.

  创建 issue：`glab issue create --title "..." --description "..."`。多行描述用 heredoc。传入 `--description -` 可打开编辑器。

- Read an issue: `glab issue view <number> --comments`. Use `-F json` for machine-readable output.

  读取 issue：`glab issue view <number> --comments`。用 `-F json` 获得机器可读的输出。

- List issues: `glab issue list -O json` with appropriate `--label` filters.

  列出 issue：`glab issue list -O json`，并配合适当的 `--label` 过滤条件。

- Comment on an issue: `glab issue note <number> --message "..."`. GitLab calls comments "notes".

  评论 issue：`glab issue note <number> --message "..."`。GitLab 把评论称为 “notes” 。

- Apply / remove labels: `glab issue update <number> --label "..."` / `--unlabel "..."`. Multiple labels can be comma-separated or by repeating the flag.

  添加 / 移除标签：`glab issue update <number> --label "..."` / `--unlabel "..."`。多个标签可以用逗号分隔，也可以重复该参数。

- Close: `glab issue close <number>`. `glab issue close` does not accept a closing comment, so post the explanation first with `glab issue note <number> --message "..."`, then close.

  关闭：`glab issue close <number>`。`glab issue close` 不接受关闭评论，所以先用 `glab issue note <number> --message "..."` 发布说明，再关闭。

- Merge requests: GitLab calls PRs "merge requests". Use `glab mr create`, `glab mr view`, `glab mr note`, etc., the same shape as `gh pr ...` with `mr` in place of `pr` and `note/--message` in place of `comment/--body`.

  合并请求：GitLab 把 PR 称为 “merge requests” 。使用 `glab mr create`、`glab mr view`、`glab mr note` 等，形式与 `gh pr ...` 相同，只是用 `mr` 代替 `pr`，用 `note/--message` 代替 `comment/--body`。

Infer the repo from `git remote -v`; `glab` does this automatically when run inside a clone.

从 `git remote -v` 推断仓库；在克隆目录中运行时，`glab` 会自动完成这一步。

Merge requests as a triage surface 把合并请求作为分诊入口

MRs as a request surface: no. _(Set to `yes` if this repo treats external merge requests as feature requests; `/triage` reads this flag.)_

把 MR 作为请求入口：no。_（如果本仓库把外部合并请求视为功能请求，则设为 `yes`；`/triage` 会读取这个标志。）_

When set to `yes`, MRs run through the same labels and states as issues, using the `glab mr` equivalents:

设为 `yes` 时，MR 会和 issue 走同样的标签和状态，使用对应的 `glab mr` 命令：

- Read an MR: `glab mr view <number> --comments` and `glab mr diff <number>` for the diff.

  读取 MR：`glab mr view <number> --comments`，用 `glab mr diff <number>` 查看 diff。

- List external MRs for triage: `glab mr list -F json`, then keep only MRs whose author is not a project member/owner (a contributor's MR, not a maintainer's in-flight work).

  列出待分诊的外部 MR：`glab mr list -F json`，然后只保留作者不是项目成员/所有者的 MR（即贡献者的 MR，而不是维护者正在进行中的工作）。

- Comment / label / close: `glab mr note`, `glab mr update --label/--unlabel`, `glab mr close`.

  评论 / 标签 / 关闭：`glab mr note`、`glab mr update --label/--unlabel`、`glab mr close`。

Unlike GitHub, GitLab numbers issues and MRs separately, so `#42` is unambiguous once you know which surface the maintainer means.

与 GitHub 不同，GitLab 对 issue 和 MR 分别编号，所以一旦知道维护者指的是哪个入口，`#42` 就没有歧义。

When a skill says "publish to the issue tracker" 当某个技能说“发布到 issue 跟踪器”时

Create a GitLab issue.

创建一个 GitLab issue。

When a skill says "fetch the relevant ticket" 当某个技能说“获取相关票据”时

Run `glab issue view <number> --comments`.

运行 `glab issue view <number> --comments`。

Wayfinding operations 寻路操作

Used by `/wayfinder`. The map is a single issue with child issues as tickets.

供 `/wayfinder` 使用。地图是单个 issue，其子 issue 即为票据。

- Map: a single issue labelled `wayfinder:map`, holding the Notes / Decisions-so-far / Fog body. `glab issue create --label wayfinder:map`. (On GitLab tiers with native epics, an epic may hold the map instead; a labelled issue works everywhere.)

  地图：一个带有 `wayfinder:map` 标签的 issue，承载 Notes / Decisions-so-far / Fog 正文。`glab issue create --label wayfinder:map`。（在有原生 epic 的 GitLab 版本上，也可以用一个 epic 来承载地图；带标签的 issue 则在任何地方都可用。）

- Child ticket: an issue carrying `Part of #<map>` at the top of its description and labels `wayfinder:<type>` (`research`/`prototype`/`grilling`/`task`). Once claimed, the ticket is assigned to the driving dev.

  子票据：一个在描述顶部写有 `Part of #<map>`、并带有 `wayfinder:<type>`（`research`/`prototype`/`grilling`/`task`）标签的 issue。一旦被认领，票据就指派给推进它的开发者。

- Blocking: GitLab's native blocking link, the canonical, UI-visible representation. Add it with the `/blocked_by #<n>` quick action, posted as a note (`glab issue note <child> --message "/blocked_by #<blocker>"`). Native blocking links are a Premium/Ultimate feature; on the free tier (or where unavailable) fall back to a `Blocked by: #<n>, #<n>` line at the top of the description. A ticket is unblocked when every blocker is closed.

  阻塞：GitLab 的原生阻塞链接，这是规范的、在 UI 中可见的表示方式。用 `/blocked_by #<n>` 快捷操作添加，以 note 形式发布（`glab issue note <child> --message "/blocked_by #<blocker>"`）。原生阻塞链接是 Premium/Ultimate 功能；在免费版（或不可用的情况下）回退为在描述顶部写一行 `Blocked by: #<n>, #<n>`。当所有阻塞方都已关闭时，该票据即解除阻塞。

- Frontier query: `glab issue list -O json` scoped to the map's children, drop any with an open blocker: a native `blocked_by` link to an open issue (`glab api projects/:id/issues/<child-iid>/links`), or an open issue in the `Blocked by` line, or an assignee; first in map order wins.

  前沿查询：`glab issue list -O json`，范围限定为地图的子票据，剔除任何有打开阻塞方的票据：指向仍打开 issue 的原生 `blocked_by` 链接（`glab api projects/:id/issues/<child-iid>/links`），或 `Blocked by` 行中有仍打开的 issue，或已有指派人；按地图顺序排在最前者优先。

- Claim: `glab issue update <n> --assignee @me`, the session's first write.

  认领：`glab issue update <n> --assignee @me`，这是本次会话的第一个写操作。

- Resolve: `glab issue note <n> --message "<answer>"`, then `glab issue close <n>`, then append a context pointer (gist + link) to the map's Decisions-so-far.

  解决：`glab issue note <n> --message "<answer>"`，然后 `glab issue close <n>`，再在地图的 Decisions-so-far 中追加一个上下文指针（要点 + 链接）。

REFERENCE 参考文档 · issue-tracker-local.md

Issue tracker: Local Markdown Issue 跟踪器：本地 Markdown

Source 原文

Issues and specs for this repo live as markdown files in `.scratch/`.

本仓库的 issue 和规范以 markdown 文件的形式存放在 `.scratch/` 中。

Conventions 约定

- One feature per directory: `.scratch/<feature-slug>/`

  每个功能一个目录：`.scratch/<feature-slug>/`

- The spec is `.scratch/<feature-slug>/spec.md`

  规范是 `.scratch/<feature-slug>/spec.md`

- Implementation issues are one file per ticket at `.scratch/<feature-slug>/issues/<NN>-<slug>.md`, numbered from `01`, never a single combined tickets file

  实现 issue 每张票据一个文件，位于 `.scratch/<feature-slug>/issues/<NN>-<slug>.md`，从 `01` 开始编号，绝不合并成一个汇总所有票据的文件

- Triage state is recorded as a `Status:` line near the top of each issue file (see `triage-labels.md` for the role strings)

  分诊状态记录为每个 issue 文件顶部附近的一行 `Status:`（角色字符串见 `triage-labels.md`）

- Comments and conversation history append to the bottom of the file under a `## Comments` heading

  评论和对话历史追加到文件底部的 `## Comments` 标题下

When a skill says "publish to the issue tracker" 当某个技能说“发布到 issue 跟踪器”时

Create a new file under `.scratch/<feature-slug>/` (creating the directory if needed).

在 `.scratch/<feature-slug>/` 下新建一个文件（必要时先创建该目录）。

When a skill says "fetch the relevant ticket" 当某个技能说“获取相关票据”时

Read the file at the referenced path. The user will normally pass the path or the issue number directly.

读取所引用路径下的文件。用户通常会直接给出路径或 issue 编号。

Wayfinding operations 寻路操作

Used by `/wayfinder`. The map is a file with one child file per ticket.

供 `/wayfinder` 使用。地图是一个文件，每张票据对应一个子文件。

- Map: `.scratch/<effort>/map.md` (the Notes / Decisions-so-far / Fog body).

  地图：`.scratch/<effort>/map.md`（包含 Notes / Decisions-so-far / Fog 正文）。

- Child ticket: `.scratch/<effort>/issues/NN-<slug>.md`, numbered from `01`, with the question in the body. A `Type:` line records the ticket type (`research`/`prototype`/`grilling`/`task`); a `Status:` line records `claimed`/`resolved`.

  子票据：`.scratch/<effort>/issues/NN-<slug>.md`，从 `01` 开始编号，问题写在正文中。用一行 `Type:` 记录票据类型（`research`/`prototype`/`grilling`/`task`）；用一行 `Status:` 记录 `claimed`/`resolved`。

- Blocking: a `Blocked by: NN, NN` line near the top. A ticket is unblocked when every file it lists is `resolved`.

  阻塞：顶部附近的一行 `Blocked by: NN, NN`。当它列出的每个文件都是 `resolved` 时，该票据即解除阻塞。

- Frontier: scan `.scratch/<effort>/issues/` for files that are open, unblocked, and unclaimed; first by number wins.

  前沿：扫描 `.scratch/<effort>/issues/`，找出处于打开、未被阻塞且未被认领状态的文件；编号最小者优先。

- Claim: set `Status: claimed` and save before any work.

  认领：在开始任何工作之前，设置 `Status: claimed` 并保存。

- Resolve: append the answer under an `## Answer` heading, set `Status: resolved`, then append a context pointer (gist + link) to the map's Decisions-so-far in `map.md`.

  解决：把答案追加到 `## Answer` 标题下，设置 `Status: resolved`，然后在 `map.md` 中地图的 Decisions-so-far 里追加一个上下文指针（要点 + 链接）。

REFERENCE 参考文档 · triage-labels.md

Triage Labels 分诊标签

Source 原文

The skills speak in terms of five canonical triage roles. This file maps those roles to the actual label strings used in this repo's issue tracker.

这些技能以五个标准分诊角色来表述。本文件把这些角色映射到本仓库 issue 跟踪器中实际使用的标签字符串。

Label in mattpocock/skills	Label in our tracker	Meaning
needs-triage	needs-triage	Maintainer needs to evaluate this issue
needs-info	needs-info	Waiting on reporter for more information
ready-for-agent	ready-for-agent	Fully specified, ready for an AFK agent
ready-for-human	ready-for-human	Requires human implementation
wontfix	wontfix	Will not be actioned

mattpocock/skills 中的标签	我们跟踪器中的标签	含义
needs-triage	needs-triage	需要维护者评估此 issue
needs-info	needs-info	等待报告者提供更多信息
ready-for-agent	ready-for-agent	已完整说明，可交给 AFK 智能体
ready-for-human	ready-for-human	需要由人来实现
wontfix	wontfix	不会处理

When a skill mentions a role (e.g. "apply the AFK-ready triage label"), use the corresponding label string from this table.

当某个技能提到某个角色（例如“应用 AFK-ready 分诊标签”）时，使用此表中对应的标签字符串。

Edit the "Label in our tracker" column to match whatever vocabulary you actually use.

编辑“我们跟踪器中的标签”一列，使其与你实际使用的词汇一致。

CHAPTER 07 · SKILL

Test-Driven Development 测试驱动开发

Test-driven development. Use when the user wants to build features or fix bugs test-first, mentions "red-green-refactor", or wants integration tests.

测试驱动开发。当用户想以测试先行的方式构建功能或修复 bug、提到“red-green-refactor”（红-绿-重构），或想要集成测试时使用。

Source 原文 · v1.3.1 · b0618bc4

TDD is the red → green loop. This skill is the reference that makes that loop produce tests worth keeping: what a good test is, where tests go, the anti-patterns, and the rules of the loop. Every section applies on every cycle: consult them before and during the loop, not after.

TDD 就是红 → 绿的循环。本技能是一份参考，让这个循环产出值得保留的测试：什么是好测试、测试放在哪里、有哪些反模式，以及循环的规则。每一节在每个周期都适用：在循环之前和循环之中查阅它们，而不是事后。

When exploring the codebase, read `GLOSSARY.md` (if it exists) so test names and interface vocabulary match the project's domain language, and respect ADRs in the area you're touching.

探索代码库时，阅读 `GLOSSARY.md`（如果存在），让测试名称和接口词汇与项目的领域语言保持一致，并遵守你所触及区域内的 ADR（架构决策记录）。

What a good test is 什么是好测试

Tests verify behavior through public interfaces, not implementation details. Code can change entirely; tests shouldn't. A good test reads like a specification: "user can checkout with valid cart" tells you exactly what capability exists, and it survives refactors because it doesn't care about internal structure.

测试通过公共接口验证行为，而不是实现细节。代码可以彻底改变；测试不应该随之改变。好测试读起来就像一份规范：“user can checkout with valid cart”准确告诉你存在什么能力，而且它能在重构中存活下来，因为它不关心内部结构。

See tests.md for examples and mocking.md for mocking guidelines.

示例见 tests.md，mock（模拟）准则见 mocking.md。

Seams: where tests go 接缝：测试放在哪里

A seam is the public boundary you test at: the interface where you observe behavior without reaching inside. Tests live at seams, never against internals.

接缝是你进行测试的公共边界：在这个接口上，你无需伸手进入内部就能观察行为。测试位于接缝处，绝不针对内部实现。

Test only at pre-agreed seams. Before writing any test, write down the seams under test and confirm them with the user. No test is written at an unconfirmed seam. You can't test everything, so agreeing the seams up front is how testing effort lands on the critical paths and complex logic instead of every edge case.

只在事先约定的接缝处测试。 在写任何测试之前，先写下要测试的接缝，并与用户确认。不在未经确认的接缝处编写任何测试。你不可能测试所有东西，所以预先约定接缝，才能让测试投入落在关键路径和复杂逻辑上，而不是分散到每一个边界情况。

Ask: "What's the public interface, and which seams should we test?" Give each proposed seam a one-line note on what it catches and what it misses.

问一句：“公共接口是什么？我们应该测试哪些接缝？”给每个提议的接缝附上一行说明：它能捕获什么、会漏掉什么。

When the shape of that interface is itself in question (how deep the module is, where the seam belongs, what the interface should expose), call the Skill tool with "codebase-design" for the vocabulary. It is the shared source of the module, interface, depth, seam, adapter, leverage and locality terms, and it is a reference to consult, not a session to run.

当这个接口本身的形态存疑时（模块有多深、接缝应该放在哪里、接口应该暴露什么），用 "codebase-design" 调用 Skill 工具来获取相关词汇。它是模块、接口、深度、接缝、适配器、杠杆和局部性这些术语的共同来源，是一份供查阅的参考，而不是一个要运行的会话。

Anti-patterns 反模式

- **Implementation-coupled:** mocks internal collaborators, tests private methods, or verifies through a side channel (querying the database instead of using the interface). The tell: the test breaks when you refactor but behavior hasn't changed.

  与实现耦合：mock 内部协作者、测试私有方法，或通过旁路进行验证（查询数据库而不是使用接口）。识别信号：你重构时测试挂了，但行为并没有变。

- **Tautological:** the assertion recomputes the expected value the way the code does (`expect(add(a, b)).toBe(a + b)`, a snapshot derived by hand the same way, a constant asserted equal to itself), so it passes by construction and can never disagree with the code. Expected values must come from an independent source of truth: a known-good literal, a worked example, the spec.

  同义反复：断言用代码本身的计算方式重新算一遍期望值（`expect(add(a, b)).toBe(a + b)`、用同样方式手工推导出的快照、断言一个常量等于它自己），因此它天然通过，永远不可能与代码产生分歧。期望值必须来自独立的事实来源：已知正确的字面量、手算过的示例、规范。

- **Horizontal slicing:** writing all tests first, then all implementation. Bulk tests verify _imagined_ behavior: you test the _shape_ of things rather than user-facing behavior, the tests go insensitive to real changes, and you commit to test structure before understanding the implementation. Work in **vertical slices** instead: one test → one implementation → repeat, each test a **tracer bullet** that responds to what the last cycle taught you.

  水平切分：先写完所有测试，再写所有实现。批量写出的测试验证的是_想象中_的行为：你测的是事物的_形状_而不是面向用户的行为，测试会对真实变化变得不敏感，而且你在理解实现之前就锁定了测试结构。改为按**垂直切片**工作：一个测试 → 一个实现 → 重复，每个测试都是一颗**曳光弹**，回应上一个周期让你学到的东西。

Rules of the loop 循环的规则

- **Red before green.** Write the failing test first, then only enough code to pass it. Don't anticipate future tests or add speculative features.

  先红后绿。 先写失败的测试，然后只写刚好让它通过的代码。不要预判未来的测试，也不要添加臆测的功能。

- **One slice at a time.** One seam, one test, one minimal implementation per cycle.

  一次一个切片。 每个周期只针对一个接缝、一个测试、一个最小实现。

- **Refactoring is not part of the loop.** It belongs to the review stage (see the `code-review` skill), not the red → green implementation cycle.

  重构不属于这个循环。 它属于评审阶段（见 `code-review` 技能），而不属于红 → 绿的实现周期。

REFERENCE 参考文档 · mocking.md

When to Mock 何时使用 mock

Source 原文

Mock at **system boundaries** only:

只在**系统边界**处 mock：

- External APIs (payment, email, etc.)

  外部 API（支付、邮件等）

- Databases (sometimes - prefer test DB)

  数据库（有时--优先使用测试数据库）

- Time/randomness

  时间/随机性

- File system (sometimes)

  文件系统（有时）

Don't mock:

不要 mock：

- Your own classes/modules

  你自己的类/模块

- Internal collaborators

  内部协作者

- Anything you control

  任何你能控制的东西

Designing for Mockability 为可 mock 性而设计

At system boundaries, design interfaces that are easy to mock:

在系统边界处，设计易于 mock 的接口：

1. Use dependency injection

1. 使用依赖注入

Pass external dependencies in rather than creating them internally:

把外部依赖传进来，而不是在内部创建它们：

```
// Easy to mock
function processPayment(order, paymentClient) {
  return paymentClient.charge(order.total);
}

// Hard to mock
function processPayment(order) {
  const client = new StripeClient(process.env.STRIPE_KEY);
  return client.charge(order.total);
}
```


```
// Easy to mock
function processPayment(order, paymentClient) {
  return paymentClient.charge(order.total);
}

// Hard to mock
function processPayment(order) {
  const client = new StripeClient(process.env.STRIPE_KEY);
  return client.charge(order.total);
}
```


2. Prefer SDK-style interfaces over generic fetchers

2. 优先使用 SDK 风格的接口，而不是通用的获取函数

Create specific functions for each external operation instead of one generic function with conditional logic:

为每个外部操作创建专门的函数，而不是用一个带条件逻辑的通用函数：

```
// GOOD: Each function is independently mockable
const api = {
  getUser: (id) => fetch(`/users/${id}`),
  getOrders: (userId) => fetch(`/users/${userId}/orders`),
  createOrder: (data) => fetch('/orders', { method: 'POST', body: data }),
};

// BAD: Mocking requires conditional logic inside the mock
const api = {
  fetch: (endpoint, options) => fetch(endpoint, options),
};
```


```
// GOOD: Each function is independently mockable
const api = {
  getUser: (id) => fetch(`/users/${id}`),
  getOrders: (userId) => fetch(`/users/${userId}/orders`),
  createOrder: (data) => fetch('/orders', { method: 'POST', body: data }),
};

// BAD: Mocking requires conditional logic inside the mock
const api = {
  fetch: (endpoint, options) => fetch(endpoint, options),
};
```


The SDK approach means:

SDK 方式意味着：

- Each mock returns one specific shape

  每个 mock 只返回一种特定的形状

- No conditional logic in test setup

  测试准备中没有条件逻辑

- Easier to see which endpoints a test exercises

  更容易看出一个测试涉及了哪些端点

- Type safety per endpoint

  每个端点都有类型安全

REFERENCE 参考文档 · tests.md

Good and Bad Tests 好测试与坏测试

Source 原文

Good Tests 好测试

Integration-style: Test through real interfaces, not mocks of internal parts.

集成风格：通过真实接口测试，而不是 mock 内部部件。

```
// GOOD: Tests observable behavior
test("user can checkout with valid cart", async () => {
  const cart = createCart();
  cart.add(product);
  const result = await checkout(cart, paymentMethod);
  expect(result.status).toBe("confirmed");
});
```


```
// GOOD: Tests observable behavior
test("user can checkout with valid cart", async () => {
  const cart = createCart();
  cart.add(product);
  const result = await checkout(cart, paymentMethod);
  expect(result.status).toBe("confirmed");
});
```


Characteristics:

特征：

- Tests behavior users/callers care about

  测试用户/调用方关心的行为
- Uses public API only

  只使用公共 API
- Survives internal refactors

  能在内部重构中存活
- Describes WHAT, not HOW

  描述做“什么”，而不是“怎么做”
- One logical assertion per test

  每个测试一个逻辑断言

Bad Tests 坏测试

Implementation-detail tests: Coupled to internal structure.

实现细节测试：与内部结构耦合。

```
// BAD: Tests implementation details
test("checkout calls paymentService.process", async () => {
  const mockPayment = jest.mock(paymentService);
  await checkout(cart, payment);
  expect(mockPayment.process).toHaveBeenCalledWith(cart.total);
});
```


```
// BAD: Tests implementation details
test("checkout calls paymentService.process", async () => {
  const mockPayment = jest.mock(paymentService);
  await checkout(cart, payment);
  expect(mockPayment.process).toHaveBeenCalledWith(cart.total);
});
```


Red flags:

危险信号：

- Mocking internal collaborators

  mock 内部协作者

- Testing private methods

  测试私有方法

- Asserting on call counts/order

  对调用次数/顺序做断言

- Test breaks when refactoring without behavior change

  重构时行为未变，测试却挂了

- Test name describes HOW not WHAT

  测试名描述的是“怎么做”而不是“什么”

- Verifying through external means instead of interface

  通过外部手段而不是接口进行验证

```
// BAD: Bypasses interface to verify
test("createUser saves to database", async () => {
  await createUser({ name: "Alice" });
  const row = await db.query("SELECT * FROM users WHERE name = ?", ["Alice"]);
  expect(row).toBeDefined();
});

// GOOD: Verifies through interface
test("createUser makes user retrievable", async () => {
  const user = await createUser({ name: "Alice" });
  const retrieved = await getUser(user.id);
  expect(retrieved.name).toBe("Alice");
});
```


```
// BAD: Bypasses interface to verify
test("createUser saves to database", async () => {
  await createUser({ name: "Alice" });
  const row = await db.query("SELECT * FROM users WHERE name = ?", ["Alice"]);
  expect(row).toBeDefined();
});

// GOOD: Verifies through interface
test("createUser makes user retrievable", async () => {
  const user = await createUser({ name: "Alice" });
  const retrieved = await getUser(user.id);
  expect(retrieved.name).toBe("Alice");
});
```


Tautological tests: Expected value restates the implementation, so the test passes by construction.

同义反复测试：期望值复述了实现，因此测试天然通过。

```
// BAD: Expected value is recomputed the way the code computes it
test("calculateTotal sums line items", () => {
  const items = [{ price: 10 }, { price: 5 }];
  const expected = items.reduce((sum, i) => sum + i.price, 0);
  expect(calculateTotal(items)).toBe(expected);
});

// GOOD: Expected value is an independent, known literal
test("calculateTotal sums line items", () => {
  expect(calculateTotal([{ price: 10 }, { price: 5 }])).toBe(15);
});
```


```
// BAD: Expected value is recomputed the way the code computes it
test("calculateTotal sums line items", () => {
  const items = [{ price: 10 }, { price: 5 }];
  const expected = items.reduce((sum, i) => sum + i.price, 0);
  expect(calculateTotal(items)).toBe(expected);
});

// GOOD: Expected value is an independent, known literal
test("calculateTotal sums line items", () => {
  expect(calculateTotal([{ price: 10 }, { price: 5 }])).toBe(15);
});
```

CHAPTER 08 · SKILL

To Spec
转为规范

Turn the current conversation into a spec and publish it to the project issue tracker: no interview, just synthesis of what you've already discussed.

把当前对话转化为一份规范并发布到项目的 issue 跟踪器：不做访谈，只综合你们已经讨论过的内容。

Source 原文 · v1.3.1 · b0618bc4

This skill takes the current conversation context and codebase understanding and produces a spec. Do NOT interview the user; just synthesize what you already know.

本技能利用当前对话上下文和对代码库的理解来产出一份规范。不要访谈用户；只需综合你已经知道的内容。

The issue tracker and triage label vocabulary should have been provided to you. If not, tell the user to run `/setup-matt-pocock-skills`.

issue 跟踪器和分诊标签词汇应该已经提供给你了。如果没有，请告诉用户运行 `/setup-matt-pocock-skills`。

Process 流程

1. Explore the repo to understand the current state of the codebase, if you haven't already. Use the project's domain glossary vocabulary throughout the spec, and respect any ADRs in the area you're touching.

   如果你还没有这样做过，先探索仓库，了解代码库的当前状态。在整份规范中使用项目领域术语表中的词汇，并尊重你所涉及领域中的任何 ADR（架构决策记录）。

2. Sketch out the seams at which you're going to test the feature. Existing seams should be preferred to new ones. Use the highest seam possible. If new seams are needed, propose them at the highest point you can. The fewer seams across the codebase, the better - the ideal number is one.

   勾勒出你将在哪些接缝处测试该功能。优先使用已有的接缝，而不是新建接缝。使用尽可能高层的接缝。如果需要新的接缝，就在你能做到的最高点提出它们。整个代码库中的接缝越少越好--理想数量是一个。

Check with the user that these seams match their expectations.

和用户确认这些接缝符合他们的预期。

3. Write the spec using the template below, then publish it to the project issue tracker. Apply the `ready-for-agent` triage label - no need for additional triage.

   使用下面的模板撰写规范，然后把它发布到项目的 issue 跟踪器。打上 `ready-for-agent` 分诊标签--无需额外分诊。

<spec-template>

<spec-template>

Problem Statement 问题陈述

The problem that the user is facing, from the user's perspective.

从用户的视角描述用户面临的问题。

Solution 解决方案

The solution to the problem, from the user's perspective.

从用户的视角描述该问题的解决方案。

User Stories 用户故事

A LONG, numbered list of user stories. Each user story should be in the format of:

一份很长的、带编号的用户故事列表。每个用户故事都应采用如下格式：

1. As an <actor>, I want a <feature>, so that <benefit>

   作为 <actor>，我想要 <feature>，以便 <benefit>

<user-story-example>

<user-story-example>

1. As a mobile bank customer, I want to see balance on my accounts, so that I can make better informed decisions about my spending </user-story-example>

   作为一名手机银行客户，我想要看到我各个账户的余额，以便我能就自己的开支做出更明智的决定 </user-story-example>

This list of user stories should be extremely extensive and cover all aspects of the feature.

这份用户故事列表应当极其详尽，覆盖该功能的方方面面。

Implementation Decisions 实现决策

A list of implementation decisions that were made. This can include:

一份已做出的实现决策列表。可以包括：

- The modules that will be built/modified

  将要构建/修改的模块

- The interfaces of those modules that will be modified

  这些模块中将被修改的接口

- Technical clarifications from the developer

  开发者给出的技术澄清

- Architectural decisions

  架构决策

- Schema changes

  schema 变更

- API contracts

  API 契约

- Specific interactions

  具体的交互

Do NOT include specific file paths or code snippets. They may end up being outdated very quickly.

不要包含具体的文件路径或代码片段。它们可能很快就会过时。

Exception: if a prototype produced a snippet that encodes a decision more precisely than prose can (state machine, reducer, schema, type shape), inline it within the relevant decision and note briefly that it came from a prototype. Trim to the decision-rich parts, not a working demo, just the important bits.

例外：如果某个原型产出了一段代码片段，它对某项决策的表达比文字更精确（状态机、reducer、schema、类型结构），就把它内联到相关决策中，并简短注明它来自原型。只保留富含决策的部分，不要做成可运行的演示，只要重要的部分。

Testing Decisions 测试决策

A list of testing decisions that were made. Include:

一份已做出的测试决策列表。包括：

- A description of what makes a good test (only test external behavior, not implementation details)

  对什么是好测试的描述（只测试外部行为，不测试实现细节）

- Which modules will be tested

  哪些模块将被测试

- Prior art for the tests (i.e. similar types of tests in the codebase)

  测试的先例（即代码库中类似类型的测试）

Out of Scope 范围之外

A description of the things that are out of scope for this spec.

描述哪些事情不在本规范的范围之内。

Further Notes 补充说明

Any further notes about the feature.

关于该功能的任何补充说明。

</spec-template>

</spec-template>

CHAPTER 09 · SKILL

To Tickets
转为票据

Break a plan, spec, or the current conversation into a set of tracer-bullet tickets, each declaring its blocking edges, published to the configured tracker (edges as text in one file per ticket locally, or native blocking links on a real tracker).

把一份计划、规范或当前对话拆分为一组曳光弹式票据，每张票据都声明自己的阻塞边，并发布到已配置的跟踪器（本地以每张票据一个文件、边以文本形式记录；在真实跟踪器上则使用原生的阻塞链接）。

Source 原文 · v1.3.1 · b0618bc4

Break a plan, spec, or conversation into a set of **tickets**: tracer-bullet vertical slices, each declaring the tickets that **block** it.

把一份计划、规范或对话拆分为一组**票据**：曳光弹式的垂直切片，每张票据都声明**阻塞**它的那些票据。

The issue tracker and triage label vocabulary should have been provided to you. If not, tell the user to run `/setup-matt-pocock-skills`.

issue 跟踪器和分诊标签词汇本应已提供给你。如果没有，告诉用户运行 `/setup-matt-pocock-skills`。

Process 流程

1. Gather context 1. 收集上下文

Work from whatever is already in the conversation context. If the user passes a reference (a spec path, an issue number or URL) as an argument, fetch it and read its full body and comments.

基于对话上下文中已有的任何内容开展工作。如果用户以参数形式传入了引用（规范路径、issue 编号或 URL），就去获取它，并阅读其完整正文和评论。

2. Explore the codebase (optional) 2. 探索代码库（可选）

If you have not already explored the codebase, do so to understand the current state of the code. Ticket titles and descriptions should use the project's domain glossary vocabulary, and respect ADRs in the area you're touching.

如果你还没有探索过代码库，就去探索一下，以了解代码的现状。票据的标题和描述应使用项目领域术语表中的词汇，并遵守你所改动区域的 ADR（架构决策记录）。

Look for opportunities to prefactor the code to make the implementation easier. "Make the change easy, then make the easy change."

寻找预重构（prefactor）代码的机会，让实现变得更容易。“先让改动变得容易，再去做那个容易的改动。”

3. Draft vertical slices 3. 起草垂直切片

Break the work into **tracer bullet** tickets.

把工作拆分为**曳光弹**票据。

<vertical-slice-rules>

<vertical-slice-rules>

- Each slice cuts a narrow but COMPLETE path through every layer (schema, API, UI, tests): vertical, NOT a horizontal slice of one layer

  每个切片都贯穿每一层（schema、API、UI、测试），切出一条窄但**完整**的路径：是垂直的，**绝不是**某一层的水平切片

- A completed slice is demoable or verifiable on its own

  完成的切片本身就可以演示或验证

- Each slice is sized to fit in a single fresh context window

  每个切片的大小都要能放进一个全新的上下文窗口

- Any prefactoring should be done first

  任何预重构都应先做

</vertical-slice-rules>

</vertical-slice-rules>

Give each ticket its **blocking edges**: the other tickets that must complete before it can start. A ticket with no blockers can start immediately.

为每张票据给出它的**阻塞边**：在它开始之前必须完成的其他票据。没有阻塞项的票据可以立即开始。

Wide refactors are the exception to vertical slicing. A **wide refactor** is one mechanical change (rename a column, retype a shared symbol) whose **blast radius** fans across the whole codebase, so a single edit breaks thousands of call sites at once and no vertical slice can land green. Don't force it into a tracer bullet; sequence it as **expand-contract**. First expand: add the new form beside the old so nothing breaks. Then migrate the call sites over in batches sized by blast radius (per package, per directory), each batch its own ticket blocked by the expand, keeping CI green batch to batch because the old form still exists. Finally contract: delete the old form once no caller remains, in a ticket blocked by every migrate batch. When even the batches can't stay green alone, keep the sequence but let them share an integration branch that all block a final integrate-and-verify ticket; green is promised only there.

大范围重构是垂直切片的例外。 **大范围重构**是一项机械性改动（重命名一列、改变一个共享符号的类型），其**爆炸半径**波及整个代码库，单次编辑就会一下子破坏成千上万个调用点，任何垂直切片都无法在保持绿色的情况下落地。不要硬把它塞进曳光弹里；把它按**扩张-收缩（expand-contract）**来排序。先扩张：在旧形式旁边加上新形式，这样什么都不会坏。然后按爆炸半径划定批次（按包、按目录），分批把调用点迁移过去，每个批次是一张独立的票据，被扩张票据阻塞；由于旧形式仍然存在，每个批次之间 CI 都能保持绿色。最后收缩：在没有任何调用方之后删除旧形式，这张票据被每一个迁移批次阻塞。如果连单个批次都无法独立保持绿色，就保留这个顺序，但让它们共用一个集成分支，并全部阻塞一张最终的集成与验证票据；只在那里承诺绿色。

4. Quiz the user 4. 向用户确认

Present the proposed breakdown as a numbered list. For each ticket, show:

把拟定的拆分以编号列表的形式呈现出来。对每张票据，展示：

- **Title**: short descriptive name

  标题：简短的描述性名称

- **Blocked by**: which other tickets (if any) must complete first

  被阻塞于：哪些其他票据（如果有）必须先完成

- **What it delivers**: the end-to-end behaviour this ticket makes work

  交付内容：这张票据让哪一项端到端行为得以运作

Ask the user:

询问用户：

- Does the granularity feel right? (too coarse / too fine)

  粒度感觉合适吗？（太粗 / 太细）

- Are the blocking edges correct: does each ticket only depend on tickets that genuinely gate it?

  阻塞边是否正确：每张票据是否只依赖真正卡住它的票据？

- Should any tickets be merged or split further?

  有没有票据应该合并或进一步拆分？

Iterate until the user approves the breakdown.

反复迭代，直到用户认可这份拆分。

5. Publish the tickets to the configured tracker 5. 把票据发布到已配置的跟踪器

Publish the approved tickets. How depends on the tracker `/setup-matt-pocock-skills` configured; the tickets are the same either way, only the shape of the blocking edges changes:

发布已认可的票据。**怎么**发布取决于 `/setup-matt-pocock-skills` 配置的是哪种跟踪器；票据本身在两种情况下都一样，变化的只是阻塞边的形态：

- **Local files** → write one file per ticket under `.scratch/<feature-slug>/issues/<NN>-<slug>.md`, numbered from `01` in dependency order (blockers first). Each file's "Blocked by" lists the numbers/titles it depends on. Use the per-ticket file template below: one ticket per file, never a single combined file.

  本地文件 → 每张票据写一个文件，路径为 `.scratch/<feature-slug>/issues/<NN>-<slug>.md`，从 `01` 开始按依赖顺序编号（阻塞项在前）。每个文件的“Blocked by”列出它所依赖的编号/标题。使用下面的单票据文件模板：一个文件一张票据，绝不合并成单个文件。

- A real issue tracker (GitHub, Linear, …) → publish one issue per ticket in dependency order (blockers first) so each ticket's blocking edges can reference real identifiers. Use the platform's native blocking relationship where it has one; otherwise set each ticket's "Blocked by" to the blocking issues. If the source was an existing issue, make each ticket its sub-issue (tracker doc's operation). Apply the `ready-for-agent` triage label unless instructed otherwise; the tickets are agent-grabbable by construction.

  真实的 issue 跟踪器（GitHub、Linear……）→ 按依赖顺序（阻塞项在前）为每张票据发布一个 issue，这样每张票据的阻塞边都能引用真实的标识符。平台有原生阻塞关系的就用原生的；否则把每张票据的“Blocked by”设为阻塞它的那些 issue。如果来源是一个已有的 issue，就把每张票据设为它的子 issue（按跟踪器文档中的操作）。除非另有指示，否则打上 `ready-for-agent` 分诊标签；这些票据从构造上就可以由智能体直接领取。

Work the **frontier**: any ticket whose blockers are all done. For a purely linear chain that means top to bottom.

沿着**前沿**推进：即所有阻塞项都已完成的票据。对于纯线性的链条，这就意味着从上到下依次完成。

Do NOT close or modify any parent issue.

不要关闭或修改任何父 issue。

<local-ticket-template>

<local-ticket-template>

<NN>: <Ticket title> <NN>: <票据标题>

What to build: the end-to-end behaviour this ticket makes work, from the user's perspective, not a layer-by-layer implementation list.

要构建什么： 这张票据让哪一项端到端行为得以运作，从用户视角描述，而不是一份逐层的实现清单。

Blocked by: the numbers/titles of the tickets that gate this one, or "None (can start immediately)".

被阻塞于： 卡住这张票据的那些票据的编号/标题，或“None (can start immediately)”。

Status: ready-for-agent

状态： ready-for-agent

- [ ] Acceptance criterion 1

  [ ] 验收标准 1

- [ ] Acceptance criterion 2

  [ ] 验收标准 2

</local-ticket-template>

</local-ticket-template>

<issue-template>

<issue-template>

Parent 父 issue

A reference to the parent issue on the tracker (if the source was an existing issue, otherwise omit this section).

指向跟踪器上父 issue 的引用（如果来源是一个已有的 issue；否则省略本节）。

What to build 要构建什么

The end-to-end behaviour this ticket makes work, from the user's perspective, not layer-by-layer implementation.

这张票据让哪一项端到端行为得以运作，从用户视角描述，而不是逐层的实现。

Acceptance criteria 验收标准

- [ ] Criterion 1

  [ ] 标准 1

- [ ] Criterion 2

  [ ] 标准 2

Blocked by 被阻塞于

- A reference to each blocking ticket, or "None (can start immediately)". Omit this section when blockers were set as native edges.

  指向每张阻塞票据的引用，或“None (can start immediately)”。如果阻塞项已设为原生边，则省略本节。

</issue-template>

</issue-template>

In either form, avoid specific file paths or code snippets: they go stale fast. Exception: if a prototype produced a snippet that encodes a decision more precisely than prose can (state machine, reducer, schema, type shape), inline it and note briefly that it came from a prototype. Trim to the decision-rich parts, not a working demo, just the important bits.

无论哪种形式，都要避免写具体的文件路径或代码片段：它们很快就会过时。例外：如果某个原型产出了一段代码片段，它比文字更精确地表达了某项决策（状态机、reducer、schema、类型结构），就把它内联进来，并简要注明它来自原型。只保留富含决策信息的部分，不要做成可运行的演示，只留关键部分。

CHAPTER 10 · SKILL

Wayfinder
寻路者

Plan a huge chunk of work (more than one agent session can hold) as a shared map of decision tickets on your issue tracker, and resolve them one at a time until the way to the destination is clear.

把一大块工作（超出单个智能体会话所能容纳的规模）规划成 issue 跟踪器上一张共享的决策票据地图，并一次解决一张票据，直到通往目的地的路线清晰为止。

Source 原文 · v1.3.1 · b0618bc4

A loose idea has arrived, too big for one agent session, and wrapped in fog: the way from here to the destination isn't visible yet. Wayfinding is about finding that way, not charging at the destination. This skill charts the way as a shared map on the repo's issue tracker, then works its decision tickets (questions whose resolution is a decision, not slices of a build to execute) one at a time until the route is clear.

一个松散的想法来了：它太大，一个智能体会话装不下，而且笼罩在迷雾中--从当前位置到**目的地**的路还看不见。寻路的要义是找到那条路，而不是朝目的地硬冲。本技能把路线绘制成仓库 issue 跟踪器上的一张**共享地图**，然后一次处理一张**决策票据**（其解决结果是一个决策的问题，而不是待执行的构建切片），直到路线清晰。

The destination varies per effort, and naming it is the first act of charting: it shapes every ticket. It might be a spec to hand off and iterate on, a decision to lock before planning starts, or a change made in place like a data-structure migration. The map is domain-agnostic: engineering work, course content, whatever fits the shape.

每项工作的目的地各不相同，而给它命名是绘制地图的第一步：它决定了每一张票据的形态。它可能是一份要交接出去并反复迭代的规范，一个必须在规划开始前锁定的决策，或者是一项就地完成的变更，比如一次数据结构迁移。地图与领域无关：工程工作、课程内容，任何符合这种形态的东西都可以。

Plan, don't do 只规划，不动手

Wayfinder is planning by default: each ticket resolves a decision, and the map is done when the way is clear, with nothing left to decide before someone goes and does the thing. The pull to just do the work is usually the signal you've reached the edge of the map and it's time to hand off. An effort can override this in its Notes, carrying execution into the map itself, but absent that, produce decisions, not deliverables.

寻路者默认是**规划**：每张票据解决一个决策，当路线清晰、在有人去真正做这件事之前已没有任何待决事项时，地图就完成了。想直接动手干活的冲动，通常意味着你已经走到了地图的边缘，是时候交接了。某项工作可以在其 Notes 中推翻这一点，把执行也纳入地图本身；但若没有这样的说明，就产出决策，而不是交付物。

Refer by name 用名称指代

Every map and ticket is an issue, so it has a **name**: its title. In everything the human reads (narration, the map's Decisions-so-far), refer to it by that name, never by a bare id, number, or slug. A wall of `#42, #43, #44` is illegible; names read at a glance. The id and URL don't vanish; a name wraps its link, but they ride _inside_ the name, never stand in for it.

每张地图和每张票据都是一个 issue，所以它有一个**名称**：它的标题。在人类阅读的所有内容中（叙述、地图的 Decisions-so-far），都要用这个名称来指代它，绝不能只用光秃秃的 id、编号或 slug。一墙的 `#42, #43, #44` 根本没法读；名称则一目了然。id 和 URL 并不会消失；名称包裹着它的链接，但它们是藏在名称_里面_的，绝不能取代名称。

The Map 地图

The map is a single issue on this repo's issue tracker, labelled `wayfinder:map`, the canonical artifact. Its tickets are child issues of the map.

地图是本仓库 issue 跟踪器上的单个 issue，带有 `wayfinder:map` 标签，是权威产物。它的票据是地图的子 issue。

The map is an **index**, not a store. It lists the decisions made and points at the tickets that hold their detail; a decision lives in exactly one place, its ticket, so the map never restates it, only gists it and links.

地图是一个**索引**，而不是一个存储库。它列出已做出的决策，并指向保存其细节的票据；一个决策只存在于一个地方，即它的票据，所以地图从不复述它，只给出要点并附上链接。

Where the map, its child tickets, blocking, and frontier queries physically live is tracker-specific. The issue tracker should have been provided to you. If not, tell the user to run `/setup-matt-pocock-skills`. Consult the tracker doc's "Wayfinding operations" section for how _this_ repo expresses them. If no tracker has been provided, default to the local-markdown tracker.

地图、其子票据、阻塞关系以及前沿查询在物理上存放在哪里，取决于具体的跟踪器。 issue 跟踪器应该已经提供给你了。如果没有，请告诉用户运行 `/setup-matt-pocock-skills`。查阅跟踪器文档的 “Wayfinding operations” 一节，了解_本_仓库如何表达这些操作。如果没有提供任何跟踪器，默认使用 local-markdown 跟踪器。

The map body 地图正文

The whole map at low resolution, loaded once per session. Open tickets are **not** listed: they are open child issues, found by query.

低分辨率的整张地图，每个会话加载一次。未关闭的票据**不**列在其中：它们是未关闭的子 issue，通过查询找到。

```
## Destination

<what reaching the end of this map looks like: the spec, decision, or change this effort is finding its way to. One or two lines; ever
  y session orients to it before choosing a ticket.>

## Notes

<domain; skills every session should consult; standing preferences for this effort>

## Decisions so far

<!-- the index: one line per closed ticket, enough to judge relevance, then zoom the link for the detail the ticket holds -->

- [<closed ticket title>](link): <one-line gist of the answer>

## Not yet specified

<!-- see "Fog of war": in-scope fog you can't ticket yet; graduates as the frontier advances -->

## Out of scope

<!-- see "Out of scope": work ruled beyond the destination; closed, never graduates -->
```

```
## Destination

<到达这张地图的终点是什么样子：这项工作正在寻路前往的规范、决策或变更。一两行即可；每个会话在选择票据之前都要先对
  准它。>

## Notes

<领域；每个会话都应查阅的技能；这项工作的长期偏好>

## Decisions so far

<!-- 索引：每张已关闭票据一行，足以判断相关性，然后点开链接查看票据保存的细节 -->

- [<已关闭票据的标题>](link): <答案的一行要点>

## Not yet specified

<!-- 见“战争迷雾”：范围内但还无法立票的迷雾；随着前沿推进而升级为票据 -->

## Out of scope

<!-- 见“范围之外”：被判定在目的地之外的工作；已关闭，永不升级 -->
```

Tickets 票据

Each ticket is a child issue of the map; the tracker's issue id is its identity. Its body is the question, sized to one 100K token agent session:

每张票据都是地图的一个子 issue；跟踪器的 issue id 就是它的身份。它的正文是一个问题，规模以一个 100K token 的智能体会话为限：

```
## Question

<the decision or investigation this ticket resolves>
```

```
## Question

<这张票据要解决的决策或调查>
```

Each ticket carries a `wayfinder:<type>` label, one of `research`, `prototype`, `grilling`, `task` (see Ticket Types). `wayfinder:` labels are the only labels a map and its tickets carry, never a triage label like `ready-for-agent`: they are decisions, not implementation work.

每张票据带有一个 `wayfinder:<type>` 标签，取值为 `research`、`prototype`、`grilling`、`task` 之一（见票据类型）。`wayfinder:` 标签是地图及其票据唯一携带的标签，绝不带像 `ready-for-agent` 这样的分诊标签：它们是决策，不是实现工作。

A session **claims** a ticket by assigning it to the dev driving the map, **first**, before any work, so concurrent sessions skip it. That assignee _is_ the claim: an open, unassigned ticket is unclaimed.

会话通过把票据指派给主导这张地图的开发者来**认领**它，而且要**最先**做，在任何工作之前，这样并发的会话就会跳过它。这个指派人_就是_认领本身：一张未关闭、未指派的票据就是未被认领的。

Blocking uses the tracker's **native** dependency relationship: essential because it renders the frontier _visually_ in the tracker's own UI, so the human sees what's takeable without opening the map. Only a tracker that lacks native blocking falls back to a body convention. A ticket is **unblocked** when every ticket blocking it is closed; the **frontier** is the open, unblocked, unclaimed children, the edge of the known.

阻塞关系使用跟踪器的**原生**依赖关系：这一点至关重要，因为它在跟踪器自己的 UI 中_直观地_呈现出前沿，让人类无需打开地图就能看到哪些可以领取。只有缺少原生阻塞功能的跟踪器才退而采用正文约定。当所有阻塞某张票据的票据都已关闭时，该票据就**解除阻塞**了；**前沿**就是那些未关闭、已解除阻塞、未被认领的子票据，即已知世界的边缘。

The answer isn't part of the body; it's recorded on resolution (see Work through the map). Assets created while resolving a ticket are linked from the issue, not pasted in.

答案不属于正文；它在解决时被记录下来（见推进地图）。解决票据过程中创建的资产从 issue 中链接过去，而不是粘贴进来。

Ticket Types 票据类型

Every ticket is either **HITL** (human in the loop, worked _with_ a human who speaks for themselves) or **AFK**, driven by the agent alone. A HITL ticket only resolves through that live exchange; the agent never stands in for the human's side of it (a grilling agent that answers its own questions has broken this).

每张票据要么是 **HITL**（human in the loop，人在回路中，_与_一位亲自发言的人类协作完成），要么是 **AFK**，完全由智能体独自推进。HITL 票据只能通过这种实时交流来解决；智能体绝不能代替人类那一方发言（一个自问自答的深度追问智能体就破坏了这一点）。

- Research (AFK): Reading documentation, third-party APIs, or local resources like knowledge bases to surface a fact a decision waits on. Resolved by a subagent that calls the Skill tool with "research". Use when knowledge outside the current working directory is required.

  研究（AFK）：阅读文档、第三方 API 或知识库之类的本地资源，找出某个决策所等待的事实。由一个以 "research" 调用 Skill 工具的子智能体来解决。当需要当前工作目录之外的知识时使用。

- Prototype (HITL): Raise the fidelity of the discussion by making a cheap, rough, concrete artifact to react to (an outline, a rough take, a stub, or UI/logic code) by calling the Skill tool with "prototype". Links the prototype as an asset. Use when "how should it look" or "how should it behave" is the key question.

  原型（HITL）：通过以 "prototype" 调用 Skill 工具，制作一个廉价、粗糙、具体的产物供大家反应（一份大纲、一个粗略版本、一个桩，或 UI/逻辑代码），以提升讨论的保真度。把原型作为资产链接上去。当“它应该长什么样”或“它应该如何表现”是关键问题时使用。

- Grilling (HITL): Conversation. The default case. Always call the Skill tool twice, for "grilling" and "domain-modeling".

  深度追问（HITL）：对话。这是默认情形。始终调用 Skill 工具两次，分别用 "grilling" 和 "domain-modeling"。

- Task (HITL or AFK): Manual work that must happen before a _decision_ can be made: nothing to decide, prototype, or research, but the discussion is blocked until it's done. Signing up for a service so its API can be judged, provisioning access, moving data so its shape can be seen. This is the one type that _does_ rather than decides, and it earns its place by unblocking a decision, not by delivering the destination. The agent drives it alone where it can (AFK); otherwise it hands the human a precise checklist (HITL). Resolved when the work is done; the answer records what was done and any resulting facts (credentials location, new URLs, row counts) later tickets depend on.

  任务（HITL 或 AFK）：在做出某个_决策_之前必须完成的手工工作：没有什么要决定、做原型或研究的，但在它完成之前讨论被卡住了。注册某个服务以便评估它的 API、开通访问权限、搬移数据以便看清其形态。这是唯一一种_动手做_而非做决策的类型，它之所以有存在的价值，是因为它为某个决策解除阻塞，而不是因为它交付了目的地。智能体能独自完成的就独自推进（AFK）；否则就给人类一份精确的清单（HITL）。工作完成即告解决；答案记录做了什么，以及后续票据所依赖的任何由此产生的事实（凭据存放位置、新的 URL、行数）。

Fog of war 战争迷雾

The map is _deliberately_ incomplete: don't chart what you can't yet see. Beyond the live tickets lies the fog of war: the dim view of decisions and investigations you can tell are coming but can't yet pin down, because they hang on questions still open. Resolving a ticket clears the fog ahead of it, graduating whatever's now specifiable into fresh tickets, one at a time, until the way to the destination is clear and no tickets remain.

地图是_有意_不完整的：不要绘制你还看不见的东西。在活跃票据之外是**战争迷雾**：那些你能感觉到即将到来、却还无法确定的决策和调查的模糊景象，因为它们取决于仍然悬而未决的问题。解决一张票据会驱散它前方的迷雾，把此时已能明确表述的内容升级为新的票据，一次一张，直到通往目的地的路线清晰、不再剩下任何票据。

The map's **Not yet specified** section is where that dim view is written down: the suspected question, the area to revisit later. It's the undiscovered frontier _toward_ the destination: everything here is in scope, just not sharp enough to ticket. Write as loosely or as fully as the view allows; it doubles as a signpost for collaborators reading where the effort is headed.

地图的 **Not yet specified** 一节就是写下这种模糊景象的地方：猜测中的问题、以后要回头看的领域。它是_朝向_目的地的未探明前沿：这里的一切都在范围之内，只是还不够清晰，无法立票。景象允许你写多松散或多完整，就写多松散或多完整；它同时也是一块路标，供协作者了解这项工作的走向。

Fog or ticket? The test is whether you can state the question precisely now, _not_ whether you can answer it now.

迷雾还是票据？ 判断标准是你现在能否精确地陈述这个问题，_而不是_你现在能否回答它。

- **Ticket when** the question is already sharp, even if it's blocked and you can't act on it yet.

  立票：问题已经很清晰时，即使它被阻塞、你暂时无法对它采取行动。

- **Not yet specified when** you can't yet phrase it that sharply. Don't pre-slice the fog into ticket-sized pieces: it's coarser than a ticket, and one patch may graduate into several tickets, or none, once the frontier reaches it.

  放入 Not yet specified：你还无法把它表述得那么清晰时。不要把迷雾预先切成票据大小的碎片：迷雾比票据更粗粒度，等前沿抵达时，一片迷雾可能升级为好几张票据，也可能一张都没有。

Not yet specified excludes what's already decided (Decisions so far), what's already a live ticket, and what's out of scope (the next section).

Not yet specified 不包括已经决定的内容（Decisions so far）、已经是活跃票据的内容，以及范围之外的内容（下一节）。

Out of scope 范围之外

Fog only ever gathers _toward_ the destination. The destination fixes the scope, so work beyond it is **out of scope**: it isn't fog, and it doesn't belong in **Not yet specified**. It gets its own **Out of scope** section on the map: work you've consciously ruled out of _this_ effort. Scope, not sharpness, lands it here.

迷雾只会_朝着_目的地聚集。目的地确定了范围，因此超出目的地的工作属于**范围之外**：它不是迷雾，不属于 **Not yet specified**。它在地图上有自己的 **Out of scope** 一节：存放你有意识地排除在_这项_工作之外的工作。让它落到这里的是范围，而不是清晰度。

Out-of-scope work never graduates (the frontier stops at the destination), so it returns only if the destination is redrawn, and then as a fresh effort, not a resumption.

范围之外的工作永远不会升级（前沿止步于目的地），所以只有在目的地被重新划定时它才会回来，而且是作为一项全新的工作，而不是继续原来的工作。

Ruling something out of scope is a scoping act, not a step on the route. When a ticket that already exists turns out to sit past the destination (mis-scoped in while charting, or exposed by a resolution), **close it** (a closed ticket is unambiguously off the frontier) and leave one line in the **Out of scope** section: the gist plus why it's out of scope, linking the closed ticket. It stays out of **Decisions so far**, which records the route actually walked; a scope boundary isn't a step on it.

把某件事判定为范围之外是一种界定范围的行为，而不是路线上的一步。当一张已存在的票据被发现位于目的地之外时（绘制时被错误地纳入范围，或被某次解决结果暴露出来），**关闭它**（已关闭的票据毫无疑问地离开了前沿），并在 **Out of scope** 一节留下一行：要点加上它为何在范围之外，并链接到这张已关闭的票据。它不进入 **Decisions so far**，后者记录的是实际走过的路线；范围边界不是路线上的一步。

Invocation 调用

Two modes. Either way, **never resolve more than one ticket per session**, with the exception of research tickets.

两种模式。无论哪种，**每个会话绝不解决超过一张票据**，研究票据除外。

Chart the map 绘制地图

User invokes with a loose idea.

用户带着一个松散的想法调用。

1. **Name the destination.** Call the Skill tool twice, for "grilling" and "domain-modeling", to pin down what this map is finding its way to: the spec, decision, or change. The destination fixes the scope, so it's settled first.

   命名目的地。 调用 Skill 工具两次，分别用 "grilling" 和 "domain-modeling"，确定这张地图要寻路前往的地方：规范、决策或变更。目的地确定范围，所以要最先把它定下来。

2. **Map the frontier.** Grill again, **breadth-first** this time: fan out across the whole space rather than deep on any one thread, surfacing the open decisions and the first steps takeable now. **If this surfaces no fog** (the way to the destination is already clear, the whole journey small enough for one session), you don't need a map. Stop and ask the user how they'd like to proceed.

   绘出前沿。 再次深度追问，这次是**广度优先**：在整个空间中铺开，而不是在任何一条线索上深挖，把待决的决策和现在就能迈出的第一步都找出来。**如果这一步没有发现任何迷雾**（通往目的地的路已经清晰，整趟旅程小到一个会话就能完成），你就不需要地图。停下来，问用户想怎么继续。

3. **Create the map** (label `wayfinder:map`): Destination and Notes filled in, Decisions-so-far empty, the fog sketched into **Not yet specified**.

   创建地图（标签 `wayfinder:map`）：填好 Destination 和 Notes，Decisions-so-far 留空，把迷雾草拟进 **Not yet specified**。

4. Create the tickets you can specify now as child issues of the map, then wire blocking edges in a second pass (issues need ids before they can reference each other). Write cross-references in that pass too, with real ids: a placeholder `#<n>` auto-links to an unrelated issue. Wiring sorts them into the frontier and the blocked; everything you can't yet specify stays in the fog: the Not yet specified section.

   创建你现在能明确表述的票据，作为地图的子 issue，然后在**第二遍**中连接阻塞关系（issue 需要先有 id 才能相互引用）。交叉引用也在这一遍中写，用真实的 id：占位符 `#<n>` 会自动链接到一个无关的 issue。连接关系会把它们分为前沿和被阻塞两类；所有你还无法明确表述的东西都留在迷雾中，也就是 Not yet specified 一节。

5. Fire the research subagents. For each `research` ticket you just created, spin up a subagent that calls the Skill tool with "research" to resolve it in parallel, capturing its findings on a throwaway `research/<name>` branch with a context pointer from the ticket. Push the branch but open no PR: it is never merged.

   派出研究子智能体。 对你刚创建的每张 `research` 票据，启动一个以 "research" 调用 Skill 工具的子智能体去并行解决它，把发现记录在一个用完即弃的 `research/<name>` 分支上，并从票据放一个上下文指针指向它。推送该分支，但不要开 PR：它永远不会被合并。

6. Stop: charting is one session's work; it hand-resolves nothing.

   停下：绘制地图是一个会话的工作量；它不亲手解决任何票据。

Work through the map 推进地图

User invokes with a map (URL or number). A ticket is optional: without one, you pick the next decision, not the user.

用户带着一张地图（URL 或编号）调用。票据是**可选的**：如果没有指定，由你来挑选下一个决策，而不是用户。

1. Load the map: the low-res view, not every ticket body.

   加载**地图**：低分辨率视图，而不是每张票据的正文。

2. Choose the ticket. If the user named one, use it. Otherwise take the first frontier ticket in order. Claim it: assign it to yourself before any work.

   选择票据。如果用户指定了一张，就用它。否则取前沿中按顺序排列的第一张票据。**认领它**：在任何工作之前把它指派给自己。

3. Resolve it as the type its `wayfinder:<type>` label names (see Ticket Types). Read the label, not just the body: the body never states the type. Zoom as needed: fetch the full body of any related or closed ticket on demand; call the Skill tool for whichever skills the `## Notes` block names. If in doubt, call the Skill tool twice, for "grilling" and "domain-modeling".

   按照其 `wayfinder:<type>` 标签所指明的类型来解决它（见票据类型）。要读标签，而不只是正文：正文从不说明类型。**按需放大**：按需获取任何相关或已关闭票据的完整正文；对 `## Notes` 块中点名的任何技能调用 Skill 工具。拿不准时，调用 Skill 工具两次，分别用 "grilling" 和 "domain-modeling"。

4. Record the resolution: post the answer as a **resolution comment**, **close** the issue, and **append a context pointer** to the map's Decisions-so-far.

   记录解决结果：把答案作为一条**解决评论**发布，**关闭**该 issue，并在地图的 Decisions-so-far 中**追加一个上下文指针**。

5. Add newly-surfaced tickets (create-then-wire); graduate any fog the answer has made specifiable, clearing each graduated patch from **Not yet specified** so it lives only as its new ticket. If the answer reveals that a ticket (this one or another) sits beyond the destination, **rule it out of scope** rather than resolving it on the route. If the decision invalidates other parts of the map, update or delete those tickets.

   添加新浮现的票据（先创建，再连接）；把答案已使之可明确表述的迷雾升级为票据，并把每块已升级的迷雾从 **Not yet specified** 中清除，使它只以新票据的形式存在。如果答案表明某张票据（这张或另一张）位于目的地之外，就**把它判定为范围之外**，而不是在路线上解决它。如果这个决策使地图的其他部分失效，就更新或删除那些票据。

The user may run unblocked tickets in parallel, so expect other sessions to be editing the tracker concurrently.

用户可能会并行处理多张已解除阻塞的票据，所以要预料到会有其他会话同时在编辑跟踪器。

CHAPTER 11 · SKILL

Implement
实现

Implement a piece of work based on a spec or set of tickets.

根据一份规范或一组票据实现一项工作。

Source 原文 · v1.3.1 · b0618bc4

Implement the work described by the user in the spec or tickets.

实现用户在规范或票据中描述的工作。

If the user passes a ticket reference, fetch it from the issue tracker and state its title before starting. If the reference is ambiguous, ask.

如果用户传入的是票据引用，先从 issue 跟踪器中获取它，并在开始之前说出它的标题。如果引用有歧义，就询问。

Call the Skill tool with "tdd" where possible, at pre-agreed seams.

尽可能以 "tdd" 调用 Skill 工具，在事先约定好的接缝处进行。

Run typechecking regularly, single test files regularly, and the full test suite once at the end.

定期运行类型检查，定期运行单个测试文件，并在最后运行一次完整的测试套件。

Once done, call the Skill tool with "code-review" to review the work.

完成后，以 "code-review" 调用 Skill 工具来审查这些工作。

Commit your work to the current branch.

把你的工作提交到当前分支。

CHAPTER 12 · SKILL

Implement Spec
实现规范

Implement the result of /to-spec and /to-tickets in code.

把 /to-spec 和 /to-tickets 的产出用代码实现出来。

Source 原文 · v1.3.1 · b0618bc4

You have been provided a spec. This spec should have tickets associated with it, describing how to implement the spec.

你已经拿到了一份规范。这份规范应当关联着若干票据，描述如何实现该规范。

The issue tracker should have been provided to you. If not, tell the user to run `/setup-matt-pocock-skills`.

issue 跟踪器应该已经提供给你了。如果没有，请告诉用户运行 `/setup-matt-pocock-skills`。

The goal is the entire spec implemented on a single **integration branch**, with every ticket resolved the way the issue tracker closes work.

目标是在单个**集成分支**上实现整份规范，并让每张票据都按 issue 跟踪器关闭工作的方式得到解决。

The tickets are not a list of steps. They are a **task graph** with blocking relationships between them. This means there is always a **frontier** of tickets which are ready to be grabbed.

这些票据不是一份步骤清单。它们是一张**任务图**，彼此之间有阻塞关系。这意味着始终存在一个由已可领取的票据构成的**前沿**。

Communication to and from subagents should be sparse. Communicate primarily through **context pointers**: to the spec, tickets, research notes, and previous commits. Don't duplicate information already available via pointers.

与子智能体之间的往来沟通应当精简。主要通过**上下文指针**来沟通：指向规范、票据、研究笔记和之前的提交。不要重复已经可以通过指针获取的信息。

Implementer subagents should be run in the background where possible for maximum concurrency.

实现者子智能体应尽可能在后台运行，以获得最大的并发度。

Steps 步骤

1. Read the spec and tickets to understand the task graph.

   阅读规范和票据，理解任务图。

2. (optional) Use an **exploration subagent** to conduct any exploration required by the tickets - relevant codebase files or external documentation. Ensure the exploration subagent can save files - it should save its markdown notes in a directory outside the repo, accessible by all future subagents. This lets **implementer subagents** focus on implementation rather than exploration.

   （可选）使用一个**探索子智能体**来完成票据所需的任何探索--相关的代码库文件或外部文档。确保探索子智能体能够保存文件--它应当把 Markdown 笔记保存在仓库之外的一个目录中，让之后所有的子智能体都能访问。这样**实现者子智能体**就能专注于实现，而不是探索。

3. Create the integration branch. If the issue tracker closes work through PRs, or the user asks for one, open a draft PR after the first merge in step 5 (a branch with no commits ahead of main can't open one), marked as closing the spec and tickets.

   创建集成分支。如果 issue 跟踪器通过 PR 关闭工作，或者用户要求开 PR，就在第 5 步的第一次合并之后开一个草稿 PR（一个相对 main 没有任何领先提交的分支无法开 PR），并标记为关闭该规范和这些票据。

4. Use **implementer subagents** to implement each ticket, each in its own worktree on its own branch. Each implementer subagent:

   使用**实现者子智能体**来实现每张票据，每个都在自己的 worktree 中、自己的分支上。每个实现者子智能体：

- confirms its worktree is based on the integration branch before starting, and resets onto it if not;

  在开始之前确认自己的 worktree 基于集成分支，如果不是，就重置到集成分支上；

- calls the Skill tool with `tdd` to build the ticket;

  以 `tdd` 调用 Skill 工具来构建该票据；

- merges the integration branch tip into its own branch before reporting done

  在报告完成之前，把集成分支的最新提交合并进自己的分支

5. Once an **implementer subagent** completes, merge its work to the integration branch with a **merger subagent**.

   一旦某个**实现者子智能体**完成，就用一个**合并者子智能体**把它的工作合并到集成分支。

6. If this changes the **frontier** of available tickets, kick off more **implementer subagents** to work on the new tickets. This allows for maximum concurrency.

   如果这改变了可用票据的**前沿**，就启动更多**实现者子智能体**来处理新的票据。这样可以获得最大的并发度。

7. Once all tickets are complete, call the Skill tool with `code-review` on the integration branch. Fix all issues raised by the code review in a single **implementer subagent**.

   所有票据完成后，在集成分支上以 `code-review` 调用 Skill 工具。在单个**实现者子智能体**中修复代码审查提出的所有问题。

8. If a draft PR exists, mark it ready for review. Otherwise, resolve each ticket the way the issue tracker closes work, and report the integration branch.

   如果存在草稿 PR，就把它标记为可供审查。否则，按 issue 跟踪器关闭工作的方式解决每张票据，并报告集成分支。

9. Clean up all **implementer subagent** worktrees.

   清理所有**实现者子智能体**的 worktree。

CHAPTER 13 · SKILL

Prototype
原型

Build a throwaway prototype to answer a design question. Use when the user wants to sanity-check whether a state model or logic feels right, or explore what a UI should look like.

构建一次性原型来回答某个设计问题。当用户想快速检验某个状态模型或逻辑是否合理，或想探索某个 UI 应该长什么样时使用。

Source 原文 · v1.3.1 · b0618bc4

A prototype is **throwaway code that answers a question**. The question decides the shape.

原型是**用来回答一个问题的一次性代码**。问题决定原型的形态。

Pick a branch 选择分支

Identify which question is being answered, using the user's prompt, the surrounding code, or by asking if the user is around:

借助用户的提示、周边代码，或在用户在场时直接询问，判断要回答的是哪个问题：

- "Does this logic / state model feel right?" → LOGIC.md. Build a single shareable HTML file (free-play buttons plus tabbed guided walkthroughs) that pushes the state machine through cases that are hard to reason about on paper, and that a non-developer can drive.

  “这段逻辑 / 这个状态模型感觉对吗？” → LOGIC.md。构建一个可分享的单一 HTML 文件（自由操作按钮加分标签页的引导式演练），把状态机推过那些在纸面上难以推演的情形，并且让非开发者也能上手操作。

- "What should this look like?" → UI.md. Generate several radically different UI variations on a single route, switchable via a URL search param and a floating bottom bar.

  “这个应该长什么样？” → UI.md。在同一个路由上生成若干截然不同的 UI 变体，可通过 URL 查询参数和一个浮动底栏切换。

The two branches produce very different artifacts, so getting this wrong wastes the whole prototype. If the question is genuinely ambiguous and the user isn't reachable, default to whichever branch better matches the surrounding code (a backend module → logic; a page or component → UI) and state the assumption at the top of the prototype.

两个分支产出的东西截然不同，选错了整个原型就白做了。如果问题确实模棱两可且联系不上用户，就默认选更贴合周边代码的分支（后端模块 → 逻辑；页面或组件 → UI），并在原型顶部写明这一假设。

Rules that apply to both 两个分支共同适用的规则

1. **Throwaway from day one, and clearly marked as such.** Locate the prototype code close to where it will actually be used (next to the module or page it's prototyping for) so context is obvious, but name it so a casual reader can see it's a prototype, not production. For throwaway UI routes, obey whatever routing convention the project already uses; don't invent a new top-level structure.

   从第一天起就是一次性的，并且明确标注。 把原型代码放在它将来真正被使用的位置附近（紧挨着它所服务的模块或页面），让上下文一目了然；但命名要让随便扫一眼的读者也能看出这是原型，而不是生产代码。对于一次性 UI 路由，遵循项目已有的路由约定；不要另起一套新的顶层结构。

2. **Trivial to run.** A UI prototype starts from one command in the project's task runner: `pnpm <name>`, `python <path>`, `bun <path>`, etc. A logic demo is a single HTML file the user double-clicks. Either way, no thinking required to start it.

   运行起来毫不费力。 UI 原型通过项目任务运行器中的一条命令启动：`pnpm <name>`、`python <path>`、`bun <path>` 等。逻辑演示是一个用户双击即可打开的 HTML 文件。无论哪种，启动时都不需要动脑。

3. **No persistence by default.** State lives in memory. Persistence is the thing the prototype is _checking_, not something it should depend on. If the question explicitly involves a database, hit a scratch DB or a local file with a clear "PROTOTYPE, wipe me" name.

   默认不做持久化。 状态存在内存里。持久化是原型要_检验_的对象，而不是它应该依赖的东西。如果问题明确涉及数据库，就连一个临时数据库或本地文件，并起一个醒目的“PROTOTYPE, wipe me”之类的名字。

4. **Skip the polish.** No tests, no error handling beyond what makes the prototype _runnable_, no abstractions. The point is to learn something fast.

   跳过打磨。 不写测试，除了让原型_能跑起来_所必需的部分外不做错误处理，不做抽象。目的就是快速学到东西。

5. **Surface the state.** After every action (logic) or on every variant switch (UI), print or render the full relevant state so the user can see what changed.

   把状态摆出来。 每次执行操作（逻辑）或每次切换变体（UI）之后，打印或渲染完整的相关状态，让用户看到发生了什么变化。

6. **Capture it when done.** Fold any validated decision into the real code, then capture the prototype itself as a **primary source**: commit it to a throwaway branch, out of main, and leave a context pointer to that branch on the implementation issue. Capture the answer too (the verdict and the question it settled) in the issue or a commit. The main branch keeps only the validated decision.

   完成后留存下来。 把任何经过验证的决策并入真实代码，然后把原型本身作为**一手资料**留存：提交到一个一次性分支上，不进 main，并在实现 issue 上留下指向该分支的上下文指引。答案也要留存（结论以及它所解决的问题），写在 issue 或某次提交里。main 分支只保留经过验证的决策。

REFERENCE 参考文档 · LOGIC.md

Logic Prototype 逻辑原型

Source 原文

A single, self-contained HTML file (a **shareable demo**) that lets anyone drive a state model by clicking buttons. Use this when the question is about **business logic, state transitions, or data shape**: the kind of thing that looks reasonable on paper but only feels wrong once you push it through real cases.

一个自包含的单一 HTML 文件（一个**可分享的演示**），让任何人都能通过点击按钮来驱动一个状态模型。当问题关乎**业务逻辑、状态转换或数据形态**时使用它：也就是那些在纸面上看着合理、一旦用真实案例推一遍才觉得不对劲的东西。

Because it's one file with nothing to install, you can hand it to a non-developer (a designer, a PM, a domain expert) and let them feel the model for themselves. So it speaks their language, not the code's.

因为它只是一个文件、无需安装任何东西，你可以把它交给非开发者（设计师、产品经理、领域专家），让他们亲自感受这个模型。所以它要说他们的语言，而不是代码的语言。

When this is the right shape 何时适合用这种形态

- "I'm not sure if this state machine handles the edge case where X then Y."

  “我不确定这个状态机能不能处理先 X 后 Y 的边界情况。”

- "Does this data model actually let me represent the case where..."

  “这个数据模型真的能表示这样一种情况吗……”

- "I want to feel out what the API should look like before writing it."

  “我想在动手写 API 之前先摸索一下它应该长什么样。”

- Anything where someone wants to **press buttons and watch state change**.

  任何有人想要**按按钮、看状态变化**的场景。

If the question is "what should this look like," this is the wrong branch. Use UI.md.

如果问题是“这个应该长什么样”，那就走错分支了。请使用 UI.md。

Process 流程

1. State the question 1. 陈述问题

Before writing code, write down what state model and what question you're prototyping. One paragraph, at the top of the demo (in a visible intro, not just a comment). A logic prototype that answers the wrong question is pure waste, so make the question explicit so it can be checked later, whether the user is watching now or returning to it AFK.

写代码之前，先写下你要原型化的是什么状态模型、要回答什么问题。一段话，放在演示的顶部（放在可见的简介里，而不只是一条注释）。回答错问题的逻辑原型纯属浪费，所以要把问题明确写出来，以便日后核对--无论用户此刻就在旁边看着，还是 AFK 之后回来再看。

2. Isolate the logic in a portable module 2. 把逻辑隔离到一个可移植的模块里

Put the actual logic (the bit that's answering the question) in a single `<script>` block written as a small, pure module that could be lifted out and dropped into the real codebase later. The page around it is throwaway; this module isn't.

把真正的逻辑（负责回答问题的那部分）放在单个 `<script>` 块中，写成一个小而纯的模块，以后可以原样拎出来放进真实代码库。它周围的页面是一次性的；这个模块不是。

The right shape depends on the question:

合适的形态取决于问题本身：

- A pure reducer: `(state, action) => state`. Good when actions are discrete events and state is a single value.

  纯 reducer：`(state, action) => state`。适合操作是离散事件、状态是单个值的情况。

- A state machine: explicit states and transitions. Good when "which actions are even legal right now" is part of the question.

  状态机：显式的状态和转换。适合“当前到底哪些操作是合法的”本身就是问题一部分的情况。

- A small set of pure functions over a plain data type. Good when there's no implicit current state, just transformations.

  一小组纯函数，作用于一个普通数据类型。适合没有隐式的当前状态、只有变换的情况。

- A class or module with a clear method surface when the logic genuinely owns ongoing internal state.

  一个具有清晰方法面的类或模块，适合逻辑确实持有持续的内部状态的情况。

Pick whichever shape best fits the question being asked, not whichever is easiest to wire to a page. Keep it pure: no DOM, no `document`, no button handlers reaching inside it. The page calls into it; nothing flows the other direction. This is what makes the prototype useful past its own lifetime: once the question's answered, the validated reducer / machine / function set lifts into the real module on its own.

选择最贴合当前问题的形态，而不是最容易接到页面上的那种。保持纯粹：不碰 DOM，不碰 `document`，也不允许按钮处理函数伸手进去。由页面调用它；不存在反方向的流动。正是这一点让原型在自身生命周期结束后依然有用：问题一旦得到回答，经过验证的 reducer / 状态机 / 函数集就能直接搬进真实模块。

3. Build the shareable HTML file 3. 构建可分享的 HTML 文件

One file, plain HTML/CSS/JS: no framework, no bundler, no server, everything inline so it opens by double-click and survives being emailed around. Anyone should be able to run it by opening it.

一个文件，纯 HTML/CSS/JS：不用框架，不用打包器，不用服务器，所有内容内联，双击即可打开，通过邮件转来转去也不会坏。任何人打开它就应该能运行。

Write it for a non-developer. Every label is in **domain language**, not code: buttons and state read like the business, not the reducer. Explain in plain words what's happening.

为非开发者而写。每个标签都使用**领域语言**，而不是代码：按钮和状态读起来要像业务，而不是像 reducer。用平实的话解释正在发生什么。

Lay it out with a clean hierarchy, top to bottom:

用清晰的层次从上到下排布：

1. **Title and one-line explanation** of what this demo lets you explore (the question from step 1).

   标题和一句话说明：这个演示让你探索什么（即第 1 步中的问题）。

2. **Current state**: the full relevant state, rendered as a readable panel (labelled fields, not a raw JSON dump), re-rendered after every click so the change is visible. Where it helps a non-developer follow, call out what just changed.

   当前状态：完整的相关状态，渲染成易读的面板（带标签的字段，而不是原始 JSON 转储），每次点击后重新渲染，让变化清晰可见。如果有助于非开发者跟上，就标出刚刚改变了什么。

3. **Free-play buttons**: one button per action, always available, so anyone can poke at the model in any order. Each click dispatches its action and re-renders the state.

   自由操作按钮：每个操作一个按钮，始终可用，让任何人都能以任意顺序戳一戳这个模型。每次点击都派发对应操作并重新渲染状态。

4. **Guided walkthroughs**: a set of **scenarios**, one per tab. Each tab holds a short plain-language description of the scenario (the situation it sets up and what to watch for) and underneath it, the ordered **buttons to press** for that scenario. Each step is a real button: clicking it performs that action and moves to the next step. Starting a walkthrough resets to a known initial state so the scenario runs the same way every time.

   引导式演练：一组**场景**，每个标签页一个。每个标签页包含一段简短的平实语言描述（这个场景设定了什么情境、需要留意什么），下方是该场景要依次**按下的按钮**。每一步都是真实的按钮：点击它就执行该操作并进入下一步。开始一次演练时会重置到已知的初始状态，确保场景每次都以同样的方式运行。

Choose scenarios that demonstrate the awkward cases, the ones hard to reason about on paper: the happy path, a tricky edge case, an attempt at something that should be illegal.

挑选那些能展示别扭情况的场景，也就是在纸面上难以推演的那些：正常路径、一个棘手的边界情况、一次本应非法的尝试。

Keep it beautiful but restrained: clean typography, generous spacing, one accent colour. No animations, no gimmicks: nothing that competes with the state and the buttons.

做得好看但要克制：干净的排版、宽松的留白、一种强调色。不要动画，不要花哨的噱头：任何东西都不应与状态和按钮争夺注意力。

4. Hand it over 4. 交付出去

Send them the file, or open it for them. They'll click through the walkthroughs and free-play whenever they get to it; the interesting moments are when they say "wait, that shouldn't be possible" or "huh, I assumed X would be different"; those are the bugs in the _idea_, which is the whole point. If they want new actions or a new scenario, add them. Prototypes evolve.

把文件发给他们，或者帮他们打开。他们有空时会点一遍演练、自由操作一番；有意思的时刻是他们说“等等，这不应该能做到”或者“咦，我原以为 X 会不一样”的时候；那些就是_想法_里的 bug，而这正是全部意义所在。如果他们想要新的操作或新的场景，就加上。原型是会演进的。

5. Capture the answer and the prototype 5. 留存答案和原型

Once the prototype has answered its question, capture the answer, then capture the prototype the way the SKILL describes. The logic-specific mapping: the validated reducer / machine / function set lifts into the real module (the decision, absorbed); the HTML shell rides along to the throwaway branch that keeps the prototype as a primary source, and being one self-contained file, it stays trivially re-runnable there.

原型回答了它的问题之后，先把答案留存下来，再按照 SKILL 所描述的方式留存原型。逻辑原型特有的对应关系是：经过验证的 reducer / 状态机 / 函数集搬进真实模块（决策被吸收）；HTML 外壳则随之进入那个一次性分支，在那里把原型保留为一手资料，而且由于它是一个自包含文件，在那里重新运行也毫不费力。

Anti-patterns 反模式

- Don't add tests. A prototype that needs tests is no longer a prototype.

  不要加测试。 需要测试的原型就不再是原型了。

- Don't wire it to the real database. Use in-memory state unless the question is specifically about persistence.

  不要接入真实数据库。 使用内存状态，除非问题本身恰好就是关于持久化的。

- Don't generalise. No "what if we wanted to support X later." The prototype answers one question.

  不要泛化。 不要“万一以后要支持 X 呢”。原型只回答一个问题。

- **Don't blur the logic and the page together.** If the pure module references the DOM, `document`, or button handlers, it's no longer liftable. Keep the page as a thin shell over a pure module.

  不要把逻辑和页面搅在一起。 如果纯模块引用了 DOM、`document` 或按钮处理函数，它就不再能被拎出来。让页面只是纯模块之上的一层薄壳。

- **Don't reach for a framework, bundler, or server.** One file the recipient double-clicks; a React app or a dev server defeats "shareable".

  不要动用框架、打包器或服务器。 一个让接收者双击即可打开的文件；React 应用或开发服务器会让“可分享”落空。

- **Don't ship the HTML shell into production.** The page is optimised for being clicked through by hand. The logic module behind it is the bit worth keeping.

  不要把 HTML 外壳发布到生产环境。 这个页面是为手动点击浏览而优化的。它背后的逻辑模块才是值得保留的部分。

REFERENCE 参考文档 · UI.md

UI Prototype UI 原型

Source 原文

Generate **several radically different UI variations** on a single route, switchable from a floating bottom bar. The user flips between variants in the browser, picks one (or steals bits from each), then throws the rest away.

在同一个路由上生成**若干截然不同的 UI 变体**，通过一个浮动底栏切换。用户在浏览器里来回翻看各个变体，挑出一个（或者从每个里各取一些），然后把其余的扔掉。

If the question is about logic/state rather than what something looks like, this is the wrong branch. Use LOGIC.md.

如果问题关乎逻辑/状态，而不是某样东西长什么样，那就走错分支了。请使用 LOGIC.md。

When this is the right shape 何时适合用这种形态

- "What should this page look like?"

  “这个页面应该长什么样？”

- "I want to see a few options for this dashboard before committing."

  “在拍板之前，我想先看看这个仪表盘的几种方案。”

- "Try a different layout for the settings screen."

  “给设置页面换一种布局试试。”

- Any time the user would otherwise spend a day picking between three vague mockups in their head.

  任何用户原本要花一整天在脑子里纠结三个模糊草图的时候。

Two sub-shapes: strongly prefer sub-shape A 两种子形态：强烈优先选择子形态 A

A UI prototype is much easier to judge when it's **butting up against the rest of the app**: real header, real sidebar, real data, real density. A throwaway route on its own is a vacuum: every variant looks fine in isolation. Default to sub-shape A whenever there's a plausible existing page to host the variants. Only reach for sub-shape B if the prototype genuinely has no nearby home.

当 UI 原型**紧贴着应用的其余部分**时，评判起来要容易得多：真实的页头、真实的侧边栏、真实的数据、真实的信息密度。孤立的一次性路由就是一个真空：每个变体单独看都还行。只要有一个说得过去的现有页面可以承载这些变体，就默认使用子形态 A。只有当原型确实在附近找不到归宿时，才考虑子形态 B。

Sub-shape A: adjustment to an existing page (preferred) 子形态 A：调整现有页面（首选）

The route already exists. Variants are rendered **on the same route**, gated by a `?variant=` URL search param. The existing data fetching, params, and auth all stay. Only the rendering swaps. This is the default; pick it unless there's a specific reason not to.

路由已经存在。变体**在同一路由上**渲染，由 `?variant=` URL 查询参数控制。现有的数据获取、参数和鉴权全部保留。只替换渲染部分。这是默认选项；除非有具体理由，否则就选它。

If the prototype is for something that doesn't yet have a page but would naturally live inside one (a new section of the dashboard, a new card on the settings screen, a new step in an existing flow), it's still sub-shape A. Mount the variants inside the host page.

如果原型针对的东西还没有自己的页面，但自然会存在于某个页面之中（仪表盘的一个新区块、设置页面上的一张新卡片、现有流程中的一个新步骤），那它仍然属于子形态 A。把变体挂载到宿主页面里。

Sub-shape B: a new page (last resort) 子形态 B：新页面（最后手段）

Only use this when the thing being prototyped genuinely has no existing page to live inside (e.g. an entirely new top-level surface, or a flow that can't be embedded anywhere sensible).

只有当被原型化的东西确实没有任何现有页面可以容身时才使用（例如一个全新的顶层界面，或一个无法合理嵌入任何地方的流程）。

Create a **throwaway route** following whatever routing convention the project already uses. Don't invent a new top-level structure. Name it so it's obviously a prototype (e.g. include the word `prototype` in the path or filename). Same `?variant=` pattern.

按照项目已有的路由约定创建一个**一次性路由**。不要另起一套新的顶层结构。命名要让人一眼看出这是原型（例如在路径或文件名中包含 `prototype` 一词）。同样使用 `?variant=` 模式。

Before committing to sub-shape B, sanity-check: is there really no existing page this could be embedded in? An empty route hides design problems that a populated one would expose.

在决定采用子形态 B 之前，先核实一下：真的没有任何现有页面可以嵌入吗？一个空路由会掩盖那些在内容充实的页面上本会暴露出来的设计问题。

In both sub-shapes the floating bottom bar is identical.

两种子形态下的浮动底栏完全相同。

Process 流程

1. State the question and pick N 1. 陈述问题并选定 N

Default to **3 variants**. More than 5 stops being radically different and starts being noise, so cap there.

默认 **3 个变体**。超过 5 个就不再是截然不同，而开始变成噪音，所以上限定在 5。

Write down the plan in one line, in the prototype's location or a top-of-file comment:

用一句话写下计划，放在原型所在位置或文件顶部的注释里：

> "Three variants of the settings page, switchable via `?variant=`, on the existing `/settings` route."

> “设置页面的三个变体，可通过 `?variant=` 切换，位于现有的 `/settings` 路由上。”

This works whether the user is here to push back or not.

无论用户是否在场提出异议，这样做都行得通。

2. Generate radically different variants 2. 生成截然不同的变体

Draft each variant. Hold each one to:

逐个起草变体。每个变体都要遵守：

- The page's purpose and the data it has access to.

  页面的用途以及它能访问的数据。

- The project's component library / styling system (TailwindCSS, shadcn, MUI, plain CSS, whatever).

  项目的组件库 / 样式体系（TailwindCSS、shadcn、MUI、纯 CSS，不管是什么）。

- A clear exported component name, e.g. `VariantA`, `VariantB`, `VariantC`.

  一个清晰的导出组件名，例如 `VariantA`、`VariantB`、`VariantC`。

Variants must be **structurally different**: different layout, different information hierarchy, different primary affordance, not just different colours. Three slightly-tweaked card grids isn't a UI prototype, it's wallpaper. If two drafts come out too similar, redo one with explicit "do not use a card grid" guidance.

变体必须在**结构上不同**：不同的布局、不同的信息层级、不同的主要交互方式，而不只是颜色不同。三个稍作调整的卡片网格不是 UI 原型，那是墙纸。如果两份草稿出来太相似，就带着“不要使用卡片网格”这样的明确指引重做其中一个。

3. Wire them together 3. 把它们串起来

Create a single switcher component on the route:

在路由上创建一个单一的切换器组件：

```
// pseudo-code, adapt to the project's framework
const variant = searchParams.get('variant') ?? 'A';
return (
  <>
    {variant === 'A' && <VariantA {...data} />}
    {variant === 'B' && <VariantB {...data} />}
    {variant === 'C' && <VariantC {...data} />}
    <PrototypeSwitcher variants={['A','B','C']} current={variant} />
  </>
);
```


```
// pseudo-code, adapt to the project's framework
const variant = searchParams.get('variant') ?? 'A';
return (
  <>
    {variant === 'A' && <VariantA {...data} />}
    {variant === 'B' && <VariantB {...data} />}
    {variant === 'C' && <VariantC {...data} />}
    <PrototypeSwitcher variants={['A','B','C']} current={variant} />
  </>
);
```


For sub-shape A (existing page): keep all the existing data fetching above the switcher; only the rendered subtree changes per variant.

对于子形态 A（现有页面）：把所有现有的数据获取保留在切换器之上；每个变体只改变被渲染的子树。

For sub-shape B (new page): the throwaway route under `/prototype/<name>` mounts the same switcher.

对于子形态 B（新页面）：位于 `/prototype/<name>` 下的一次性路由挂载同一个切换器。

4. Build the floating switcher 4. 构建浮动切换器

A small fixed-position bar at the bottom-centre of the screen with three pieces:

在屏幕底部居中放一个固定定位的小栏，包含三部分：

- Left arrow: cycles to the previous variant (wraps around).

  左箭头：切换到上一个变体（循环）。

- Variant label: shows the current variant key and, if the variant exports a name, that name too. e.g. `B (Sidebar layout)`.

  变体标签：显示当前变体的键名；如果该变体导出了名称，也一并显示。例如 `B (Sidebar layout)`。

- Right arrow: cycles forward (wraps around).

  右箭头：向后切换（循环）。

Behaviour:

行为：

- Clicking an arrow updates the URL search param (use the framework's router, e.g. `router.replace` on Next, `navigate` on React Router, etc) so the variant is shareable and reload-stable.

  点击箭头会更新 URL 查询参数（使用框架的路由器，例如 Next 上用 `router.replace`，React Router 上用 `navigate`，等等），这样变体可以分享，刷新后也保持不变。

- Keyboard: `←` and `→` arrow keys also cycle. Don't intercept arrow keys when an `<input>`, `<textarea>`, or `[contenteditable]` is focused.

  键盘：`←` 和 `→` 方向键同样可以切换。当 `<input>`、`<textarea>` 或 `[contenteditable]` 处于聚焦状态时，不要拦截方向键。

- Visually distinct from the page (e.g. high-contrast pill, subtle shadow) so it's obviously not part of the design being evaluated.

  在视觉上与页面区分开（例如高对比度的胶囊形状、淡淡的阴影），让人一看就知道它不属于正在评估的设计。

- Hidden in production builds: gate on `process.env.NODE_ENV !== 'production'` or an equivalent check, so a stray prototype merge can't ship the bar to users.

  在生产构建中隐藏：用 `process.env.NODE_ENV !== 'production'` 或等效的检查加以控制，这样即使原型被误合并，也不会把这个栏发布给用户。

Put the switcher in a single shared component so both sub-shapes can reuse it. Locate it wherever shared UI lives in the project.

把切换器放进一个共享组件里，让两种子形态都能复用。放在项目中共享 UI 所在的位置即可。

5. Hand it over 5. 交付出去

Surface the URL (and the `?variant=` keys). The user will flip through whenever they get to it. The interesting feedback is usually "I want the header from B with the sidebar from C", which is the actual design they want.

给出 URL（以及各个 `?variant=` 键）。用户有空时会翻看。最有价值的反馈通常是“**我想要 B 的页头配 C 的侧边栏**”，那才是他们真正想要的设计。

6. Capture the answer and clean up 6. 留存答案并清理

Once a variant has won, capture the answer (which variant and why), then capture the prototype the way the SKILL describes. Fold the winner into the real code and move the rest onto the throwaway branch, not into main:

某个变体胜出之后，先留存答案（哪个变体以及为什么），再按照 SKILL 所描述的方式留存原型。把胜出者并入真实代码，其余的移到一次性分支上，而不是留在 main 里：

- Sub-shape A: fold the winner into the existing page; drop the losing variants and the switcher from main.

  子形态 A：把胜出者并入现有页面；从 main 中删掉落选的变体和切换器。

- Sub-shape B: promote the winning variant to a real route; drop the throwaway route and the switcher from main.

  子形态 B：把胜出的变体提升为正式路由；从 main 中删掉一次性路由和切换器。

The full set of variants is the primary source, so it lands on the throwaway branch, not the bin, since variant components and the switcher left in the main branch rot fast and confuse the next reader.

完整的变体集合是一手资料，所以它进入一次性分支，而不是垃圾桶；之所以不留在 main 分支，是因为留在那里的变体组件和切换器会很快腐烂，并让下一位读者困惑。

Anti-patterns 反模式

- Variants that differ only in colour or copy. That's a tweak, not a prototype. Real variants disagree about structure.

  变体之间只有颜色或文案不同。 那是微调，不是原型。真正的变体在结构上各执己见。

- Sharing too much code between variants. A shared `<Header>` is fine; a shared `<Layout>` defeats the point. Each variant should be free to throw out the layout.

  变体之间共享过多代码。 共享一个 `<Header>` 没问题；共享一个 `<Layout>` 就失去了意义。每个变体都应该可以自由地推翻布局。

- Wiring variants to real mutations. Read-only prototypes are fine. If a variant needs to mutate, point it at a stub: the question is "what should this look like", not "does the backend work".

  把变体接到真实的数据变更上。 只读原型没问题。如果某个变体需要做变更，就把它指向一个桩：问题是“这个应该长什么样”，而不是“后端能不能用”。

- Promoting the prototype directly to production. The variant code was written under prototype constraints (no tests, minimal error handling). Rewrite it properly when you fold it in.

  把原型直接提升到生产环境。 变体代码是在原型约束下写的（没有测试，错误处理极少）。并入时要好好重写。

CHAPTER 14 · SKILL

Research
调研

Investigate a question against high-trust primary sources and capture the findings as a Markdown file in the repo. Use when the user wants a topic researched, docs or API facts gathered, or reading legwork delegated to a background agent.

针对高可信度的一手来源调查一个问题，并把发现记录为仓库中的一个 Markdown 文件。当用户想要研究某个主题、收集文档或 API 方面的事实，或把阅读类的跑腿工作委派给后台智能体时使用。

Source 原文 · v1.3.1 · b0618bc4

Spin up a **background agent** to do the research, so you keep working while it reads.

启动一个**后台智能体**来做研究，这样在它阅读时你可以继续工作。

Its job:

它的任务：

1. Investigate the question against **primary sources** (official docs, source code, specs, first-party APIs), not a secondary write-up of them. Follow every claim back to the source that owns it.

   针对**一手来源**（官方文档、源代码、规范文档、第一方 API）调查这个问题，而不是针对对它们的二手总结。把每一条论断都追溯到拥有它的那个来源。

2. Write the findings to a single Markdown file, citing each claim's source.

   把发现写进单个 Markdown 文件，为每条论断注明来源。

3. Save it where the repo already keeps such notes; match the existing convention, and if there is none, put it somewhere sensible and say where.

   把它保存在仓库已经存放此类笔记的地方；遵循现有约定，如果没有约定，就放在一个合理的位置，并说明放在了哪里。

CHAPTER 15 · SKILL

Domain Modeling
领域建模

Build and sharpen a project's domain model. Use when discussing codebase terminology, writing or editing a GLOSSARY.md, or recording or editing an ADR.

构建并打磨项目的领域模型。在讨论代码库术语、编写或编辑 GLOSSARY.md，或者记录或编辑 ADR（架构决策记录）时使用。

Source 原文 · v1.3.1 · b0618bc4

Actively build and sharpen the project's domain model as you design. This is the active discipline: challenging terms, inventing edge-case scenarios, and writing the glossary and decisions down the moment they crystallise. (Merely reading `GLOSSARY.md` for vocabulary is not this skill: that's a one-line habit any skill can do. This skill is for when you're changing the model, not just consuming it.)

在设计过程中主动构建并打磨项目的领域模型。这是一门主动的功夫：质疑术语、构造边界情况场景，并在术语表和决策一成形时就立刻写下来。（仅仅为了词汇而阅读 `GLOSSARY.md` 不属于本技能：那是任何技能都能做到的一行习惯。本技能用于你在修改模型、而不只是使用模型的时候。）

File structure 文件结构

Most repos have a single context:

大多数仓库只有一个上下文：

```
/
├── GLOSSARY.md
├── docs/
│   └── adr/
│       ├── 0001-event-sourced-orders.md
│       └── 0002-postgres-for-write-model.md
└── src/
```


```
/
├── GLOSSARY.md
├── docs/
│   └── adr/
│       ├── 0001-event-sourced-orders.md
│       └── 0002-postgres-for-write-model.md
└── src/
```


If a `GLOSSARY-MAP.md` exists at the root, the repo has multiple contexts. The map points to where each one lives:

如果根目录存在 `GLOSSARY-MAP.md`，说明该仓库有多个上下文。这份映射指出每个上下文位于何处：

```
/
├── GLOSSARY-MAP.md
├── docs/
│   └── adr/                    ← system-wide decisions
├── src/
│   ├── ordering/
│   │   ├── GLOSSARY.md
│   │   └── docs/adr/           ← context-specific decisions
│   └── billing/
│       ├── GLOSSARY.md
│       └── docs/adr/
```

```
/
├── GLOSSARY-MAP.md
├── docs/
│   └── adr/                    ← 全系统范围的决策
├── src/
│   ├── ordering/
│   │   ├── GLOSSARY.md
│   │   └── docs/adr/           ← 特定于该上下文的决策
│   └── billing/
│       ├── GLOSSARY.md
│       └── docs/adr/
```

Create files lazily: only when you have something to write. If no `GLOSSARY.md` exists, create one when the first term is resolved. If no `docs/adr/` exists, create it when the first ADR is needed.

惰性创建文件：只在确实有内容要写时才创建。如果 `GLOSSARY.md` 不存在，就在第一个术语被确定时创建它。如果 `docs/adr/` 不存在，就在第一次需要 ADR 时创建它。

During the session 会话进行中

Challenge against the glossary 对照术语表提出质疑

When the user uses a term that conflicts with the existing language in `GLOSSARY.md`, call it out immediately. "Your glossary defines 'cancellation' as X, but you seem to mean Y. Which is it?"

当用户使用的术语与 `GLOSSARY.md` 中已有的用语冲突时，立即指出来。“你的术语表把‘cancellation’定义为 X，但你似乎指的是 Y。到底是哪一个？”

Sharpen fuzzy language 打磨含糊的用语

When the user uses vague or overloaded terms, propose a precise canonical term. "You're saying 'account': do you mean the Customer or the User? Those are different things."

当用户使用含糊或一词多义的术语时，提出一个精确的规范术语。“你说的是‘account’：你指的是 Customer 还是 User？这是两回事。”

Discuss concrete scenarios 讨论具体场景

When domain relationships are being discussed, stress-test them with specific scenarios. Invent scenarios that probe edge cases and force the user to be precise about the boundaries between concepts.

在讨论领域关系时，用具体场景对其进行压力测试。构造能探查边界情况的场景，迫使用户把概念之间的边界说清楚。

Cross-reference with code 与代码交叉核对

When the user states how something works, check whether the code agrees. If you find a contradiction, surface it: "Your code cancels entire Orders, but you just said partial cancellation is possible. Which is right?"

当用户陈述某样东西如何运作时，检查代码是否与之一致。如果发现矛盾，就把它摆到台面上：“你的代码会取消整个 Order，但你刚才说可以部分取消。哪个才对？”

Update GLOSSARY.md inline 随时更新 GLOSSARY.md

When a term is resolved, update `GLOSSARY.md` right there. Don't batch these up: capture them as they happen. Use the format in GLOSSARY-FORMAT.md.

一个术语一旦确定，就当场更新 `GLOSSARY.md`。不要攒起来批量处理：发生一个就记录一个。使用 GLOSSARY-FORMAT.md 中的格式。

`GLOSSARY.md` should be totally devoid of implementation details. Do not treat `GLOSSARY.md` as a spec, a scratch pad, or a repository for implementation decisions. It is a glossary and nothing else.

`GLOSSARY.md` 中应完全不含任何实现细节。不要把 `GLOSSARY.md` 当作规范、草稿本或实现决策的存放处。它就是一份术语表，别无其他。

Offer ADRs sparingly 谨慎地提议 ADR

Only offer to create an ADR when all three are true:

只有在以下三条全部成立时，才提议创建 ADR：

1. Hard to reverse: the cost of changing your mind later is meaningful

   难以逆转：日后改变主意的代价不可忽视

2. Surprising without context: a future reader will wonder "why did they do it this way?"

   缺乏上下文时令人意外：未来的读者会疑惑“他们为什么要这样做？”

3. The result of a real trade-off: there were genuine alternatives and you picked one for specific reasons

   真实权衡的结果：确实存在其他备选方案，而你出于特定理由选了其中之一

If any of the three is missing, skip the ADR. Use the format in ADR-FORMAT.md.

三条中任何一条不满足，就跳过 ADR。使用 ADR-FORMAT.md 中的格式。

REFERENCE 参考文档 · ADR-FORMAT.md

ADR Format ADR 格式

Source 原文

ADRs live in `docs/adr/` and use sequential numbering: `0001-slug.md`, `0002-slug.md`, etc.

ADR（架构决策记录）存放在 `docs/adr/` 中，并按顺序编号：`0001-slug.md`、`0002-slug.md`，依此类推。

Create the `docs/adr/` directory lazily: only when the first ADR is needed.

惰性创建 `docs/adr/` 目录：只在第一次需要 ADR 时才创建。

Template 模板

```
# {Short title of the decision}

{1-3 sentences: what's the context, what did we decide, and why.}
```

```
# {决策的简短标题}

{1-3 句话：背景是什么，我们决定了什么，以及为什么。}
```

That's it. An ADR can be a single paragraph. The value is in recording that a decision was made and why, not in filling out sections.

就这么多。一份 ADR 可以只有一段话。它的价值在于记录确实做出了某个决策以及为什么，而不在于把各个小节填满。

Optional sections 可选小节

Only include these when they add genuine value. Most ADRs won't need them.

只有在它们能带来真正价值时才加入。大多数 ADR 都用不上。

- Status frontmatter (`proposed | accepted | deprecated | superseded by ADR-NNNN`): useful when decisions are revisited

  Status frontmatter（`proposed | accepted | deprecated | superseded by ADR-NNNN`）：在决策会被重新审视时有用

- Considered Options: only when the rejected alternatives are worth remembering

  Considered Options：仅当被否决的备选方案值得记住时

- Consequences: only when non-obvious downstream effects need to be called out

  Consequences：仅当需要特别指出不明显的下游影响时

Numbering 编号

Scan `docs/adr/` for the highest existing number and increment by one.

扫描 `docs/adr/`，找到现有的最大编号，然后加一。

When to offer an ADR 何时提议 ADR

All three of these must be true:

以下三条必须全部成立：

1. Hard to reverse: the cost of changing your mind later is meaningful

   难以逆转：日后改变主意的代价不可忽视

2. Surprising without context: a future reader will look at the code and wonder "why on earth did they do it this way?"

   缺乏上下文时令人意外：未来的读者看到代码时会纳闷“他们到底为什么要这样做？”

3. The result of a real trade-off: there were genuine alternatives and you picked one for specific reasons

   真实权衡的结果：确实存在其他备选方案，而你出于特定理由选了其中之一

If a decision is easy to reverse, skip it: you'll just reverse it. If it's not surprising, nobody will wonder why. If there was no real alternative, there's nothing to record beyond "we did the obvious thing."

如果一个决策很容易逆转，就跳过它：到时候直接逆转就行。如果它并不令人意外，就没人会疑惑为什么。如果根本没有真正的备选方案，那么除了“我们做了显而易见的事”之外就没什么可记录的。

What qualifies 哪些够格

- Architectural shape. "We're using a monorepo." "The write model is event-sourced, the read model is projected into Postgres."

  架构形态。 “我们使用 monorepo。” “写模型采用事件溯源，读模型投影到 Postgres 中。”

- Integration patterns between contexts. "Ordering and Billing communicate via domain events, not synchronous HTTP."

  上下文之间的集成模式。 “Ordering 和 Billing 通过领域事件通信，而不是同步 HTTP。”

- Technology choices that carry lock-in. Database, message bus, auth provider, deployment target. Not every library: just the ones that would take a quarter to swap out.

  会带来锁定的技术选型。 数据库、消息总线、认证提供方、部署目标。并非每个库都算：只算那些需要花一个季度才能替换掉的。

- **Boundary and scope decisions.** "Customer data is owned by the Customer context; other contexts reference it by ID only." The explicit no-s are as valuable as the yes-s.

  边界与范围决策。 “Customer 数据归 Customer 上下文所有；其他上下文只通过 ID 引用它。”明确的“不”和“是”同样有价值。

- **Deliberate deviations from the obvious path.** "We're using manual SQL instead of an ORM because X." Anything where a reasonable reader would assume the opposite. These stop the next engineer from "fixing" something that was deliberate.

  刻意偏离显而易见的路径。 “我们使用手写 SQL 而不是 ORM，因为 X。”任何一个理性读者会默认相反做法的地方都算。这些记录能阻止下一位工程师去“修复”本来就是刻意为之的东西。

- **Constraints not visible in the code.** "We can't use AWS because of compliance requirements." " Response times must be under 200ms because of the partner API contract."

  代码中看不到的约束。 “由于合规要求，我们不能使用 AWS。”“由于合作方 API 合同，响应时间必须低于 200ms。”

- **Rejected alternatives when the rejection is non-obvious.** If you considered GraphQL and picked REST for subtle reasons, record it; otherwise someone will suggest GraphQL again in six months.

  否决理由不明显的备选方案。 如果你考虑过 GraphQL，却出于微妙的原因选择了 REST，就把它记下来；否则六个月后又会有人提议用 GraphQL。

REFERENCE 参考文档 · GLOSSARY-FORMAT.md

GLOSSARY.md Format GLOSSARY.md 格式

Source 原文

Structure 结构

```
# {Context Name}

{One or two sentence description of what this context is and why it exists.}

## Language

**Order**:
{A one or two sentence description of the term}
_Avoid_: Purchase, transaction

**Invoice**:
A request for payment sent to a customer after delivery.
_Avoid_: Bill, payment request

**Customer**:
A person or organization that places orders.
_Avoid_: Client, buyer, account
```

```
# {上下文名称}

{用一两句话描述这个上下文是什么、为什么存在。}

## Language

**Order**:
{用一两句话描述该术语}
_Avoid_: Purchase, transaction

**Invoice**:
A request for payment sent to a customer after delivery.
_Avoid_: Bill, payment request

**Customer**:
A person or organization that places orders.
_Avoid_: Client, buyer, account
```

Rules 规则

- **Be opinionated.** When multiple words exist for the same concept, pick the best one and list the others under `_Avoid_`.

  要有主见。 当同一个概念有多个词可用时，挑出最好的那个，把其余的列在 `_Avoid_` 下。

- **Keep definitions tight.** One or two sentences max. Define what it IS, not what it does.

  定义要紧凑。 最多一两句话。定义它是什么，而不是它做什么。

- **Only include terms specific to this project's context.** General programming concepts (timeouts, error types, utility patterns) don't belong even if the project uses them extensively. Before adding a term, ask: is this a concept unique to this context, or a general programming concept? Only the former belongs.

  只收录本项目上下文特有的术语。 通用编程概念（超时、错误类型、工具类模式）即便项目大量使用也不应收录。添加术语之前先问：这是本上下文独有的概念，还是通用编程概念？只有前者才应收录。

- Group terms under subheadings when natural clusters emerge. If all terms belong to a single cohesive area, a flat list is fine.

  出现自然的聚类时，用子标题对术语分组。 如果所有术语都属于同一个内聚的领域，用平铺列表即可。

Single vs multi-context repos 单上下文与多上下文仓库

Single context (most repos): One `GLOSSARY.md` at the repo root.

单上下文（大多数仓库）： 在仓库根目录放一个 `GLOSSARY.md`。

Multiple contexts: A `GLOSSARY-MAP.md` at the repo root lists the contexts, where they live, and how they relate to each other:

多上下文： 在仓库根目录放一个 `GLOSSARY-MAP.md`，列出各个上下文、它们位于何处以及彼此之间的关系：

```
# Glossary Map

## Contexts

- [Ordering](./src/ordering/GLOSSARY.md): receives and tracks customer orders
- [Billing](./src/billing/GLOSSARY.md): generates invoices and processes payments
- [Fulfillment](./src/fulfillment/GLOSSARY.md): manages warehouse picking and shipping

## Relationships

- **Ordering → Fulfillment**: Ordering emits `OrderPlaced` events; Fulfillment consumes them to start picking
- **Fulfillment → Billing**: Fulfillment emits `ShipmentDispatched` events; Billing consumes them to generate invoices
- **Ordering ↔ Billing**: Shared types for `CustomerId` and `Money`
```


```
# Glossary Map

## Contexts

- [Ordering](./src/ordering/GLOSSARY.md): receives and tracks customer orders
- [Billing](./src/billing/GLOSSARY.md): generates invoices and processes payments
- [Fulfillment](./src/fulfillment/GLOSSARY.md): manages warehouse picking and shipping

## Relationships

- **Ordering → Fulfillment**: Ordering emits `OrderPlaced` events; Fulfillment consumes them to start picking
- **Fulfillment → Billing**: Fulfillment emits `ShipmentDispatched` events; Billing consumes them to generate invoices
- **Ordering ↔ Billing**: Shared types for `CustomerId` and `Money`
```


The skill infers which structure applies:

本技能会推断适用哪种结构：

- If `GLOSSARY-MAP.md` exists, read it to find contexts

  如果存在 `GLOSSARY-MAP.md`，读取它以找到各个上下文

- If only a root `GLOSSARY.md` exists, single context

  如果只存在根目录下的 `GLOSSARY.md`，则为单上下文

- If neither exists, create a root `GLOSSARY.md` lazily when the first term is resolved

  如果两者都不存在，就在第一个术语被确定时惰性创建根目录下的 `GLOSSARY.md`

When multiple contexts exist, infer which one the current topic relates to. If unclear, ask.

当存在多个上下文时，推断当前话题属于哪一个。如果不清楚，就问。

CHAPTER 16 · SKILL

Codebase Design
代码库设计

Shared vocabulary for designing deep modules. Use when the user wants to design or improve a module's interface, find deepening opportunities, decide where a seam goes, make code more testable or AI-navigable, or when another skill needs the deep-module vocabulary.

设计深模块的共享词汇。当用户想要设计或改进模块的接口、寻找加深机会、决定接缝放在哪里、让代码更可测试或更便于 AI 导航，或者其他技能需要深模块词汇时使用。

Source 原文 · v1.3.1 · b0618bc4

Design **deep modules**: a lot of behaviour behind a small interface, placed at a clean seam, testable through that interface. Use this language and these principles wherever code is being designed or restructured. The aim is leverage for callers, locality for maintainers, and testability for everyone.

设计**深模块**：在一个小接口背后放大量行为，置于干净的接缝处，并可通过该接口进行测试。凡是在设计或重构代码的地方，都使用这套语言和这些原则。目标是：为调用方提供杠杆，为维护者提供局部性，为所有人提供可测试性。

Glossary 术语表

Use these terms exactly: don't substitute "component," "service," "API," or "boundary." Consistent language is the whole point.

严格使用这些术语：不要用“组件”“服务”“API”或“边界”来替代。语言一致正是全部意义所在。

Module: anything with an interface and an implementation. Deliberately scale-agnostic: a function, class, package, or tier-spanning slice. _Avoid_: unit, component, service.

模块（Module）：任何具有接口和实现的东西。刻意不限定规模：一个函数、类、包，或一个跨越多个层级的切片。避免使用：单元（unit）、组件（component）、服务（service）。

Interface: everything a caller must know to use the module correctly: the type signature, but also invariants, ordering constraints, error modes, required configuration, and performance characteristics. _Avoid_: API, signature (too narrow, they refer only to the type-level surface).

接口（Interface）：调用方要正确使用该模块所必须知道的一切：类型签名，还包括不变式、顺序约束、错误模式、必需的配置以及性能特征。避免使用：API、签名（signature）（太窄，它们只指类型层面的表面）。

Implementation: what's inside a module, its body of code. Distinct from **Adapter**: a thing can be a small adapter with a large implementation (a Postgres repo) or a large adapter with a small implementation (an in-memory fake). Reach for "adapter" when the seam is the topic; "implementation" otherwise.

实现（Implementation）：模块内部的东西，即它的代码主体。与**适配器**不同：一个东西可以是适配器小而实现大（一个 Postgres 仓储），也可以是适配器大而实现小（一个内存假实现）。当话题是接缝时用“适配器”；其他情况用“实现”。

Depth: leverage at the interface. The amount of behaviour a caller (or test) can exercise per unit of interface they have to learn. A module is **deep** when a large amount of behaviour sits behind a small interface, **shallow** when the interface is nearly as complex as the implementation.

深度（Depth）：接口处的杠杆。调用方（或测试）每学习一单位接口所能调动的行为量。当大量行为位于一个小接口之后时，模块是**深**的；当接口几乎和实现一样复杂时，模块是**浅**的。

Seam _(Michael Feathers)_: a place where you can alter behaviour without editing in that place; the location at which a module's interface lives. Where to put the seam is its own design decision, distinct from what goes behind it. _Avoid_: boundary (overloaded with DDD's bounded context).

接缝（Seam）（Michael Feathers）：一个无需在该处修改代码就能改变行为的位置；即模块接口所在的位置。接缝放在哪里本身就是一个设计决策，与接缝背后放什么相互独立。避免使用：边界（boundary）（与 DDD 的限界上下文（bounded context）含义重叠）。

Adapter: a concrete thing that satisfies an interface at a seam. Describes role (what slot it fills), not substance (what's inside).

适配器（Adapter）：在接缝处满足某个接口的具体事物。描述的是角色（它填补哪个槽位），而不是实质（内部是什么）。

Leverage: what callers get from depth. More capability per unit of interface they learn. One implementation pays back across N call sites and M tests.

杠杆（Leverage）：调用方从深度中获得的东西。每学习一单位接口就能获得更多能力。一份实现在 N 个调用点和 M 个测试中反复带来回报。

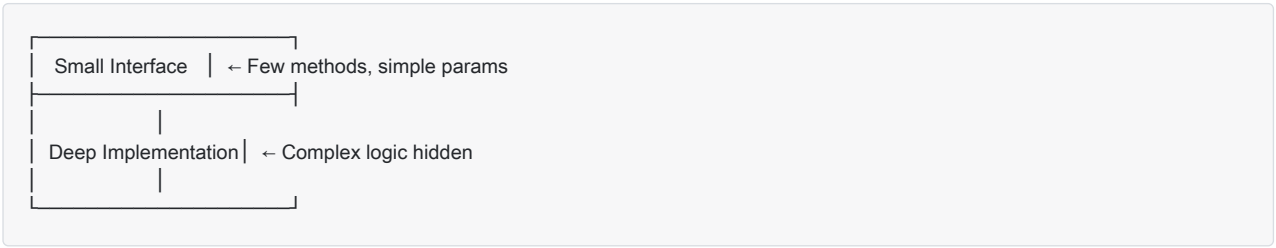
Locality: what maintainers get from depth. Change, bugs, knowledge, and verification concentrate in one place rather than spreading across callers. Fix once, fixed everywhere.

局部性（Locality）：维护者从深度中获得的东西。变更、bug、知识和验证都集中在一处，而不是散布到各个调用方。修一次，处处修好。

Deep vs shallow 深与浅

Deep module = small interface + lots of implementation:

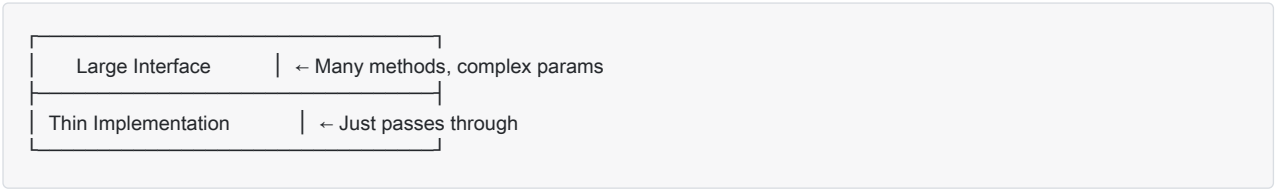
深模块 = 小接口 + 大量实现：



Small Interface ← Few methods, simple params
Deep Implementation ← Complex logic hidden

Shallow module = large interface + little implementation (avoid):

浅模块 = 大接口 + 少量实现（应避免）：



Large Interface ← Many methods, complex params
Thin Implementation ← Just passes through

When designing an interface, ask:

设计接口时，问自己：

- Can I reduce the number of methods?

  能减少方法的数量吗？
- Can I simplify the parameters?

  能简化参数吗？
- Can I hide more complexity inside?

  能把更多复杂度藏到内部吗？

Principles 原则

- **Depth is a property of the interface, not the implementation.** A deep module can be internally composed of small, mockable, swappable parts; they just aren't part of the interface. A module can have **internal seams** (private to its implementation, used by its own tests) as well as the **external seam** at its interface.

  深度是接口的属性，而不是实现的属性。 深模块内部可以由小的、可 mock（模拟）的、可替换的部件组成；只是这些部件不属于接口。一个模块既可以有**内部接缝**（为其实现所私有，供它自己的测试使用），也可以有位于其接口处的**外部接缝**。

- **The deletion test.** Imagine deleting the module. If complexity vanishes, it was a pass-through. If complexity reappears across N callers, it was earning its keep.

  删除测试。 设想把这个模块删掉。如果复杂度随之消失，它只是个直通层。如果复杂度在 N 个调用方中重新冒出来，它就是在发挥价值。

- **The interface is the test surface.** Callers and tests cross the same seam. If you want to test past the interface, the module is probably the wrong shape.

  接口即测试面。 调用方和测试跨越的是同一个接缝。如果你想越过接口去测试，这个模块的形状多半不对。

- **One adapter means a hypothetical seam. Two adapters means a real one.** Don't introduce a seam unless something actually varies across it.

  一个适配器意味着假想的接缝。两个适配器才意味着真实的接缝。 除非确实有东西会在接缝两侧变化，否则不要引入接缝。

Designing for testability 为可测试性而设计

Good interfaces make testing natural:

好的接口让测试水到渠成：

1. **Accept dependencies, don't create them.**

   接收依赖，而不是自己创建依赖。

```
// Testable
function processOrder(order, paymentGateway) {}

// Hard to test
function processOrder(order) {
  const gateway = new StripeGateway();
}
```


```
// Testable
function processOrder(order, paymentGateway) {}

// Hard to test
function processOrder(order) {
  const gateway = new StripeGateway();
}
```

2. Return results, don't produce side effects.

   返回结果，而不是产生副作用。

```
// Testable
function calculateDiscount(cart): Discount {}

// Hard to test
function applyDiscount(cart): void {
  cart.total -= discount;
}
```


```
// Testable
function calculateDiscount(cart): Discount {}

// Hard to test
function applyDiscount(cart): void {
  cart.total -= discount;
}
```


3. Small surface area. Fewer methods = fewer tests needed. Fewer params = simpler test setup.

   表面积小。 方法越少 = 需要的测试越少。参数越少 = 测试准备越简单。

Relationships 关系

- A Module has exactly one Interface (the surface it presents to callers and tests).

  一个**模块**恰好有一个**接口**（它向调用方和测试呈现的表面）。

- Depth is a property of a Module, measured against its Interface.

  深度是**模块**的属性，相对于其**接口**来衡量。

- A Seam is where a Module's Interface lives.

  接缝是**模块**的**接口**所在之处。

- An Adapter sits at a Seam and satisfies the Interface.

  适配器位于**接缝**处，并满足**接口**。

- Depth produces Leverage for callers and Locality for maintainers.

  深度为调用方带来**杠杆**，为维护者带来**局部性**。

Rejected framings 被否定的框架

- Depth as ratio of implementation-lines to interface-lines (Ousterhout): rewards padding the implementation. We use depth-as-leverage instead.

  把深度视为实现行数与接口行数之比（Ousterhout）：这会奖励往实现里注水。我们改用“深度即杠杆”。

- "Interface" as the TypeScript `interface` keyword or a class's public methods: too narrow: interface here includes every fact a caller must know.

  把“接口”理解为 TypeScript 的 `interface` 关键字或类的公共方法：太窄了：这里的接口包括调用方必须知道的每一个事实。

- "Boundary": overloaded with DDD's bounded context. Say seam or interface.

  “**边界**”（Boundary）：与 DDD 的限界上下文含义重叠。请说**接缝**或**接口**。

Going deeper 更进一步

- Deepening a cluster given its dependencies, see DEEPENING.md: dependency categories, seam discipline, and replace-don't-layer testing.

  在给定依赖的情况下加深一组模块，参见 DEEPENING.md：依赖类别、接缝纪律，以及“替换而非叠加”的测试策略。

- Exploring alternative interfaces, see DESIGN-IT-TWICE.md: spin up parallel sub-agents to design the interface several radically different ways, then compare on depth, locality, and seam placement.

  探索不同的接口方案，参见 DESIGN-IT-TWICE.md：并行启动多个子智能体，用几种截然不同的方式设计接口，然后从深度、局部性和接缝位置几个方面进行比较。

REFERENCE 参考文档 · DEEPENING.md

Deepening 加深

Source 原文

How to deepen a cluster of shallow modules safely, given its dependencies. Assumes the vocabulary in SKILL.md: module, interface, seam, adapter.

在给定依赖的情况下，如何安全地加深一组浅模块。假定你已掌握 SKILL.md 中的词汇：**模块**、**接口**、**接缝**、**适配器**。

Dependency categories 依赖类别

When assessing a candidate for deepening, classify its dependencies. The category determines how the deepened module is tested across its seam.

评估一个加深候选项时，先对它的依赖进行分类。类别决定了加深后的模块如何跨越其接缝进行测试。

1. In-process 1. 进程内

Pure computation, in-memory state, no I/O. Always deepenable: merge the modules and test through the new interface directly. No adapter needed.

纯计算、内存状态、无 I/O。总是可以加深：合并这些模块，并直接通过新接口进行测试。不需要适配器。

2. Local-substitutable 2. 可本地替代

Dependencies that have local test stand-ins (PGLite for Postgres, in-memory filesystem). Deepenable if the stand-in exists. The deepened module is tested with the stand-in running in the test suite. The seam is internal; no port at the module's external interface.

拥有本地测试替身的依赖（用 PGLite 替代 Postgres，内存文件系统）。只要替身存在就可以加深。加深后的模块在测试套件中运行替身来测试。接缝是内部的；模块的外部接口上没有端口。

3. Remote but owned (Ports & Adapters) 3. 远程但自有（端口与适配器）

Your own services across a network boundary (microservices, internal APIs). Define a **port** (interface) at the seam. The deep module owns the logic; the transport is injected as an **adapter**. Tests use an in-memory adapter. Production uses an HTTP/gRPC/queue adapter.

你自己的、跨越网络边界的服务（微服务、内部 API）。在接缝处定义一个**端口**（接口）。深模块拥有逻辑；传输层作为**适配器**注入。测试使用内存适配器。生产环境使用 HTTP/gRPC/队列适配器。

Recommendation shape: "Define a port at the seam, implement an HTTP adapter for production and an in-memory adapter for testing, so the logic sits in one deep module even though it's deployed across a network."

建议的表述形式：“在接缝处定义一个端口，为生产环境实现一个 HTTP 适配器、为测试实现一个内存适配器，这样即便部署跨越网络，逻辑也仍位于一个深模块之中。”

4. True external (Mock) 4. 真正的外部依赖（Mock）

Third-party services (Stripe, Twilio, etc.) you don't control. The deepened module takes the external dependency as an injected port; tests provide a mock adapter.

你无法控制的第三方服务（Stripe、Twilio 等）。加深后的模块以注入端口的方式接收外部依赖；测试提供一个 mock（模拟）适配器。

Seam discipline 接缝纪律

- **One adapter means a hypothetical seam. Two adapters means a real one.** Don't introduce a port unless at least two adapters are justified (typically production + test). A single-adapter seam is just indirection.

  一个适配器意味着假想的接缝。两个适配器才意味着真实的接缝。 除非至少有两个适配器站得住脚（通常是生产 + 测试），否则不要引入端口。只有单个适配器的接缝只是多绕了一层。

- Internal seams vs external seams. A deep module can have internal seams (private to its implementation, used by its own tests) as well as the external seam at its interface. Don't expose internal seams through the interface just because tests use them.

  内部接缝与外部接缝。 深模块既可以有内部接缝（为其实现所私有，供它自己的测试使用），也可以有位于其接口处的外部接缝。不要仅仅因为测试用到了内部接缝，就把它们通过接口暴露出去。

Testing strategy: replace, don't layer 测试策略：替换，而非叠加

- Old unit tests on shallow modules become waste once tests at the deepened module's interface exist; delete them.

  一旦在加深后模块的接口上有了测试，原先针对浅模块的旧单元测试就成了废物；删掉它们。

- Write new tests at the deepened module's interface. The interface is the test surface.

  在加深后模块的接口上编写新测试。**接口即测试面**。

- Tests assert on observable outcomes through the interface, not internal state.

  测试通过接口断言可观察的结果，而不是内部状态。

- Tests should survive internal refactors, since they describe behaviour, not implementation. If a test has to change when the implementation changes, it's testing past the interface.

  测试应当能在内部重构后依然存活，因为它们描述的是行为，而不是实现。如果实现一变测试就得跟着改，那它就是在越过接口进行测试。

REFERENCE 参考文档 · DESIGN-IT-TWICE.md

Design It Twice 设计两次

Source 原文

When the user wants to explore alternative interfaces for a chosen deepening candidate, use this parallel sub-agent pattern. Based on "Design It Twice" (Ousterhout): your first idea is unlikely to be the best.

当用户想为选定的加深候选项探索不同的接口方案时，使用这种并行子智能体模式。基于 “Design It Twice” （Ousterhout）：你的第一个想法不太可能是最好的。

Uses the vocabulary in SKILL.md: module, interface, seam, adapter, leverage.

使用 SKILL.md 中的词汇：**模块**、**接口**、**接缝**、**适配器**、**杠杆**。

Process 流程

1. Frame the problem space 1. 界定问题空间

Before spawning sub-agents, write a user-facing explanation of the problem space for the chosen candidate:

在启动子智能体之前，为选定的候选项写一份面向用户的问题空间说明：

- The constraints any new interface would need to satisfy

  任何新接口都需要满足的约束

- The dependencies it would rely on, and which category they fall into (see DEEPENING.md)

  它将依赖哪些东西，以及这些依赖属于哪个类别（参见 DEEPENING.md）

- A rough illustrative code sketch to ground the constraints, not a proposal, just a way to make the constraints concrete

  一段粗略的示意性代码草图，用来让约束落地--不是方案，只是让约束变得具体的一种方式

Show this to the user, then immediately proceed to Step 2. The user reads and thinks while the sub-agents work in parallel.

把这份说明展示给用户，然后立即进入第 2 步。子智能体并行工作期间，用户可以阅读和思考。

2. Spawn sub-agents 2. 启动子智能体

Spawn 3+ sub-agents in parallel. Each must produce a **radically different** interface for the deepened module.

并行启动 3 个或更多子智能体。每个都必须为加深后的模块给出一个**截然不同**的接口。

Prompt each sub-agent with a separate technical brief (file paths, coupling details, dependency category from DEEPENING.md, what sits behind the seam). The brief is independent of the user-facing problem-space explanation in Step 1. Give each agent a different design constraint:

为每个子智能体提供一份单独的技术简报（文件路径、耦合细节、来自 DEEPENING.md 的依赖类别、接缝背后放什么）。这份简报独立于第 1 步中面向用户的问题空间说明。给每个智能体一个不同的设计约束：

- Agent 1: "Minimize the interface: aim for 1-3 entry points max. Maximise leverage per entry point."

  智能体 1：“最小化接口：入口点最多 1-3 个。最大化每个入口点的杠杆。”

- Agent 2: "Maximise flexibility: support many use cases and extension."

  智能体 2：“最大化灵活性：支持多种用例和扩展。”

- Agent 3: "Optimise for the most common caller: make the default case trivial."

  智能体 3：“为最常见的调用方优化：让默认情况变得极其简单。”

- Agent 4 (if applicable): "Design around ports & adapters for cross-seam dependencies."

  智能体 4（如适用）：“围绕端口与适配器来设计跨接缝的依赖。”

Include both SKILL.md vocabulary and GLOSSARY.md vocabulary in the brief so each sub-agent names things consistently with the architecture language and the project's domain language.

在简报中同时包含 SKILL.md 的词汇和 GLOSSARY.md 的词汇，让每个子智能体的命名都与架构语言和项目的领域语言保持一致。

Each sub-agent outputs:

每个子智能体输出：

1. Interface (types, methods, params, plus invariants, ordering, error modes)

   接口（类型、方法、参数，以及不变式、顺序约束、错误模式）

2. Usage example showing how callers use it

   展示调用方如何使用它的用法示例

3. What the implementation hides behind the seam

   实现在接缝背后隐藏了什么

4. Dependency strategy and adapters (see DEEPENING.md)

   依赖策略和适配器（参见 DEEPENING.md）

5. Trade-offs: where leverage is high, where it's thin

   权衡取舍：哪里杠杆高，哪里杠杆薄弱

3. Present and compare 3. 展示与比较

Present designs sequentially so the user can absorb each one, then compare them in prose. Contrast by **depth** (leverage at the interface), **locality** (where change concentrates), and **seam placement**.

依次展示各个设计，让用户能逐一消化，然后用文字对它们进行比较。从**深度**（接口处的杠杆）、**局部性**（变更集中在哪里）和**接缝位置**几个方面进行对比。

After comparing, give your own recommendation: which design you think is strongest and why. If elements from different designs would combine well, propose a hybrid. Be opinionated: the user wants a strong read, not a menu.

比较完之后，给出你自己的推荐：你认为哪个设计最强，以及为什么。如果不同设计中的元素可以很好地组合，就提出一个混合方案。要有主见：用户想要的是一个明确的判断，而不是一份菜单。

CHAPTER 17 · SKILL

Code Review
代码审查

Review the changes since a fixed point (commit, branch, tag, or merge-base) along two axes: Standards (does the code follow this repo's documented coding standards?) and Spec (does the code match what the originating issue/spec asked for?). Runs both reviews in parallel sub-agents and reports them side by side. Use when the user wants to review a branch, a PR, work-in-progress changes, or asks to \"review since X\".

沿两个维度审查自某个固定点（commit、分支、tag 或 merge-base）以来的变更：标准（Standards，代码是否遵循本仓库成文的编码标准？）和规范（Spec，代码是否符合最初的 issue/规范所要求的内容？）。在并行的子智能体中同时运行这两项审查，并将结果并排报告。当用户想要审查一个分支、一个 PR、进行中的变更，或者要求“review since X”（审查自 X 以来的变更）时使用。

Source 原文 · v1.3.1 · b0618bc4

Two-axis review of the diff between `HEAD` and a fixed point the user supplies:

对 `HEAD` 与用户提供的固定点之间的 diff 进行双维度审查：

- **Standards**: does the code conform to this repo's documented coding standards?

  标准（Standards）：代码是否符合本仓库成文的编码标准？
- **Spec**: does the code faithfully implement the originating issue / spec?

  规范（Spec）：代码是否忠实地实现了最初的 issue / 规范？

Both axes run as **parallel sub-agents** so they don't pollute each other's context, then this skill aggregates their findings.

两个维度作为**并行子智能体**运行，以免互相污染上下文，然后由本技能汇总它们的发现。

The issue tracker should have been provided to you. If not, tell the user to run `/setup-matt-pocock-skills`.

issue 跟踪器应该已经提供给你了。如果没有，告诉用户运行 `/setup-matt-pocock-skills`。

Process 流程

1. Pin the fixed point 1. 锁定固定点

Whatever the user said is the fixed point (a commit SHA, branch name, tag, `main`, `HEAD~5`, etc.). If they didn't specify one, ask for it.

用户说的是什么，固定点就是什么（commit SHA、分支名、tag、`main`、`HEAD~5` 等）。如果用户没有指定，就问他要。

Capture the diff command once: `git diff <fixed-point>...HEAD` (three-dot, so the comparison is against the merge-base). Also note the list of commits via `git log <fixed-point>..HEAD --oneline`.

一次性确定 diff 命令：`git diff <fixed-point>...HEAD`（三个点，这样比较的基准是 merge-base）。同时通过 `git log <fixed-point>..HEAD --oneline` 记下 commit 列表。

Before going further, confirm the fixed point resolves (`git rev-parse <fixed-point>`) and the diff is non-empty. A bad ref or empty diff should fail here, not inside two parallel sub-agents.

在继续之前，确认固定点能被解析（`git rev-parse <fixed-point>`），并且 diff 非空。错误的 ref 或空 diff 应该在这里就失败，而不是在两个并行子智能体内部失败。

2. Identify the spec source 2. 找到规范来源

Look for the originating spec, in this order:

按以下顺序查找最初的规范：

1. Issue references in the commit messages (`#123`, `Closes #45`, GitLab `!67`, etc.), fetched via the workflow in the tracker doc.

   commit 消息中的 issue 引用（`#123`、`Closes #45`、GitLab `!67` 等），通过跟踪器文档中的工作流获取。

2. A path the user passed as an argument.

   用户作为参数传入的路径。

3. A spec file under `docs/`, `specs/`, or `.scratch/` matching the branch name or feature.

   `docs/`、`specs/` 或 `.scratch/` 下与分支名或功能匹配的规范文件。

4. If nothing is found, ask the user where the spec is. If they say there isn't one, the **Spec** sub-agent will skip and report "no spec available".

   如果什么都没找到，就问用户规范在哪里。如果用户说没有规范，**规范**子智能体将跳过，并报告“no spec available”（没有可用的规范）。

3. Identify the standards sources 3. 找到标准来源

Search the repo for every file that documents how code should be written. When `CODING_STANDARDS.md` or `CONTRIBUTING.md` exists, it must be on the list.

在仓库中搜索每一个说明代码应如何编写的文件。如果存在 `CODING_STANDARDS.md` 或 `CONTRIBUTING.md`，它必须在列表中。

On top of whatever the repo documents, the Standards axis always carries the **smell baseline** below: a fixed set of Fowler code smells (_Refactoring_, ch.3) that applies even when a repo documents nothing. Two rules bind it:

在仓库成文的内容之外，标准维度始终带上下面的**坏味道基线**：一组固定的 Fowler 代码坏味道（《_重构_》第 3 章），即使仓库什么都没写，它也适用。有两条规则约束它：

- **The repo overrides.** A documented repo standard always wins; where it endorses something the baseline would flag, suppress the smell.

  仓库优先。 成文的仓库标准永远胜出；如果仓库标准认可了基线会标记的某种做法，就压下这个坏味道。

- **Always a judgement call.** Each smell is a labelled heuristic ("possible Feature Envy"), never a hard violation. Like any standard here, skip anything tooling already enforces.

  永远是判断题。 每种坏味道都是一个带标签的启发式判断（“可能存在依恋情结”），绝不是硬性违规。和这里的任何标准一样，跳过工具已经强制执行的内容。

Each smell reads what it is → how to fix; match it against the diff:

每种坏味道的写法是它是什么 → 如何修复；拿它去对照 diff：

- **Mysterious Name:** a function, variable, or type whose name doesn't reveal what it does or holds. → rename it; if no honest name comes, the design's murky.

  神秘命名（Mysterious Name）：函数、变量或类型的名字无法揭示它做什么或装着什么。→ 重命名；如果想不出一个诚实的名字，说明设计本身是浑浊的。

- **Duplicated Code:** the same logic shape appears in more than one hunk or file in the change. → extract the shared shape, call it from both.

  重复代码（Duplicated Code）：同样的逻辑形态在本次变更的多个 hunk 或文件中出现。→ 提取出共享的形态，在两处都调用它。

- **Feature Envy:** a method that reaches into another object's data more than its own. → move the method onto the data it envies.

  依恋情结（Feature Envy）：一个方法访问另一个对象的数据比访问自己的还多。→ 把这个方法搬到它所依恋的数据那里。

- **Data Clumps:** the same few fields or params keep travelling together (a type wanting to be born). → bundle them into one type, pass that.

  数据泥团（Data Clumps）：同样的几个字段或参数总是结伴出现（一个呼之欲出的类型）。→ 把它们打包成一个类型，传递这个类型。

- **Primitive Obsession:** a primitive or string standing in for a domain concept that deserves its own type. → give the concept its own small type.

  基本类型偏执（Primitive Obsession）：用基本类型或字符串来代表一个本该拥有自己类型的领域概念。→ 给这个概念一个小巧的专属类型。

- **Repeated Switches:** the same `switch`/`if`-cascade on the same type recurs across the change. → replace with polymorphism, or one map both sites share.

  重复的 switch（Repeated Switches）：针对同一类型的同一个 `switch`/`if` 级联在变更中反复出现。→ 用多态替换，或者用一个两处共享的映射表替换。

- **Shotgun Surgery:** one logical change forces scattered edits across many files in the diff. → gather what changes together into one module.

  霰弹式修改（Shotgun Surgery）：一个逻辑上的变更迫使 diff 中许多文件都要零散地修改。→ 把一起变化的东西聚拢到一个模块里。

- Divergent Change: one file or module is edited for several unrelated reasons. → split so each module changes for one reason.

  发散式变化（Divergent Change）：一个文件或模块因为好几个互不相关的原因被修改。→ 拆分，让每个模块只因一个原因而变化。

- Speculative Generality: abstraction, parameters, or hooks added for needs the spec doesn't have. → delete it; inline back until a real need shows.

  夸夸其谈通用性（Speculative Generality）：为规范中并不存在的需求添加了抽象、参数或钩子。→ 删掉它；内联回去，直到真正的需求出现。

- Message Chains: long `a.b().c().d()` navigation the caller shouldn't depend on. → hide the walk behind one method on the first object.

  过长的消息链（Message Chains）：调用方不应依赖的冗长 `a.b().c().d()` 导航。→ 把这段遍历藏到第一个对象的一个方法后面。

- Middle Man: a class or function that mostly just delegates onward. → cut it, call the real target direct.

  中间人（Middle Man）：一个类或函数基本上只是把调用转交出去。→ 砍掉它，直接调用真正的目标。

- Refused Bequest: a subclass or implementer that ignores or overrides most of what it inherits. → drop the inheritance, use composition.

  被拒绝的遗赠（Refused Bequest）：子类或实现者忽略或覆写了它所继承的大部分内容。→ 去掉继承，改用组合。

4. Spawn both sub-agents in parallel 4. 并行派生两个子智能体

Issue both sub-agent calls together, in the foreground, and aggregate the reports they return.

同时发出两个子智能体调用，在前台运行，并汇总它们返回的报告。

Standards sub-agent prompt should include:

标准子智能体的提示词应包含：

- The full diff command and commit list.

  完整的 diff 命令和 commit 列表。

- The list of standards-source files you found in step 3, plus the smell baseline from step 3 pasted in full (the sub-agent has no other access to it).

  你在第 3 步找到的标准来源文件列表，**再加上第 3 步中的坏味道基线**，全文粘贴（子智能体没有其他途径获取它）。

- The brief: "Report, per file/hunk where relevant, (a) every place the diff violates a documented standard: cite the standard (file + the rule); and (b) any baseline smell you spot: name it and quote the hunk. Distinguish hard violations from judgement calls: documented-standard breaches can be hard, but baseline smells are always judgement calls, and a documented repo standard overrides the baseline. Skip anything tooling enforces. Under 400 words."

  任务说明：“按文件/hunk（如适用）报告：(a) diff 违反成文标准的每一处：引用该标准（文件 + 规则）；以及 (b) 你发现的任何基线坏味道：说出它的名字并引用对应的 hunk。区分硬性违规与判断题：违反成文标准可以是硬性的，但基线坏味道永远是判断题，而且成文的仓库标准优先于基线。跳过任何由工具强制执行的内容。不超过 400 词。”

Spec sub-agent prompt should include:

规范子智能体的提示词应包含：

- The diff command and commit list.

  diff 命令和 commit 列表。

- The path or fetched contents of the spec.

  规范的路径或已获取的内容。

- The brief: "Report: (a) requirements the spec asked for that are missing or partial; (b) behaviour in the diff that wasn't asked for (scope creep); (c) requirements that look implemented but where the implementation looks wrong. Quote the spec line for each finding. Under 400 words."

  任务说明：“报告：(a) 规范要求了、但缺失或只实现了一部分的需求；(b) diff 中未被要求的行为（范围蔓延）；(c) 看起来已经实现、但实现看起来有误的需求。每条发现都要引用规范中对应的那一行。不超过 400 词。”

If the spec is missing, skip the Spec sub-agent and note this in the final report.

如果规范缺失，就跳过规范子智能体，并在最终报告中注明这一点。

5. Aggregate 5. 汇总

Present the two reports under `## Standards` and `## Spec` headings, verbatim or lightly cleaned. Do **not** merge or rerank findings, because the two axes are deliberately separate (see _Why two axes_).

把两份报告分别放在 `## Standards` 和 `## Spec` 标题下呈现，原样照搬或稍加整理。**不要**合并发现或重新排序，因为两个维度是刻意分开的（参见 _为什么要分两个维度_）。

End with a one-line summary: total findings per axis, and the worst issue _within each axis_ (if any). Don't pick a single winner across axes: that's the reranking the separation exists to prevent.

最后用一行总结收尾：每个维度的发现总数，以及_每个维度内部_最严重的问题（如果有的话）。不要跨维度挑出唯一的“头号问题”：那正是维度分离所要防止的重新排序。

Why two axes 为什么要分两个维度

A change can pass one axis and fail the other:

一项变更可能通过一个维度而未通过另一个：

- Code that follows every standard but implements the wrong thing → **Standards pass, Spec fail.**

  代码遵循了每一条标准，却实现了错误的东西 → **标准通过，规范失败。**

- Code that does exactly what the issue asked but breaks the project's conventions → **Spec pass, Standards fail.**

  代码完全做到了 issue 的要求，却破坏了项目的约定 → **规范通过，标准失败。**

Reporting them separately stops one axis from masking the other.

分开报告，可以防止一个维度掩盖另一个维度。

CHAPTER 18 · SKILL

PR
PR 正文

Use when writing a PR body.

在撰写 PR 正文时使用。

Source 原文 · v1.3.1 · b0618bc4

Use this template for writing the PR body:

撰写 PR 正文时使用这个模板：

```
## Summary

<diagram, diff-sketch, or tree>

## Evidence

- **Before:** <screenshot/output/failing test run>
  **After:** <screenshot/output/passing test run>

## Merge Danger

**Door:** <one-way or two-way>

<optional: description>

**Blast Radius:** <one-word description>

<optional: potential ramifications of merge>
```

```
## Summary

<图示、diff 草图或树>

## Evidence

- **Before:** <截图/输出/失败的测试运行>
  **After:** <截图/输出/通过的测试运行>

## Merge Danger

**Door:** <单向或双向>

<可选：描述>

**Blast Radius:** <一个词的描述>

<可选：合并可能带来的连锁影响>
```

Sections 各部分说明

Skip all preambles and keep prose brief. Use the user's domain language from `GLOSSARY.md`.

省掉一切开场白，文字保持简短。使用 `GLOSSARY.md` 中用户的领域语言。

Summary Summary（摘要）

Pick the smallest view that makes the key point clear.

选择能把要点讲清楚的最小视图。

- Show logic or an algorithm as pseudocode:

  用伪代码展示逻辑或算法：

```
on(save)
  if content is unchanged
    return cached result
  write new content
  return fresh result
```


```
on(save)
  if content is unchanged
    return cached result
  write new content
  return fresh result
```


- Show runtime control flow as a call tree:

  用调用树展示运行时控制流：

```
submitForm
  createSession
    persistPrompt
    launchAgent
  navigateToSession
```


```
submitForm
  createSession
    persistPrompt
    launchAgent
  navigateToSession
```


- Show UI structure as a component tree, including state and module boundaries that matter:

  用组件树展示 UI 结构，包括关键的状态和模块边界：

```
<SessionPage> (apps/example/src/routes/session.tsx)
  useSessionEvents()
  <SessionToolbar>
    <RunSkillButton> (packages/ui)
```


```
<SessionPage> (apps/example/src/routes/session.tsx)
  useSessionEvents()
  <SessionToolbar>
    <RunSkillButton> (packages/ui)
```


- Show file responsibility or a broad refactor as a shallow file tree:

  用浅层文件树展示文件职责或大范围重构：

```
src/
├── commands/      # parses user actions
├── sessions/      # owns session state
└── transport/     # sends API requests
```


```
src/
├── commands/      # parses user actions
├── sessions/      # owns session state
└── transport/     # sends API requests
```


- Show component interaction, control flow, or data flow with Mermaid:

  用 Mermaid 展示组件交互、控制流或数据流：

```
sequenceDiagram
    participant User
    participant UI
    participant Daemon
    User->>UI: choose command
    UI->>Daemon: send expanded prompt
    Daemon-->>UI: stream result
```


```
sequenceDiagram
    participant User
    participant UI
    participant Daemon
    User->>UI: choose command
    UI->>Daemon: send expanded prompt
    Daemon-->>UI: stream result
```


- Use `diff` when the point is what changes and the surrounding shape already exists. Match the diff shape to the topic.

  当重点在于改了什么、且周围的结构已经存在时，使用 `diff`。让 diff 的形态与主题相匹配。

For a component change:

组件变更：

```
<SessionPage>
  useSessionEvents()
  <SessionToolbar>
+   <RunSkillButton />
  <SessionTimeline>
+   <SkillResultCard />
```

```
<SessionPage>
  useSessionEvents()
  <SessionToolbar>
+   <RunSkillButton />
  <SessionTimeline>
+   <SkillResultCard />
```

For a file-layout change:

文件布局变更：

```
src/
├── commands/
+│   └── show-me.ts      # expands the slash command
├── sessions/
-└── transport.ts
+└── transport/
+   ├── client.ts
+   └── stream.ts
```

```
src/
├── commands/
+│   └── show-me.ts      # expands the slash command
├── sessions/
-└── transport.ts
+└── transport/
+   ├── client.ts
+   └── stream.ts
```

For a call-tree or call-stack change:

调用树或调用栈变更：

```
submitForm
  createSession
    persistPrompt
+   expandSkillMention
    launchAgent
- navigateToSession
+ navigateToSession
+   subscribeToEvents
```

```
submitForm
  createSession
    persistPrompt
+   expandSkillMention
    launchAgent
- navigateToSession
+ navigateToSession
+   subscribeToEvents
```

For a state or control-flow change:

状态或控制流变更：

```
 on(save)
-  write content
+  if content is unchanged
+    return cached result
+  write new content
+  invalidate cache
```


```
 on(save)
-  write content
+  if content is unchanged
+    return cached result
+  write new content
+  invalidate cache
```


- Show the whole block when most of it is new, when omitted context would hide ownership or order, or when the user needs a copyable target shape:

  当大部分内容是新的、省略上下文会掩盖归属或顺序，或者用户需要一个可复制的目标形态时，展示完整的代码块：

```
function expandSkill(command: string): string {
  const skillName = command.slice(1);
  return `use the ${skillName} skill`;
}
```


```
function expandSkill(command: string): string {
  const skillName = command.slice(1);
  return `use the ${skillName} skill`;
}
```


Guidance 指导原则

Place each visual next to the short text it supports. Keep only the calls, files, props, states, and boundaries needed to answer the user's current question or the options to resolve the current discussion point.

把每个图示放在它所支撑的简短文字旁边。只保留回答用户当前问题、或给出解决当前讨论点的选项所需要的调用、文件、props、状态和边界。

You may use one of these, you may use several, it is unlikely you will use all of them. Use your judgement and don't overwhelm the user.

你可能用到其中一种，也可能用到几种，但不太可能全部用上。自行判断，别让用户应接不暇。

Evidence Evidence（证据）

Concrete evidence that the change works. Show a before and after.

证明变更有效的具体证据。展示变更前后的对比。

Screenshots are S-tier - when the environment is set up for it and the change is visual.

截图是 S 级证据--前提是环境已为此准备好，且变更是可视的。

Execution-based evidence is A-tier. Test results, console output. Show the exact test that now fails and passes, using pseudocode.

基于执行的证据是 A 级。测试结果、控制台输出。用伪代码展示那个此前失败、现在通过的具体测试。

Merge Danger Merge Danger（合并风险）

Describe whether it's a one-way or two-way door. You can walk back through two-way doors, but not one-way doors. A PR that is cheap to roll back is lower risk. Changes that involve destructive actions or hard-to-reverse decisions are one-way doors.

说明这是单向门还是双向门。双向门可以原路退回，单向门则不行。回滚代价低的 PR 风险更低。涉及破坏性操作或难以逆转的决策的变更就是单向门。

The blast radius is the potential impact or scope of the changes introduced by this PR. Consider all possibilities. Examples are layout shift, breakages for consumers, mobile responsiveness, etc.

爆炸半径（Blast Radius）是本 PR 所引入变更的潜在影响或波及范围。考虑所有可能性。例如布局偏移、对使用方造成的破坏、移动端响应式表现等。

REFERENCE 参考文档 · CREDITS.md

Credits 致谢

Source 原文

The Summary section's menu of visuals (pseudocode, call trees, component trees, file trees, Mermaid, diffs) and its placement guidance come from Dex Horthy's show-me skill, reproduced almost word for word and aimed at a diff instead of a live conversation. pr does not depend on show-me as a skill (it isn't part of this repo, and a hard dependency would break standalone installs), so the content is copied in rather than pointed at; this file is the attribution a dependency would otherwise have carried.

Summary 部分的图示菜单（伪代码、调用树、组件树、文件树、Mermaid、diff）及其摆放指导，来自 Dex Horthy 的 show-me 技能，几乎逐字照搬，只是把对象从实时对话改成了 diff。pr 并不以技能的形式依赖 show-me（它不属于本仓库，而硬依赖会破坏独立安装），因此内容是复制进来的，而不是指向它；本文件承担了原本由依赖关系所承载的署名。

CHAPTER 19 · SKILL

Retro
复盘

Conduct a retrospective on a coding session.

对一次编码会话进行复盘。

Source 原文 · v1.3.1 · b0618bc4

The user has asked for a **retrospective**. You are suggesting improvements to the coding agent's **environment** to improve future runs.

用户要求进行一次**复盘**。你要为编码智能体的**环境**提出改进建议，以提升未来的运行效果。

Steps 步骤

1. Call the Skill tool with `writing-for-agents` for the writing style guide.

   用 `writing-for-agents` 调用 Skill 工具，获取写作风格指南。

2. Read the primary sources for the session the user specifies. This may mean searching through session logs on this machine. If the user doesn't specify a session, default to the current one.

   阅读用户指定会话的一手资料。这可能意味着要在本机的会话日志中搜索。如果用户没有指定会话，默认使用当前会话。

3. Look for candidates for improvement in these categories.

   在以下类别中寻找可改进的候选项。

- **Navigation**: how easy was it for the agent to find the right files? Are there hidden dependencies between files? Would a **navigation pointer** make it easier? _Use when_ the session took a long time to find a piece of information.

  导航：智能体找到正确文件有多容易？文件之间是否存在隐藏的依赖？一个**导航指针**能否让查找更容易？_适用于_会话花了很长时间才找到某条信息时。

- Automated checks: are there automated checks that could catch errors the agent made? Linting, typing, tests, filesystem linters? Read the repo's own check command first (its `package.json`/build-tool `lint`/`check` scripts, its CI workflow), so a check that already exists but sits unwired or silently broken is the finding, not a reinvention. A repo with no guardrail (no pre-commit hook and no CI job running its lint/typecheck/test command) is itself a finding: an un-linted repo is a standing missed opportunity, not a neutral default. _Use when_ the agent made a mistake an automated check could have caught, or the repo has no guardrail at all.

  自动化检查：是否有自动化检查能捕获智能体犯下的错误？lint、类型检查、测试、文件系统 linter？先读仓库自己的检查命令（它的 `package.json`/构建工具中的 `lint`/`check` 脚本、它的 CI 工作流），这样，如果某个检查早已存在却没有接入或已悄无声息地失效，那这本身就是发现，而不是去重新发明一遍。一个没有任何**护栏**的仓库（既没有 pre-commit 钩子，也没有运行其 lint/类型检查/测试命令的 CI 任务）本身就是一个发现：没有 lint 的仓库是一个长期存在的错失良机，而不是中性的默认状态。_适用于_智能体犯了一个本可被自动化检查捕获的错误，或仓库根本没有任何护栏时。

- Coding standards: should the reviewer agent be given a new rule to enforce? Should an existing rule be removed or clarified? Classify the violation first: a mechanical one (a fixed syntactic pattern, a banned API, an import shape, a file-location rule) gets a deterministic check, full stop: a custom rule in the repo's own linter, a new pre-commit hook, or a new CI job, whichever the repo's language and existing guardrail make cheapest. Default to building the check over writing the rule. Reserve `CODING_STANDARDS.md` for genuine judgement calls (cross-file consistency, " matches the surrounding style," anything no guardrail could ever substitute for). _Use when_ the reviewer agent failed to catch a mistake.

  编码标准：是否应该给**审查智能体**一条新的规则去执行？是否应该删除或澄清某条现有规则？先给违规分类：**机械性**违规（固定的语法模式、被禁用的 API、导入形态、文件位置规则）一律用确定性检查解决，没有例外：在仓库自己的 linter 中加一条自定义规则、新增一个 pre-commit 钩子，或新增一个 CI 任务，按仓库的语言和现有护栏选最省事的那个。默认去构建检查，而不是去写规则。`CODING_STANDARDS.md` 只留给真正的**判断类问题**（跨文件一致性、“与周围风格保持一致”，以及任何护栏永远无法替代的东西）。_适用于_审查智能体没能发现某个错误时。

- Global AGENTS.md: are there any steering instructions that should be moved to coding standards (or automated checks) instead? _Use when_ the AGENTS.md file is particularly large - in the repo OR the user's global scope.

  全局 AGENTS.md：是否有某些引导指令应该移到编码标准（或自动化检查）中去？_适用于_ AGENTS.md 文件特别大时--无论是在仓库中，还是在用户的全局范围内。

- Tool economy: did the agent make expensive tool calls that could be streamlined? Is there any custom tooling (CLI's, MCP's) that is particularly token-inefficient? _Use when_ the agent made an expensive tool call.

  工具经济性：智能体是否进行了可以精简的昂贵工具调用？是否有某些自定义工具（CLI、MCP）特别耗费 token？_适用于_智能体进行了昂贵的工具调用时。

- No-ops: look for instructions in steering files that don't modify the agent's behavior. _Use when_ the steering files are large and unwieldy.

  无效指令：在引导文件中寻找那些并不会改变智能体行为的指令。_适用于_引导文件庞大且难以驾驭时。

- Information access: look for opportunities to increase the agent's access to information. Teeing dev server logs, readonly access to third-party services. _Use when_ a crucial piece of information was not available to the agent.

  信息获取：寻找能增加智能体信息获取能力的机会。例如 tee 开发服务器日志、对第三方服务的只读访问。_适用于_智能体无法获得某条关键信息时。

4. Present these candidates to the user, in order of severity.

   按严重程度排序，把这些候选项呈现给用户。

Reference 参考

Implementation vs Review 实现与审查

Remember that all work goes through two stages: implementation and review. The implementation agent has the most context pressure. They are responsible for exploration, writing code, and debugging failures.

记住，所有工作都要经过两个阶段：实现和审查。实现智能体承受的**上下文压力**最大。它负责探索、编写代码和调试失败。

The review agent has the least context pressure - it receives a diff, so no exploration needed. It often does not need to write code or debug.

审查智能体承受的上下文压力最小--它收到的是一个 diff，因此不需要探索。它通常也不需要编写代码或调试。

This means that the review agent should be responsible for imposing coding standards, not the implementation agent.

这意味着应该由审查智能体负责落实编码标准，而不是实现智能体。

Files 文件

You have access to several files in the repo:

你可以访问仓库中的若干文件：

- `CLAUDE.md/AGENTS.md`: these files are pushed to the context window of any agent working in this repo. They should be used incredibly sparingly, usually only for navigation pointers to other files.

  `CLAUDE.md/AGENTS.md`：这些文件会被推送到在该仓库中工作的任何智能体的上下文窗口中。应当极其节制地使用它们，通常只用于放置指向其他文件的**导航指针**。

- `CODING_STANDARDS.md`: this file is read during review, not implementation. Add navigation pointers to docs folders if the standards file gets more than 1,000 lines long.

  `CODING_STANDARDS.md`：这个文件在审查阶段读取，而不是在实现阶段。如果标准文件超过 1,000 行，就添加指向 docs 文件夹的**导航指针**。

- Docs: use docs as references files, pointed to by other files. Look for existing docs before writing new ones.

  文档：把文档当作参考文件使用，由其他文件指向它们。在写新文档之前，先找找已有的文档。

- Skills: use skills for docs (since their description goes into the agent's context window), or for user-invoked commands. Follow the advice in the `writing-for-agents` skill.

  技能：把技能用于文档（因为技能的描述会进入智能体的上下文窗口），或用于用户调用的命令。遵循 `writing-for-agents` 技能中的建议。

CHAPTER 20 · SKILL

Wizard
向导

Generate an interactive bash wizard that walks a human through steps only they can perform. Use when provisioning infrastructure, setting up credentials or CI secrets, walking an unfamiliar third-party dashboard, or running a one-off migration or cutover. Don't invoke this for steps the agent can perform itself.

生成一个交互式 bash 向导，一步步带领人类完成只有他们自己才能执行的步骤。在配置基础设施、设置凭据或 CI secret、操作陌生的第三方控制台，或执行一次性的迁移或切换时使用。对于智能体自己就能执行的步骤，不要调用本技能。

Source 原文 · v1.3.1 · b0618bc4

A wizard is a bash script that walks a human, step by step, through a manual procedure that's tedious to do by hand and tedious to re-explain to an AI every time. It opens each URL, says exactly what to click and copy, captures the values, writes them where they belong (`.env`, GitHub secrets), confirms at every stage, and shows how many stages are left. It might configure third-party services, run a one-off migration, or move the project from one state to another.

向导（wizard）是一个 bash 脚本，它一步一步地带领人类完成一套手动流程--这种流程亲手做起来繁琐，每次都要重新给 AI 解释一遍也同样繁琐。它会打开每个 URL，准确说明要点击什么、复制什么，捕获这些值，把它们写到该去的地方（`.env`、GitHub secrets），在每个阶段进行确认，并显示还剩多少个阶段。它可能用来配置第三方服务、执行一次性迁移，或者把项目从一种状态迁移到另一种状态。

The delightful UX is already solved by template.sh: stage-by-stage progress, confirmation gates, cross-platform URL opening (including WSL), hidden secret entry, idempotent `.env` upserts, `gh secret`/`gh variable` writes, and a closing summary. **Your job is only to scope the procedure and author its stages.** The library above the `STAGES` marker is identical in every wizard; that consistency is the point: never hand-edit it.

令人愉悦的用户体验已经由 template.sh 解决了：逐阶段的进度显示、确认关卡、跨平台打开 URL（包括 WSL）、隐藏式的 secret 输入、幂等的 `.env` upsert、`gh secret`/`gh variable` 写入，以及结尾的汇总。**你的工作只是界定流程范围并编写它的各个阶段。**`STAGES` 标记上方的库在每个向导中都完全相同；这种一致性正是关键所在：永远不要手动修改它。

A wizard is ephemeral by default: built for one run, saved to a scratch or `scripts/` path, deleted when the job's done. Commit it only when the user wants a repeatable setup path that should live in the repo.

向导默认是临时性的：为一次运行而构建，保存在草稿目录或 `scripts/` 路径下，任务完成后即删除。只有当用户希望有一条可重复使用、应当留在仓库中的设置路径时，才提交它。

Process 流程

1. Scope the procedure 1. 界定流程范围

Work out every manual step the human must take and every value that gets captured along the way. Read the repo first, don't ask cold:

弄清楚人类必须执行的每一个手动步骤，以及过程中要捕获的每一个值。先读仓库，不要一上来就问：

- For setup: `.env`, `.env.example`, `.env.*`, `README`, `docker-compose*`, framework config, and `.github/workflows/*` (every `secrets.*` / `vars.*` reference is a value the wizard must produce).

  对于设置类任务：`.env`、`.env.example`、`.env.*`、`README`、`docker-compose*`、框架配置，以及 `.github/workflows/*`（每一处 `secrets.*` / `vars.*` 引用都是向导必须产出的一个值）。

- For a migration or transition: the current state, the target state, and the irreversible actions between them.

  对于迁移或过渡类任务：当前状态、目标状态，以及两者之间不可逆的操作。

Then show the user the ordered list of stages and the values each produces, and confirm: they may add, drop, or reorder.

然后向用户展示有序的阶段列表以及每个阶段产出的值，并请其确认：用户可以增加、删除或调整顺序。

Done when: every stage is named in order, and for each captured value you know (a) where the human gets it, (b) where it's written (`.env`, a GitHub secret, both, or nowhere; some stages are pure actions), and (c) whether it's secret (hidden entry) or public.

完成标准：每个阶段都按顺序命名好了，并且对于每个捕获的值，你都知道 (a) 人类从哪里获取它，(b) 它被写到哪里（`.env`、GitHub secret、两者都写，或者哪里都不写；有些阶段是纯操作），以及 (c) 它是机密的（隐藏输入）还是公开的。

2. Map each stage's journey 2. 梳理每个阶段的路径

For each stage, write the precise path a human follows: which URL to open, what to do there, where a value is shown, which variable it fills: e.g. "Dashboard → Developers → API keys → Reveal test key → copy". Where you don't actually know the current UI or the exact command, say so and ask the user or check the docs: never invent steps that may not exist.

为每个阶段写出人类要走的精确路径：打开哪个 URL，在那里做什么，值显示在哪里，它填充哪个变量：例如“Dashboard → Developers → API keys → Reveal test key → copy”。如果你其实并不了解当前的 UI 或确切的命令，就如实说明，并询问用户或查阅文档：绝不要编造可能并不存在的步骤。

Done when: every stage traces to concrete instructions a stranger could follow.

完成标准：每个阶段都能落实为一个陌生人也能照着做的具体说明。

3. Author the wizard 3. 编写向导

Copy `template.sh` to the target path. Replace the example stage with one `stage` per step, in dependency order. Use the library helpers: `stage`, `say`/`step`, `open_url`, `ask`/`ask_secret`, `write_env`, `set_secret`/`set_var`, `pause`/`confirm`. Set `TOTAL_STAGES` to the number of stages you wrote.

把 `template.sh` 复制到目标路径。用每个步骤对应一个 `stage` 的方式替换掉示例阶段，并按依赖顺序排列。使用库提供的辅助函数：`stage`、`say`/`step`、`open_url`、`ask`/`ask_secret`、`write_env`、`set_secret`/`set_var`、`pause`/`confirm`。把 `TOTAL_STAGES` 设置为你编写的阶段数量。

Hold the bar the template sets: open the URL before asking for its value, use `ask_secret` for anything secret, `write_env` every persisted value, `set_secret` only the values CI actually needs, and `confirm` before any irreversible action. Each `stage` clears the screen so only the current step is visible: keep a stage to one focused task so nothing the human needs scrolls away. Don't touch the library above the marker.

守住模板设定的标准：先打开 URL，再询问它对应的值；凡是机密都用 `ask_secret`；每个需要持久化的值都要 `write_env`；只对 CI 真正需要的值使用 `set_secret`；在任何不可逆操作之前都要 `confirm`。每个 `stage` 都会清屏，因此屏幕上只显示当前步骤：让一个阶段只承担一项专注的任务，这样人类需要的内容就不会被滚出屏幕。不要碰标记上方的库。

4. Verify and hand off 4. 验证并交接

- `bash -n <script>`; run `shellcheck` if available.

  `bash -n <script>`；如果有 `shellcheck`，就运行它。

- `chmod +x <script>`.

  `chmod +x <script>`。

- Don't run it end-to-end yourself: it opens browsers and blocks on human input. Trace it statically instead: every value from step 1 is captured and lands where step 1 said, and every `set_secret` name exactly matches a `secrets.*` reference in CI.

  不要自己端到端地运行它：它会打开浏览器，并阻塞等待人类输入。改为静态地追踪它：第 1 步中的每个值都被捕获，并落到第 1 步所说的位置；每个 `set_secret` 的名称都与 CI 中的某个 `secrets.*` 引用完全一致。

- Tell the user how to run it. If it's a repeatable setup path, commit it and link it from the README so the next person runs the script instead of asking an AI.

  告诉用户如何运行它。如果这是一条可重复使用的设置路径，就提交它并在 README 中链接它，这样下一个人就会直接运行脚本，而不是去问 AI。

CHAPTER 21 · SKILL

Grill Me
追问我

A relentless interview to sharpen a plan or design.

一场毫不留情的访谈，用来打磨计划或设计。

Source 原文 · v1.3.1 · b0618bc4

Call the Skill tool with "grilling".

用 "grilling" 调用 Skill 工具。

CHAPTER 22 · SKILL

Grilling
深度追问

Grill the user relentlessly about a plan, decision, or idea. Use when the user wants to stress-test their thinking, or uses any 'grill' trigger phrases.

就某个计划、决策或想法对用户进行毫不留情的深度追问。当用户想对自己的思路做压力测试，或使用任何 'grill' 类触发短语时使用。

Source 原文 · v1.3.1 · b0618bc4

Interview the user relentlessly until you reach a shared understanding. Map this as a **design tree**: every decision branches into the decisions that hang off it.

毫不留情地访谈用户，直到你们达成共识。把它梳理成一棵**设计树**：每个决策都会分叉出依附于它的后续决策。

Work the tree in **rounds**. The **frontier** is every decision whose prerequisites are already settled: the questions you can ask _now_ without guessing at answers you haven't heard yet. Ask the whole frontier in one round: number each question and give your recommended answer. Then wait for the user's answers before the next round.

按**轮次**处理这棵树。**前沿**是所有前置条件都已确定的决策：也就是你现在就能问、而无需猜测尚未听到的答案的问题。在一轮中问完整个前沿：给每个问题编号，并给出你推荐的答案。然后等用户回答完，再进入下一轮。

Format a round like so:

每一轮的格式如下：

```
☐ **Q1** - **<question title>**: <question body, might be multiple paragraphs, including multiple choices>

➡☐ <your recommended answer>

---

☐ **Q2** - **<question title>**: <question body, might be multiple paragraphs, including multiple choices>

➡☐ <your recommended answer>
```

```
☐ **Q1** - **<问题标题>**：<问题正文，可能有多个段落，包括多个选项>

➡☐ <你推荐的答案>

---

☐ **Q2** - **<问题标题>**：<问题正文，可能有多个段落，包括多个选项>

➡☐ <你推荐的答案>
```

Word each question so "yes" accepts your recommended answer.

措辞每个问题时，要让“是”表示接受你推荐的答案。

Each round the user answers reshapes the tree: settled decisions push the frontier outward and unblock questions that depended on them. Recompute the frontier and ask the next round. A question whose answer depends on another question still open in this round belongs to a _later_ round, not this one.

用户每回答一轮，都会重塑这棵树：已确定的决策会把前沿向外推进，并解锁依赖于它们的问题。重新计算前沿，然后问下一轮。如果一个问题的答案取决于本轮中另一个仍未解决的问题，它就属于后面的轮次，而不是这一轮。

Finding _facts_ is your job, never the user's. When a frontier question needs a fact from the environment (filesystem, tools, etc.), dispatch a sub-agent to find it; don't ask the user for anything you could look up yourself. Don't block on it: a running exploration is an unsettled prerequisite, so only the questions downstream of it wait for the sub-agent to report; ask the rest of the frontier now. The _decisions_ are the user's: put each to them and wait.

查找事实是你的工作，绝不是用户的。当某个前沿问题需要来自环境的事实（文件系统、工具等）时，派一个子智能体去查；凡是你自己能查到的东西，都不要问用户。不要因此停下等待：一次正在进行的探索就是一个尚未确定的前置条件，所以只有依赖于它的下游问题需要等子智能体回报；前沿上其余的问题现在就问。决策归用户：把每个决策交给他们，然后等待。

The session is done when the frontier is empty: every branch of the design tree visited, nothing left silently assumed. Do not act on it until the user confirms you have reached a shared understanding.

当前沿为空时，这次会话就结束了：设计树的每个分支都已走过，没有任何东西被默默假定。在用户确认你们已经达成共识之前，不要据此采取行动。

CHAPTER 23 · SKILL

Handoff
交接

Compact the current conversation into a handoff document for another agent to pick up.

把当前对话压缩成一份交接文档，供另一个智能体接手。

Source 原文 · v1.3.1 · b0618bc4

Write a handoff document summarising the current conversation so a fresh agent can continue the work. Save to the temporary directory of the user's OS (`$TMPDIR`, else `/tmp`; `%TEMP%` on Windows) - not the current workspace.

写一份交接文档，总结当前对话，让一个全新的智能体能够继续这项工作。保存到用户操作系统的临时目录（`$TMPDIR`，否则 `/tmp`；Windows 上为 `%TEMP%`）--不要保存在当前工作区。

Include a "suggested skills" section in the document, naming which skills the next agent should call the Skill tool for.

在文档中包含一个“建议技能”（suggested skills）部分，列出下一个智能体应通过 Skill 工具调用哪些技能。

Do not duplicate content already captured in other artifacts (specs, plans, ADRs, issues, commits, diffs). Reference them by path or URL instead.

不要重复已经记录在其他产物（规范、计划、ADR、issue、commit、diff）中的内容。改为通过路径或 URL 引用它们。

Redact any sensitive information, such as API keys, passwords, or personally identifiable information.

删去任何敏感信息，例如 API 密钥、密码或个人身份信息。

If the user passed arguments, treat them as a description of what the next session will focus on and tailor the doc accordingly.

如果用户传入了参数，就把它们视为对下一个会话关注重点的描述，并据此调整文档内容。

CHAPTER 24 · SKILL

Teach
教学

Teach the user a new skill or concept, within this workspace.

在当前工作区内教用户一项新技能或一个新概念。

Source 原文 · v1.3.1 · b0618bc4

The user has asked you to teach them something. This is a stateful request - they intend to learn the topic over multiple sessions.

用户请你教他们某样东西。这是一个有状态的请求--他们打算分多次会话来学习这个主题。

Teaching Workspace 教学工作区

Treat the current directory as a teaching workspace. Workspace paths (`./lessons/` and the rest) resolve from the directory `/teach` was run in; only the `*-FORMAT.md` links resolve from this skill's folder. The state of their learning is captured in this directory in several files:

把当前目录视为教学工作区。工作区路径（`./lessons/` 及其余路径）都从运行 `/teach` 的目录解析；只有 `*-FORMAT.md` 链接从本技能所在的文件夹解析。用户的学习状态以若干文件的形式记录在这个目录中：

- `MISSION.md`: A document capturing the _reason_ the user is interested in the topic. This should be used to ground all teaching. Use the format in MISSION-FORMAT.md.

  `MISSION.md`：记录用户对该主题感兴趣的 _原因_ 的文档。所有教学都应以它为根基。使用 MISSION-FORMAT.md 中的格式。

- `./reference/*.html`: A directory of reference materials. These are the compressed learnings from the lessons - cheat sheets, reference algorithms, syntax, yoga poses, glossaries. They are the raw units of learning. They should be beautiful documents which print out well, and are designed for quick reference.

  `./reference/*.html`：参考资料目录。这些是从课程中压缩提炼出的学习成果--速查表、参考算法、语法、瑜伽体式、术语表。它们是学习的基本单元。它们应当是漂亮、打印效果好的文档，专为快速查阅而设计。

- `RESOURCES.md`: A list of resources which can be explored to ground your teaching in contextual knowledge, or to acquire knowledge and wisdom. Use the format in RESOURCES-FORMAT.md.

  `RESOURCES.md`：一份资源列表，可用来为你的教学提供情境知识作为根基，或用来获取知识与智慧。使用 RESOURCES-FORMAT.md 中的格式。

- `./learning-records/*.md`: A directory of learning records, which capture what the user has learned. These are loosely equivalent to architectural decision records in software development - they capture non-obvious lessons and key insights that may need to be revised later, or drive future sessions. These should be used to calculate the zone of proximal development. They are titled `0001-<dash-case-name>.md`, where the number increments each time. Use the format in LEARNING-RECORD-FORMAT.md.

  `./learning-records/*.md`：学习记录目录，记录用户学到了什么。它们大致相当于软件开发中的架构决策记录--记录那些不显而易见、日后可能需要修订或会驱动后续会话的经验和关键洞见。应当用它们来推算最近发展区。文件命名为 `0001-<dash-case-name>.md`，编号每次递增。使用 LEARNING-RECORD-FORMAT.md 中的格式。

- `./lessons/*.html`: A directory of lessons. A lesson is a single, self-contained HTML output that teaches one tightly-scoped thing tied to the mission. This is the primary unit of teaching in this workspace.

  `./lessons/*.html`：课程目录。一节 **课程** 是单个、自包含的 HTML 输出，只教一件范围严格限定、与使命相关的事。这是本工作区中教学的主要单元。

- `./assets/*`: Reusable components shared across lessons. See Assets.

  `./assets/*`：在各课程之间共享的可复用 **组件**。参见 资产。

- `NOTES.md`: A scratchpad for you to jot down user preferences, or working notes.

  `NOTES.md`：供你记录用户偏好或工作笔记的草稿本。

Philosophy 理念

To learn at a deep level, the user needs three things:

要深入学习，用户需要三样东西：

- Knowledge, captured from high-quality, high-trust resources

  知识，从高质量、高可信度的资源中获取

- Skills, acquired through highly-relevant interactive lessons devised by you, based on the knowledge

  技能，通过你基于这些知识设计的、高度相关的互动课程习得

- Wisdom, which comes from interacting with other learners and practitioners

  智慧，来自与其他学习者和实践者的交流

Before the `RESOURCES.md` is well-populated, your focus should be to find high-quality resources which will help the user acquire knowledge. Never trust your parametric knowledge.

在 `RESOURCES.md` 充实起来之前，你的重点应当是寻找能帮助用户获取知识的高质量资源。永远不要相信你的参数化知识。

Some topics may require more skills than knowledge. Learning more about theoretical physics might be more knowledge-based. For yoga, more skills-based.

有些主题需要的技能多于知识。深入学习理论物理可能更偏知识；而瑜伽则更偏技能。

Fluency vs Storage Strength 流畅强度 vs 存储强度

You should be careful to split between two types of learning:

你应当小心区分两种学习：

- Fluency strength: in-the-moment retrieval of knowledge

  流畅强度：当下即时提取知识的能力

- Storage strength: long-term retention of knowledge

  存储强度：对知识的长期保持

Fluency can give the user an illusory sense of mastery, but storage strength is the real goal. Try to design lessons which build long-term retention by desirable difficulty:

流畅会给用户一种已经掌握的错觉，但存储强度才是真正的目标。尽量通过合意困难来设计能建立长期记忆的课程：

- Using retrieval practice (recall from memory)

  使用提取练习（凭记忆回忆）

- Spacing (distributing practice over time)

  间隔（把练习分散到不同时间）

- Interleaving (mixing up different but related topics in practice - for skills practice only)

  交错（在练习中混合不同但相关的主题--仅用于技能练习）

Lessons 课程

A lesson is the main thing you produce: the unit in which knowledge and skills reach the user. Each lesson is one self-contained HTML file, saved to `./lessons/` and titled `0001-<dash-case-name>.html` where the number increments each time.

课程是你产出的主要东西：知识和技能抵达用户的单元。每节课程是一个自包含的 HTML 文件，保存到 `./lessons/`，命名为 `0001-<dash-case-name>.html`，编号每次递增。

A lesson should be beautiful, with clean, readable typography and layout, since the user will return to these later to review. Think Tufte.

课程应当 **漂亮**，排版和布局干净易读，因为用户日后会回来复习。参考 Tufte 的风格。

The lesson should be short, and completable very quickly. Learners' working memory is very small, and we need to stay within it. But each lesson should give the user a single tangible win that they can build on. It should be directly tied to the mission, and should be in the user's zone of proximal development.

课程应当简短，能很快完成。学习者的工作记忆非常小，我们必须待在它的容量之内。但每节课程都应给用户一个可以在此基础上继续前进的具体收获。它应当与使命直接挂钩，并且处于用户的最近发展区内。

If possible, open the lesson file for the user by running a CLI command.

如果可能，运行一条 CLI 命令为用户打开课程文件。

Each lesson should link via HTML anchors to other lessons and reference documents.

每节课程都应通过 HTML 锚点链接到其他课程和参考文档。

Each lesson should recommend a primary source for the user to read or watch. This should be the most high-quality, high-trust resource you found on the topic.

每节课程都应推荐一个供用户阅读或观看的首要来源。它应当是你在该主题上找到的质量最高、最可信的资源。

Each lesson should contain a reminder to ask followup questions to the agent. The agent is their teacher, and can assist with anything that's unclear.

每节课程都应包含一条提醒：有后续问题就向智能体提问。智能体就是他们的老师，任何不清楚的地方它都能帮忙。

Assets 资产

Lessons are built from reusable components, stored in `./assets/`: stylesheets, quiz widgets, simulators, diagram helpers, and anything else a second lesson could reuse.

课程由存放在 `./assets/` 中的可复用 **组件** 构建而成：样式表、测验小部件、模拟器、图表辅助工具，以及任何第二节课程可能复用的东西。

Reuse is the default, not the exception. Before authoring a lesson, read `./assets/` and build from the components already there. When a lesson needs something new and reusable, write it as a component in `./assets/` and link to it; never inline code a future lesson would duplicate.

复用是默认做法，而非例外。编写课程之前，先阅读 `./assets/`，并基于已有组件来构建。当一节课程需要某个新的、可复用的东西时，把它写成 `./assets/` 中的组件并链接过去；绝不要内联那些未来课程会重复编写的代码。

A shared stylesheet is the first component every workspace earns: every lesson links it, so the lessons look like one consistent course rather than a pile of one-offs. As the workspace grows, so should the component library.

共享样式表是每个工作区挣得的第一个组件：每节课程都链接它，这样所有课程看起来就像一门风格一致的课，而不是一堆零散的一次性作品。随着工作区的增长，组件库也应随之增长。

The Mission 使命

Every lesson should be tied into the mission - the reason that the user is interested in learning about the topic.

每节课程都应与使命挂钩--即用户想学习这个主题的原因。

If the user is unclear about the mission, or the `MISSION.md` is not populated, your first job should be to question the user on why they want to learn this.

如果用户对使命不清楚，或者 `MISSION.md` 还没有填写，你的首要任务就是询问用户为什么想学这个。

Failing to understand the mission will mean knowledge acquisition is not grounded in real-world goals. Lessons will feel too abstract. You will have no way of judging what the user should do next.

不理解使命，意味着知识获取没有以现实世界的目标为根基。课程会显得过于抽象。你也将无从判断用户下一步该做什么。

Missions may change as the user develops more skills and knowledge. This is normal - make sure to update the `MISSION.md` and add a learning record to capture the change. Confirm with the user before changing the mission.

随着用户技能和知识的增长，使命可能会改变。这很正常--务必更新 `MISSION.md`，并新增一条学习记录来记录这一变化。在更改使命之前先与用户确认。

Zone Of Proximal Development 最近发展区

Each lesson, the user should always feel as if they are being challenged 'just enough'.

每节课程都应让用户始终感觉受到的挑战“恰到好处”。

The user may specify an exact thing they want to learn. If they don't, figure out their zone of proximal development by:

用户可能会指定一个他们想学的具体内容。如果没有，就通过以下方式找出他们的最近发展区：

- Reading their `learning-records`

  阅读他们的 `learning-records`

- Figuring out the right thing to teach them based on their mission

  根据他们的使命推断该教他们什么

- Teach the most relevant thing that fits in their zone of proximal development

  教授最相关、且处于其最近发展区内的内容

Knowledge 知识

Lessons should be designed around a skill the user is going to learn. The knowledge in the lesson should be only what's required to acquire that skill. You teach the knowledge first, then get the user to practice the skills via an interactive feedback loop.

课程应围绕用户将要学习的一项技能来设计。课程中的知识应只包含习得该技能所需的部分。先教知识，再让用户通过互动的反馈循环练习技能。

Knowledge should first be gathered from trusted resources. Use `RESOURCES.md` to keep track of them. Lessons should be littered with citations - links to external resources to back up any claim made. This increases the trustworthiness of the lesson.

知识应首先从可信资源中收集。用 `RESOURCES.md` 来跟踪这些资源。课程中应遍布引用--链接到外部资源，为每一个论断提供支撑。这会提高课程的可信度。

For acquiring knowledge, difficulty is the enemy. It eats working memory you need for understanding.

对于获取知识，困难是敌人。它会吞噬你理解所需的工作记忆。

Skills 技能

If knowledge is all about acquisition, skills are about durability and flexibility. Make the knowledge stick.

如果说知识关乎获取，那么技能关乎持久性和灵活性。要让知识牢牢记住。

For skill acquisition, difficulty is the tool. Effortful retrieval is what builds storage strength. Skills should be taught through interactive lessons. There are several tools at your disposal:

对于习得技能，困难是工具。费力的提取才能建立存储强度。技能应通过互动课程来教授。你可以使用以下几种工具：

- Interactive lessons, using quizzes and light in-browser tasks

  互动课程，使用测验和轻量的浏览器内任务

- Lessons which guide the user through a list of real-world steps to take (for instance, yoga poses)

  引导用户按一系列现实世界步骤去做的课程（例如瑜伽体式）

Each of these should be based on a feedback loop, where the user receives feedback on their performance. This feedback loop should be as tight as possible, giving feedback immediately - and ideally automatically.

每种方式都应基于 **反馈循环**，让用户获得关于自己表现的反馈。这个反馈循环应尽可能紧凑，立即给出反馈--最好是自动给出。

For quizzes, each answer should be exactly the same number of words (and characters, if possible). Vary which position holds the correct answer across questions. Don't give the user any clues about the answer through formatting or order.

对于测验，每个选项的词数应完全相同（如果可能，字符数也相同）。在不同题目间变换正确答案所在的位置。不要通过格式或顺序给用户任何关于答案的线索。

Acquiring Wisdom 获取智慧

Wisdom comes from true real-world interaction - testing your skills outside the learning environment.

智慧来自真正的现实世界互动--在学习环境之外检验你的技能。

When the user asks a question that appears to require wisdom, your default posture should be to attempt to answer - but to ultimately delegate to a community.

当用户提出的问题看起来需要智慧时，你的默认姿态应当是尝试回答--但最终要将其交给一个 **社区**。

A community is a place (online or offline) where the user can test their skills in the real world. This might be a forum, a subreddit, a real-world class (budget permitting) or a local interest group.

社区是用户可以在现实世界中检验技能的地方（线上或线下）。它可能是一个论坛、一个 subreddit、一门现实中的课程（预算允许的话）或一个本地兴趣小组。

You should attempt to find high-reputation communities the user can join. If the user expresses a preference that they don't want to join a community, respect it.

你应当尝试找到用户可以加入的高声誉社区。如果用户表示不想加入社区，尊重这一偏好。

Reference Documents 参考文档

While creating lessons, you should also create reference documents. Lessons can reference these documents - they are useful for tracking raw units of knowledge useful across lessons.

在创建课程的同时，你也应当创建参考文档。课程可以引用这些文档--它们可用于跟踪在多节课程间通用的基本知识单元。

Lessons will rarely be revisited later - reference documents will be. They should be the compressed essence of the lesson, in a format designed for quick reference.

课程日后很少会被重温--参考文档会。它们应当是课程的压缩精华，采用专为快速查阅设计的格式。

Some learning topics lend themselves to reference:

有些学习主题天然适合做成参考资料：

- Syntax and code snippets for programming

  编程的语法和代码片段

- Algorithms and flowcharts for processes

  流程的算法和流程图

- Yoga poses and sequences for yoga

  瑜伽的体式和序列

- Exercises and routines for fitness

  健身的动作和训练计划

- Glossaries for any topic with its own nomenclature

  任何有自己专有术语的主题的术语表

Glossaries, in particular, are an essential reference. Once one is created, it should be adhered to in every lesson.

尤其是术语表，它是必不可少的参考资料。一旦创建，每节课程都应遵循它。

NOTES.md NOTES.md

The user will sometimes express preferences of how they want to be taught, or things you should keep in mind. This is the place to record those preferences, so you can refer back to them when designing lessons or working with the user.

用户有时会表达他们希望怎样被教，或者你应当记住的事情。这里就是记录这些偏好的地方，以便你在设计课程或与用户协作时回头参考。

REFERENCE 参考文档 · GLOSSARY-FORMAT.md

GLOSSARY.md Format GLOSSARY.md 格式

Source 原文

GLOSSARY.md is the canonical language for this teaching workspace. All explainers, exercises, and learning records should adhere to its terminology. Building it is itself part of learning: compressing a concept into a tight definition is evidence the user understands it.

GLOSSARY.md 是这个教学工作区的规范语言。所有讲解、练习和学习记录都应遵循其中的术语。构建术语表本身就是学习的一部分：能把一个概念压缩成精炼的定义，就证明用户理解了它。

Structure 结构

```
# {Topic} Glossary

{One or two sentence description of the topic this glossary covers.}

## Terms

**Hypertrophy**:
Muscle growth driven by mechanical tension and metabolic stress over repeated training sessions.
_Avoid_: Bulking, getting big

**Progressive overload**:
Systematically increasing the demand on a muscle over time, via load, volume, or intensity.
_Avoid_: Pushing harder, levelling up

**RPE (Rate of Perceived Exertion)**:
A 1-10 self-rating of how hard a set felt, where 10 is failure and 8 means two reps left in the tank.
_Avoid_: Effort score, intensity rating
```

```
# {Topic} Glossary

{用一两句话描述这份术语表所涵盖的主题。}

## Terms

**Hypertrophy**:
Muscle growth driven by mechanical tension and metabolic stress over repeated training sessions.
_Avoid_: Bulking, getting big

**Progressive overload**:
Systematically increasing the demand on a muscle over time, via load, volume, or intensity.
_Avoid_: Pushing harder, levelling up

**RPE (Rate of Perceived Exertion)**:
A 1-10 self-rating of how hard a set felt, where 10 is failure and 8 means two reps left in the tank.
_Avoid_: Effort score, intensity rating
```

Rules 规则

- Add a term only when the user understands it. The glossary is a record of compressed knowledge, not a dictionary the user reads to learn. If the user has just been introduced to a concept, wait until they can use it correctly before promoting it here.

  只在用户理解某个术语后才添加它。 术语表是压缩知识的记录，而不是供用户阅读学习的词典。如果用户刚刚接触某个概念，要等到他们能正确使用它，再把它提升到这里。

- Be opinionated. When several words exist for the same concept, pick the best one and list the rest as aliases to avoid. This is how language compresses.

  要有主见。 当同一个概念有多个说法时，挑出最好的那个，把其余的列为应避免的别名。语言就是这样被压缩的。

- Keep definitions tight. One or two sentences. Define what the term IS, not what it does or how to do it.

  定义要精炼。 一两句话。定义这个术语 **是什么**，而不是它做什么或怎么做。

- **Use the glossary's own terms inside definitions.** Once a term is in the glossary, prefer it everywhere, including inside other definitions. This is what makes complex terms easier to grasp later.

  在定义中使用术语表自己的术语。 一旦某个术语进入术语表，就处处优先使用它，包括在其他术语的定义里。正是这一点让复杂术语日后更容易理解。

- **Group under subheadings** when natural clusters emerge (e.g. `## Anatomy`, `## Programming`). A flat list is fine when terms cohere.

  自然形成聚类时按小标题分组（例如 `## Anatomy`、`## Programming`）。如果术语之间本就连贯，平铺列表也可以。

- **Flag ambiguities explicitly.** If a term is used loosely in the wider field, note the resolution: "In this workspace, 'set' always means a working set; warm-ups are tracked separately."

  明确标出歧义。 如果某个术语在更广泛的领域里用法松散，注明这里的取舍：“在本工作区中，‘set’始终指正式组；热身组单独记录。”

- **Revise as understanding deepens.** A definition the user wrote in week one may be wrong by week six. Update in place; do not leave stale entries.

  随着理解加深而修订。 用户第一周写下的定义到第六周可能就是错的。原地更新；不要留下过时的条目。

REFERENCE 参考文档 · LEARNING-RECORD-FORMAT.md

Learning Record Format 学习记录格式

Source 原文

Learning records live in `./learning-records/` and use sequential numbering: `0001-slug.md`, `0002-slug.md`, etc. Create the directory lazily: only when the first record is written.

学习记录存放在 `./learning-records/` 中，使用顺序编号：`0001-slug.md`、`0002-slug.md` 等。按需创建该目录：只在写第一条记录时才创建。

They are the teaching equivalent of ADRs: they capture non-obvious lessons, key insights, and stated prior knowledge that will steer future sessions. They are used to calculate the zone of proximal development.

它们在教学中相当于 ADR：记录不显而易见的经验、关键洞见，以及用户自述的已有知识，这些将引导后续会话。它们用于推算最近发展区。

Template 模板

```
# {Short title of what was learned or established}

{1-3 sentences: what was learned (or what prior knowledge was established), and why it matters for future sessions.}
```

```
# {对所学内容或所确认事项的简短标题}

{1-3 句话：学到了什么（或确认了哪些已有知识），以及它为何对后续会话重要。}
```

That is the whole format. A learning record can be a single paragraph. The value is recording _that_ this is now known and _why_ it changes what to teach next, not in filling out sections.

这就是全部格式。一条学习记录可以只有一段话。它的价值在于记下 _这件事_ 现在已经掌握，以及它 _为何_ 改变了接下来要教的内容，而不在于把各个小节填满。

Optional sections 可选小节

Only include these when they add genuine value. Most records won't need them.

只在确实有价值时才加入这些小节。大多数记录都用不上。

- Status frontmatter (`active | superseded by LR-NNNN`): useful when an earlier understanding turns out to be wrong and is replaced.

  Status frontmatter（`active | superseded by LR-NNNN`）：当早先的理解被证明有误并被替换时很有用。

- Evidence: how the user demonstrated the understanding (a question answered, an exercise completed, prior experience cited). Useful when the claim might be revisited.

  Evidence：用户如何展示了这一理解（答对了一个问题、完成了一个练习、引用了以往经验）。当这一结论可能被重新审视时很有用。

- Implications: what this unlocks or rules out for future sessions. Worth recording when non-obvious.

  Implications：这为后续会话解锁了什么或排除了什么。在不显而易见时值得记录。

Numbering 编号

Scan `./learning-records/` for the highest existing number and increment by one.

扫描 `./learning-records/`，找到现有的最大编号，然后加一。

When to write a learning record 何时写学习记录

Write one when any of these is true:

满足以下任一条件时写一条：

1. The user demonstrated genuine understanding of something non-trivial: not just exposure, but evidence they can use the concept correctly. This sets a new floor for what to teach next.

   用户对某个不简单的内容展示了真正的理解：不只是接触过，而是有证据表明他们能正确运用这个概念。这为接下来要教的内容设定了新的下限。

2. The user disclosed prior knowledge: "I already know X." Record it so future sessions don't re-teach it. Also record the _depth_ claimed.

   用户透露了已有知识：“我已经懂 X 了。”记录下来，以免后续会话重复教授。同时记录其自称的 _深度_。

3. A misconception was corrected: the user previously believed something wrong and now sees why. These are high-value: they predict future stumbling blocks for related topics.

   纠正了一个误解：用户之前相信某个错误的东西，现在明白了为什么错。这类记录价值很高：它们能预示相关主题上未来会遇到的绊脚石。

4. The mission shifted in response to learning: the user discovered they cared about something different than they thought. Cross-link to [[MISSION.md]] and update it.

   使命因学习而发生了转变：用户发现自己在意的东西和原先以为的不同。交叉链接到 [[MISSION.md]] 并更新它。

What does _not_ qualify 什么 _不_ 算

- Material that was merely covered. Coverage is not learning. Wait for evidence.

  仅仅讲过的内容。讲过不等于学会。等待证据。

- Anything already captured tersely in [[GLOSSARY.md]] as a term definition. Don't duplicate.

  已经在 [[GLOSSARY.md]] 中作为术语定义简洁记录过的任何内容。不要重复。

- Session-by-session activity logs. Learning records are not a journal: they are decision-grade insights.

  逐次会话的活动日志。学习记录不是日记：它们是决策级的洞见。

Supersession 取代

When a later record contradicts an earlier one (the user's understanding deepened or corrected), mark the old record `Status: superseded by LR-NNNN` rather than deleting it. The history of how understanding evolved is itself useful signal.

当后来的记录与早先的记录相矛盾时（用户的理解加深了或被纠正了），把旧记录标记为 `Status: superseded by LR-NNNN`，而不是删除它。理解如何演变的历史本身就是有用的信号。

REFERENCE 参考文档 · MISSION-FORMAT.md

MISSION.md Format MISSION.md 格式

Source 原文

`MISSION.md` lives at the workspace root. It captures the _reason_ the user is learning this topic. Every teaching decision (what to teach next, which resources to surface, which exercises to design) should trace back to this document.

`MISSION.md` 位于工作区根目录。它记录用户学习这个主题的 _原因_。每一个教学决策（接下来教什么、展示哪些资源、设计哪些练习）都应能追溯到这份文档。

Template 模板

```
# Mission: {Topic}

## Why
{1-3 sentences. The concrete real-world goal the user is chasing. What changes in their life or work when they have this skill? Av
  oid abstract framings like "to understand X"; push for the underlying outcome.}

## Success looks like
- {A specific, observable thing the user will be able to do}
- {Another specific thing}
- {…}

## Constraints
- {Time, budget, prior commitments, learning preferences, anything that bounds the approach}

## Out of scope
- {Adjacent topics the user explicitly does not want to chase right now, protecting the zone of proximal development}
```

```
# Mission: {Topic}

## Why
{1-3 句话。用户所追求的具体现实目标。掌握这项技能后，他们的生活或工作会有什么改变？避免“为了理解 X”这类抽象表述；追
  问出背后真正的结果。}

## Success looks like
- {用户将能够做到的一件具体、可观察的事}
- {另一件具体的事}
- {…}

## Constraints
- {时间、预算、既有安排、学习偏好，以及任何限定做法的因素}

## Out of scope
- {用户明确表示目前不想涉足的相邻主题，用以保护最近发展区}
```

Rules 规则

- **One mission per workspace.** If the user wants to learn two unrelated things, that is two workspaces.

  每个工作区只有一个使命。 如果用户想学两件不相关的事，那就是两个工作区。

- **Concrete over abstract.** "Run a half marathon by October" beats "get fitter." "Ship a Rust CLI to my team" beats "learn Rust."

  具体胜过抽象。 “在十月前跑完半程马拉松”胜过“变得更健康”。“给团队交付一个 Rust CLI”胜过“学 Rust”。

- **Push back on vagueness.** If the user cannot articulate why, interview them before writing anything. A bad mission is worse than no mission.

  对含糊其辞要追问。 如果用户说不清为什么，在写任何东西之前先访谈他们。糟糕的使命比没有使命更糟。

- **Revise when reality shifts.** Missions change. When the user's goal moves, update this file: don't leave a stale mission steering future sessions.

  现实变化时就修订。 使命会变。当用户的目标改变时，更新这个文件：不要让过时的使命继续引导后续会话。

- **Keep it short.** If `MISSION.md` runs past a screen, it has stopped being a compass and started being a plan.

  保持简短。 如果 `MISSION.md` 超过一屏，它就不再是指南针，而开始变成计划书了。

REFERENCE 参考文档 · RESOURCES-FORMAT.md

RESOURCES.md Format RESOURCES.md 格式

Source 原文

`RESOURCES.md` is the curated set of trusted sources for this topic. Knowledge for explainers should be drawn from here, not from parametric guesses. Wisdom comes from the communities listed here.

`RESOURCES.md` 是这个主题经过精选的可信来源集合。讲解所用的知识应从这里获取，而不是来自参数化的猜测。智慧则来自这里列出的社区。

Structure 结构

```
# {Topic} Resources

## Knowledge

- [Book: _The Science and Practice of Strength Training_ by Zatsiorsky & Kraemer](https://example.com)
  Foundational text on programming and adaptation. Use for: anything to do with periodisation, recovery, intensity zones.
- [Article: "How Much Should I Train?" by Greg Nuckols (Stronger By Science)](https://example.com)
  Evidence-based review of volume landmarks. Use for: weekly set targets per muscle group.

## Wisdom (Communities)

- [r/weightroom](https://reddit.com/r/weightroom)
  High-signal subreddit, moderated against bro-science. Use for: programme critique, plateau troubleshooting.
- Local: Tuesday strength class at {gym name}
  Use for: real-time coaching feedback on lifts.
```


```
# {Topic} Resources

## Knowledge

- [Book: _The Science and Practice of Strength Training_ by Zatsiorsky & Kraemer](https://example.com)
  Foundational text on programming and adaptation. Use for: anything to do with periodisation, recovery, intensity zones.
- [Article: "How Much Should I Train?" by Greg Nuckols (Stronger By Science)](https://example.com)
  Evidence-based review of volume landmarks. Use for: weekly set targets per muscle group.

## Wisdom (Communities)

- [r/weightroom](https://reddit.com/r/weightroom)
  High-signal subreddit, moderated against bro-science. Use for: programme critique, plateau troubleshooting.
- Local: Tuesday strength class at {gym name}
  Use for: real-time coaching feedback on lifts.
```


Rules 规则

- High-trust only. Prefer primary sources, recognised experts, peer-reviewed work, and communities with strong moderation. If a resource is marketing dressed as education, leave it out.

  只收高可信度来源。 优先选择一手来源、公认专家、同行评审的成果，以及管理严格的社区。如果某个资源是披着教育外衣的营销，就别收录。

- Annotate every entry. A bare link is useless in three months. Add one line: what it covers and when to reach for it.

  每个条目都要加注释。 光秃秃的链接三个月后就毫无用处。加一行说明：它涵盖什么，以及什么时候该去查它。

- Group by Knowledge / Wisdom. Mirrors the philosophy in SKILL.md. It is fine for a resource to appear in only one group.

  按 Knowledge / Wisdom 分组。 这与 SKILL.md 中的理念相呼应。一个资源只出现在其中一组也完全没问题。

- **Surface gaps explicitly.** If no good resource exists for an area the mission needs, write a `## Gaps` section listing what is missing. This drives future search.

  明确列出空白。 如果使命需要的某个领域没有好的资源，就写一个 `## Gaps` 小节列出缺失的内容。这会驱动未来的搜索。

- **Prune ruthlessly.** A resource that turned out to be wrong, shallow, or off-mission should be removed, not buried. Better five sharp sources than thirty mediocre ones.

  无情地修剪。 被证明是错误的、肤浅的或偏离使命的资源应当删除，而不是埋在列表里。五个犀利的来源胜过三十个平庸的来源。

- **Record community preferences.** If the user has opted out of joining communities, note it here so future sessions don't keep proposing them.

  记录对社区的偏好。 如果用户选择不加入社区，在这里注明，以免后续会话一再提议。

CHAPTER 25 · SKILL

To Questionnaire
转为问卷

Turn a decision you can't fully answer into a questionnaire for someone else to fill in.

把一个你无法完全回答的决策，转成一份交给别人填写的问卷。

Source 原文 · v1.3.1 · b0618bc4

Turn something the user can't answer alone into a **questionnaire**: a Markdown document they hand to one person to fill in async, or fill out together over a meeting. The recipient holds knowledge the user lacks; the questionnaire pulls it out of them.

把用户无法独自回答的事情转成一份 **问卷**：一份 Markdown 文档，交给某一个人异步填写，或在会议中一起填写。接收者掌握用户所缺的知识；问卷负责把这些知识从他们那里挖出来。

Grill the send, not the subject. Interview the user only about the _send_, which they can always answer: who it goes to, and what they need back. The questions in the document then target the **gap** between what the recipient knows and what the user needs.

深度追问的是发送这件事，而不是主题本身。 只就 _发送_ 访谈用户，这部分他们总能回答：发给谁，以及需要拿回什么。然后，文档中的问题瞄准的是接收者所知与用户所需之间的 **缺口**。

1. **Who is it going to?** Ask, in one exchange, the recipient's role, expertise, and relationship to the user. This fixes the questionnaire's tone and how much context it must carry. Done when you know who the recipient is and what they know that the user doesn't.

   发给谁？ 在一轮交流中询问接收者的角色、专长以及与用户的关系。这决定了问卷的语气，以及它需要携带多少背景。当你知道接收者是谁、以及他们知道哪些用户不知道的东西时，这一步就完成了。

2. **What do you need back?** Ask, in one exchange, the specific decisions or facts the user can't resolve alone and needs from this person. Done when you have a concrete list of what the user must walk away able to do or decide.

   你需要拿回什么？ 在一轮交流中询问用户无法独自解决、需要从这个人那里得到的具体决策或事实。当你得到一份具体清单，列明用户最终必须能够做到或决定的事情时，这一步就完成了。

3. **Write the questionnaire.** Draft questions aimed at the gap from steps 1-2, following the Document structure below. Write it to `to-questionnaire-<slug>.md` in the current directory (slug from the topic) and report the path. Done when the file exists and every item the user named in step 2 is covered by a question.

   撰写问卷。 按照下面的“文档结构”，起草针对第 1-2 步所发现缺口的问题。将其写入当前目录下的 `to-questionnaire-<slug>.md`（slug 取自主题），并报告路径。当文件已存在、且用户在第 2 步中提到的每一项都有问题覆盖时，这一步就完成了。

Document structure 文档结构

Frame the document as a **discovery questionnaire**: the user lacks context, the recipient holds it. Order questions most-important-first, since async means you may only get one pass, and group them under `##` headings by theme once there are more than a handful. Write it using the template below.

把文档定位为一份 **探索式问卷**：用户缺少背景，接收者掌握背景。按重要性从高到低排列问题，因为异步意味着你可能只有一次机会；一旦问题超过少数几个，就按主题把它们归入 `##` 标题之下。使用下面的模板撰写。

<questionnaire-template>

<questionnaire-template>

<Questionnaire title> <问卷标题>

Purpose: why this questionnaire exists and the decision riding on it.

目的： 这份问卷为何存在，以及取决于它的决策。

From: <the user>, **To:** <the recipient>, **How your answers will be used:** <where they go>

发件人： <用户>，**收件人：** <接收者>，**你的回答将如何使用：** <回答的去向>

Context 背景

One paragraph orienting a recipient who wasn't in the user's head. Enough to answer well, not a page.

用一段话让不了解用户想法的接收者快速进入状态。足以让对方答好即可，不要写成一整页。

How to answer 如何作答

Deadline and rough effort. Partial answers and "I don't know" are useful: flag anything you're unsure of rather than skipping it.

截止时间和大致工作量。部分回答和“我不知道”都有用：对于不确定的内容，请标注出来，而不是跳过。

<Theme heading> <主题标题>

One `##` section per theme. Under each, its questions, most-important-first. Every question is one idea, never compound, with an answer stub directly beneath, and a one-line _why this matters_ only where the question could be misread or invite a throwaway answer.

每个主题一个 `##` 小节。每个小节下列出其问题，按重要性从高到低排列。每个问题只问一件事，绝不复合，问题正下方紧跟一个作答占位；只有当问题可能被误读或容易招来敷衍回答时，才附上一行 _为什么重要_。

<question-example>

<question-example>

What load is the system expected to handle at launch? 系统在上线时预计要承受多大的负载？

Why this matters: it decides whether we provision for burst traffic now or defer it.

为什么重要：它决定了我们是现在就为突发流量预留资源，还是推迟处理。

> </question-example>

> </question-example>

Anything else? 还有其他吗？

A closing catch-all: anything we didn't ask that we should know?

一个收尾的兜底问题：有没有我们没问到、但应该知道的事？

</questionnaire-template>

</questionnaire-template>

CHAPTER 26 · SKILL

Wait What
等等，什么？

Stop. That last message did not land: re-pitch it.

停。上一条消息没被理解：重新讲一遍。

Source 原文 · v1.3.1 · b0618bc4

Wait, I don't understand where you've got to here. Re-pitch that: give me a little bit of context, talk in ASD-STE100 Simplified Technical English, and use the ubiquitous language from `GLOSSARY.md` (follow `GLOSSARY-MAP.md` to the right one if the repo has more than one).

等等，我不明白你讲到哪儿了。重新讲一遍：给我一点背景，用 ASD-STE100 简化技术英语（Simplified Technical English）来说，并使用 `GLOSSARY.md` 中的通用语言（如果仓库里有不止一份，就按 `GLOSSARY-MAP.md` 找到正确的那份）。

CHAPTER 27 · SKILL

Writing for Agents
为智能体写作

Writing documents for agents. Use when creating or editing skills, or modifying AGENTS.md or CLAUDE.md.

为智能体编写文档。在创建或编辑技能，或修改 AGENTS.md 或 CLAUDE.md 时使用。

Source 原文 · v1.3.1 · b0618bc4

Reference for writing any document an agent consumes: a skill, an AGENTS.md / CLAUDE.md, a doc reached by a pointer. The packaging differs; the writing does not: the same levers make each one predictable, since the agent takes the same _process_ every run rather than producing the same output.

本参考适用于编写任何供智能体读取的文档：技能、AGENTS.md / CLAUDE.md，或经由指针抵达的文档。包装形式不同，写法却一样：同一批杠杆让每一种文档都变得可预测，因为智能体每次运行都走同一套 _流程_，而不是产出同样的结果。

When the document you're writing is a skill, read SKILL-MECHANICS.md for frontmatter, invocation choice, and router skills.

如果你要写的文档是技能，请阅读 SKILL-MECHANICS.md，了解 frontmatter、调用方式的选择以及路由技能。

Context pointers 上下文指针

A **context pointer** is a reference held in the agent's context that names some out-of-context material and encodes the condition for reaching it. A skill's description is one; a line in AGENTS.md naming a doc is the same object. The pointer's _wording_, not its target, decides when the agent reaches the material, and how reliably. A must-have target behind a weakly worded pointer is a variance bug: sharpen the wording first, and inline the material only if sharpening fails.

上下文指针（context pointer）是保存在智能体上下文中的一条引用，它指名某份不在上下文中的材料，并编码了抵达该材料的条件。技能的 description 就是一种；AGENTS.md 里点名某份文档的一行字也是同一种东西。决定智能体何时抵达材料、抵达得有多可靠的，是指针的 _措辞_，而不是它指向的目标。一份必不可少的目标材料藏在措辞软弱的指针后面，就是一个方差 bug：先把措辞磨锋利，只有磨锋利仍不奏效时，才把材料内联进来。

A pointer does two jobs: state what the material is, and list the **branches** that should trigger reaching it (a branch is a distinct case the document handles, so different runs take different paths through it). Every word of an always-loaded pointer costs on every turn, so it earns even harder pruning than the body:

指针承担两项工作：说明材料是什么，并列出应当触发抵达它的**分支**（分支是文档所处理的一种独立情形，因此不同的运行会沿不同路径穿过它）。始终加载的指针，每个字在每一轮都要付出代价，所以它比正文更应接受严苛的修剪：

- Front-load the leading word: the pointer is where it does its triggering work.

  把引导词放在最前面：指针正是引导词发挥触发作用的地方。

- One trigger per branch. Synonyms that rename a single branch are one branch written twice; collapse them and keep only genuinely distinct branches.

  每个分支一个触发词。 为同一分支换个说法的同义词，只是把一个分支写了两遍；把它们合并，只保留真正不同的分支。

- Cut identity the body already carries.

  删掉正文已经承载的身份说明。

The two loads 两种负载

Every document and pointer you add spends one of two budgets:

你加入的每份文档、每个指针，都在消耗两种预算之一：

- Context load is the cost of always-loaded material on the agent's window: an `AGENTS.md` line, a skill description, anything sitting in context every turn, spending tokens and attention whether or not it fires.

  上下文负载是始终加载的材料施加在智能体窗口上的成本：`AGENTS.md` 里的一行、技能的 description，任何每一轮都待在上下文里的东西，无论是否触发，都在消耗 token 和注意力。

- Cognitive load is the cost on the human: which documents exist and when to reach for each. The human is the index. Not a cost to minimise: it is the price of human agency; spend it where human judgement matters, remove it where it does not.

  认知负载是施加在人身上的成本：有哪些文档存在，以及各自该在何时取用。人就是索引。它不是要压到最低的成本：它是人类能动性的代价；在需要人类判断的地方花掉它，在不需要的地方去掉它。

Material reached only through a pointer escapes context load at the price of the pointer's own line; material with no pointer at all rides entirely on cognitive load.

只通过指针抵达的材料，以指针自身那一行为代价逃过了上下文负载；完全没有指针的材料，则全部压在认知负载上。

Information hierarchy 信息层级

A document is built from two content types: steps (the ordered actions the agent performs) and reference (definitions, rules, facts consulted on demand). The two mix freely: all steps (a recipe), all reference (a review's rules, this skill), or both. The core decision is where each piece sits on the information hierarchy, a ladder ranked by how immediately the agent needs the material:

文档由两类内容构成：**步骤**（智能体按顺序执行的动作）和**参考**（按需查阅的定义、规则、事实）。两者可以自由混合：全是步骤（一份配方），全是参考（一次评审的规则、本技能），或两者兼有。核心决策是每一部分在**信息层级**上处于哪个位置；信息层级是一架梯子，按智能体需要该材料的紧迫程度排序：

1. In-file step is the primary tier: what the agent does, in order.

   文件内步骤是首要层级：智能体要做什么，按什么顺序做。

2. In-file reference is consulted on demand. Often a legitimately flat peer-set (every rule of a review on one rung), which is a fine arrangement, not a smell.

   文件内参考按需查阅。它常常是一组合理的扁平并列项（一次评审的所有规则处在同一级），这是很好的组织方式，不是坏味道。

3. Disclosed reference is pushed out into a separate file, reached by a context pointer, loaded only when the pointer fires. Spans a sibling file in the same folder through fully external reference that lives anywhere and any document can point at.

   披露式参考被推到单独的文件里，经由上下文指针抵达，只在指针触发时加载。它的范围从同一文件夹中的兄弟文件，一直延伸到完全外部的参考：可以放在任何地方、任何文档都能指向它。

Push too little down and the top bloats; push too much and you hide material the agent actually needs. That tension is the whole decision.

往下推得太少，顶层就会臃肿；推得太多，就会把智能体确实需要的材料藏起来。这种张力就是整个决策的全部。

Progressive disclosure is the move down the ladder (out of the main file and behind a pointer) so the top stays legible. Not primarily a token optimisation: it is how the hierarchy is protected. Branching is the cleanest disclosure test: inline what every branch needs, and push behind a pointer what only some branches reach. When a document has steps, in-file reference that should be disclosed buries them and turns attending to them into a coin-flip: a variance lever, not just a legibility one.

渐进式披露是沿梯子向下的移动（移出主文件、放到指针后面），让顶层保持清晰易读。它主要不是为了节省 token：它是保护层级的方式。分支是最干净的披露测试：每个分支都需要的内容就内联，只有部分分支会用到的内容就推到指针后面。当文档含有步骤时，本该披露出去的文件内参考会把步骤淹没，让智能体是否留意这些步骤变成抛硬币：这是一个方差杠杆，而不只是可读性杠杆。

Co-location is the within-file companion: where the ladder decides _how far down_ a piece sits, co-location decides _what sits beside it_ once there. Keep a concept's definition, rules, and caveats under one heading rather than scattered, so reading one part brings its neighbours with it. The test: the document should read like documentation written for the agent. Grouped material reads that way; scattered material does not. (Distinct from duplication: that repeats one meaning in two places; scattering fragments one meaning across many.)

就近组织（co-location）是文件内部的配套手段：梯子决定一块内容 _往下放多远_，就近组织决定它放到那里后 _旁边放什么_。把一个概念的定义、规则和注意事项放在同一个标题下，而不是四处散落，这样读到其中一部分时，相邻内容也会一并带上。检验标准：这份文档读起来应当像是专为智能体写的文档。成组的材料读起来是这样；散落的材料则不是。（这与重复不同：重复是把同一含义放在两处；散落是把同一含义拆碎到许多地方。）

Sprawl is the failure mode here: a document simply too long, even when every line is live and unique. Attention thins across the excess, and every extra line is one more to keep relevant. The cure is the ladder: disclose reference behind pointers, and split by branch or sequence so each path carries only what it needs.

蔓延（sprawl）是这里的失败模式：文档就是太长了，即便每一行都仍然有效且不重复。注意力会在多余的部分上被摊薄，而每多一行，就多一行需要保持相关。解药就是这架梯子：把参考披露到指针后面，并按分支或顺序拆分，让每条路径只携带它需要的内容。

Steps and completion criteria 步骤与完成标准

Every step ends on a **completion criterion**, the condition that tells the agent the work is done. Two properties make it a lever:

每个步骤都以一个**完成标准**结束，即告诉智能体工作已经完成的条件。有两个属性让它成为杠杆：

- **Clarity**: can the agent tell done from not-done? A vague bound ("understanding reached") invites **premature completion**: ending the step before it is genuinely done, attention slipping to _being done_. The visible steps still ahead (the **post-completion steps**) supply the pull; the criterion's clarity is the resistance. Defend in order: **sharpen the bound first** (local and cheap); only if it is irreducibly fuzzy _and_ you observe the rush, hide the later steps by splitting the sequence. Hiding only works across a real context boundary (a hand-off or a subagent dispatch; an inline call leaves the later steps in context and clears nothing).

  清晰度：智能体能否分辨完成与未完成？模糊的界限（“已达成理解”）会招致**过早完成**：在步骤真正完成之前就结束它，注意力滑向了 _已经完成_ 这件事。前方可见的步骤（**完成后步骤**）提供了拉力；标准的清晰度则是阻力。按顺序防御：**先把界限磨锋利**（局部且廉价）；只有当它本质上无法消除模糊、_并且_ 你确实观察到了赶工时，才通过拆分序列把后续步骤藏起来。隐藏只有跨越真正的上下文边界时才有效（一次交接或一次子智能体派发；内联调用会把后续步骤留在上下文中，什么也没清掉）。

- **Demand**: how much it requires. "Every modified model accounted for" forces thorough work where "produce a change list" does not. Demand drives **legwork** (the digging the agent does within the work, latent in the wording rather than written as its own step), and it is not step-bound: "every rule applied" binds a body of flat reference just as "every step done" binds a sequence, which is how an all-reference document still carries an exhaustiveness bar.

  要求度：它要求多少。“每个被修改的模型都已处理到”会逼出彻底的工作，而“产出一份变更清单”不会。要求度驱动**跑腿功夫**（legwork，智能体在工作中做的深挖，隐含在措辞里，而不是作为单独一步写出来），而且它不受步骤约束：“每条规则都已应用”约束一组扁平参考，正如“每个步骤都已完成”约束一个序列，这就是为什么一份纯参考文档同样能带有穷尽性门槛。

The strongest criteria are both checkable and exhaustive.

最强的标准既可检验，又穷尽无遗。

When to split 何时拆分

Splitting one document into two spends one of the two loads, so split only when the cut earns it:

把一份文档拆成两份，会消耗两种负载之一，所以只有在这一刀值得时才拆：

- **By sequence**: split a run of steps where the post-completion steps tempt the agent to rush the one in front of it. Keeping them out of view drives more legwork on the current task. Beware the reverse: merging sequences exposes each step's later steps to what follows, inviting premature completion.

  按顺序拆：当一串步骤中的完成后步骤诱使智能体草草对付眼前这一步时，就把这串步骤拆开。让它们不在视野内，能促使智能体在当前任务上下更多功夫。小心反方向：合并序列会把每一步的后续步骤暴露给它，招致过早完成。

- By invocation, skill-specific: see SKILL-MECHANICS.md.

  按调用方式拆，这是技能特有的：见 SKILL-MECHANICS.md。

Leading words 引导词

A leading word is a compact concept already living in the model's pretraining that the agent thinks with while running the document (_lesson_, _fog of war_, _tracer bullets_). Repeated as a token, never as a sentence, it accumulates a distributed definition and anchors a whole region of behaviour in the fewest tokens, by recruiting priors the model already holds. Coining your own works if you define it clearly, but a made-up word recruits no priors: you pay in definition tokens what a pretrained word gives free; reach for an existing word first.

引导词（leading word）是一个早已存在于模型预训练中的紧凑概念，智能体在运行文档时借助它来思考（_lesson_（教训）、_fog of war_（战争迷雾）、_tracer bullets_（曳光弹））。它以词元的形式反复出现，从不以句子的形式出现，于是积累出一个分布式的定义，并通过调动模型已有的先验，用最少的 token 锚定一整片行为区域。自己造词也行，只要你把它定义清楚；但一个生造的词调动不了任何先验：预训练词免费给你的东西，你得用定义的 token 来付费；先去找现成的词。

It anchors twice. In the body, _execution_: the agent reaches for the same behaviour every time the word appears, and inside flat reference it focuses attention on a class of thing to look for. In a pointer, _invocation_: when the same word lives in your prompts, your docs, and your codebase, the agent links that shared language to the material and reaches it more reliably.

它会锚定两次。在正文中，锚定的是 _执行_：每当这个词出现，智能体都会采取同样的行为；在扁平参考里，它能把注意力聚焦到要寻找的某一类事物上。在指针中，锚定的是 _调用_：当同一个词同时存在于你的提示词、你的文档和你的代码库中时，智能体会把这套共享语言与材料关联起来，更可靠地抵达它。

Hunt for opportunities to refactor with leading words. A triad spelled out at three sites, a pointer spending a sentence to gesture at one idea. Each is a passage begging to collapse into a single token:

主动寻找用引导词重构的机会。一个在三处被完整拼写出来的三元组，一个花一整句话只为比划出一个想法的指针。每一处都是迫切想要坍缩成单个词元的段落：

- "fast, deterministic, low-overhead" → _tight_ (a _tight_ loop).

  “快速、确定、低开销” → _tight_（紧凑；一个 _tight_ 的循环）。

- "a loop you believe in" → _red_, turning a fuzzy gate into a binary observable state (the loop goes _red_ on the bug, or it doesn't).

  “一个你信得过的循环” → _red_（变红），把模糊的关卡变成一个二元的可观察状态（循环要么在这个 bug 上变 _red_，要么不会）。

You win twice: fewer tokens, and a sharper hook for the agent to hang its thinking on. Assume every document is carrying restatements that leading words retire. Go find them.

你赢两次：token 更少，智能体也有了一个更锋利的钩子来挂住它的思考。假定每份文档都背着可以用引导词清退的重复表述。去把它们找出来。

Negation is the failure mode beside this lever: steering by prohibition drags the forbidden behaviour into context and makes it _more_ available, not less. _Don't think of an elephant_, and the elephant is all there is; the negation is a weak modifier the strongly-activated concept overruns, so the ban half-reads as an instruction to do the thing. Prompt the **positive**: state the target behaviour ("write one-line comments") so the banned one is never spoken. A prohibition earns its place only as a hard guardrail you cannot phrase positively; even then, pair it with the positive target so attention lands on what to do.

否定是这个杠杆旁边的失败模式：用禁令来引导，会把被禁止的行为拖进上下文，让它 _更_ 容易被调用，而不是更难。_别去想一头大象_，于是满脑子都是大象；否定是一个弱修饰语，会被强烈激活的概念压过去，于是禁令有一半被读成了去做那件事的指令。用**正面**表述来提示：说出目标行为（“写一行注释”），让被禁止的行为根本不被提起。只有当某条硬性护栏无法用正面方式表达时，禁令才配占一席之地；即便如此，也要把它与正面目标配对，让注意力落在该做什么上。

Pruning 修剪

- Keep each meaning in a **single source of truth**: one authoritative place, so changing the behaviour is a one-place edit. **Duplication** (the same meaning in more than one place) costs maintenance and tokens, and inflates a meaning's prominence on the ladder past its real rank. (The accidental inverse of a leading word, which repeats a token on purpose, never the meaning.)

  让每个含义都只有一个**单一事实来源**：一个权威位置，这样改变行为只需改一处。**重复**（同一含义出现在不止一处）会带来维护成本和 token 成本，并把该含义在梯子上的显著程度抬高到超过它的真实级别。（它是引导词意外形成的反面：引导词是有意重复一个词元，从不重复含义。）

- The **environment** is a source of truth too (`package.json` scripts, config files, the directory layout, `--help` output), and a document that restates it is a **cache**: a copy of a lookup, earning its load only when the lookup is expensive. Cache what the agent cannot find by looking: the unwritten convention, the reason behind a choice, the gotcha no config confesses. Leave the one-file, one-command lookups to the environment, where they cannot go stale.

  环境也是事实来源（`package.json` 脚本、配置文件、目录布局、`--help` 输出），复述环境的文档就是一份**缓存**：一次查找结果的副本，只有在查找代价高昂时才值得它的负载。缓存智能体靠查看找不到的东西：不成文的约定、某个选择背后的理由、任何配置都不会坦白的坑。把只需看一个文件、跑一条命令的查找留给环境，在那里它们不会过时。

- Check every line for **relevance**: does it still bear on what the document does? A line loses relevance by never bearing on the task (mere exposition, or a branch that should be disclosed) or by going stale as the behaviour or world it describes changes. Shorter documents are easier to keep relevant. Without a pruning discipline the default fate is **sediment**: stale layers that settle because adding feels safe and removing feels risky, until you must core down through them to find what is still live.

  逐行检查**相关性**：这一行是否仍与文档所做的事有关？一行内容失去相关性，要么是因为它从未与任务有关（纯属阐述，或是一个本该被披露出去的分支），要么是因为它所描述的行为或世界变了而过时。文档越短，越容易保持相关。没有修剪纪律，默认的结局就是**沉积层**：陈旧的层层堆积沉淀下来，因为添加感觉安全、删除感觉有风险，直到你必须钻探穿过它们，才能找到仍然有效的部分。

- Hunt no-ops sentence by sentence: an instruction the model already obeys by default pays load to say nothing. The test (does it change behaviour versus the default?) is model-relative, not reader-relative: two people disagreeing about a no-op disagree about the default, and settle it by running the document, not by debate. When a sentence fails, delete the whole sentence rather than trim words from it. The test also grades leading words: a word too weak to beat the default (_be thorough_ when the agent is already thorough-ish) is a no-op, and the fix is a stronger word (_relentless_), not a different technique.

  逐句搜捕**空操作**（no-op）：一条模型默认就会遵守的指令，付出了负载却什么也没说。检验方法（与默认行为相比，它是否改变了行为？）是相对于模型的，而不是相对于读者的：两个人对某句是不是空操作有分歧，其实是对默认行为有分歧，应当通过运行文档来解决，而不是争论。一句话不合格时，删掉整句，而不是从中删减几个词。这项检验也给引导词打分：一个弱到压不过默认行为的词（智能体本来就比较彻底时说 _be thorough_（要彻底））就是空操作，解决办法是换一个更强的词（_relentless_（锲而不舍）），而不是换一种技巧。

REFERENCE 参考文档 · SKILL-MECHANICS.md

Skill mechanics 技能机制

Source 原文

The skill-specific branch of writing-for-agents: what changes when the document is a skill (frontmatter, the invocation choice, and router skills). Everything else about writing it is the universal reference in SKILL.md.

writing-for-agents 中专属技能的分支：当文档是技能时，有哪些不同（frontmatter、调用方式的选择，以及路由技能）。关于如何写好它的其他一切，都是 SKILL.md 中的通用参考。

Invocation 调用方式

Two choices, trading the two loads:

有两种选择，在两种负载之间做取舍：

- A model-invoked skill keeps a `description`, so the agent can fire it autonomously, and other skills can reach it. You can still type its name: model-invocation always _includes_ user reach; a description only ever adds agent discovery, never removes the human's. The description is the skill's top-level context pointer, forced to stay loaded at all times: permanent context load in exchange for discoverability. A model-invoked skill whose content is all reference is also one home for shared reference: another skill can invoke it, so reference needed by several skills lives in one place. Mechanics: omit `disable-model-invocation`, and write a model-facing description carrying the trigger branches (the pointer-writing rules in `SKILL.md` apply in full).

  模型调用的技能保留 `description`，这样智能体就能自主触发它，其他技能也能抵达它。你仍然可以手动输入它的名字：模型调用始终 _包含_ 用户的可达性；description 只会额外增加智能体的发现能力，从不剥夺人的调用能力。description 是技能的顶层上下文指针，被迫始终保持加载：用永久的上下文负载换取可发现性。内容全是参考的模型调用技能，也是共享参考的一个安身之处：另一个技能可以调用它，因此多个技能都需要的参考可以只放在一个地方。机制：省略 `disable-model-invocation`，并编写一条面向模型、承载触发分支的 description（`SKILL.md` 中编写指针的规则完全适用）。

- A user-invoked skill strips the description from the agent's reach: only the human typing its name can invoke it, and no other skill can. Zero context load, but it spends cognitive load: you are the index that must remember it exists. Mechanics: set `disable-model-invocation: true`; the `description` becomes human-facing: a one-line summary, trigger lists stripped.

  用户调用的技能把 description 从智能体的可达范围中剥离：只有人输入它的名字才能调用它，任何其他技能都不能。上下文负载为零，但它消耗认知负载：你就是那个必须记得它存在的索引。机制：设置 `disable-model-invocation: true`；`description` 变成面向人的：一行摘要，去掉触发词列表。

Pick model-invocation only when the agent must reach the skill on its own, or another skill must. If it only ever fires by hand, make it user-invoked and pay no context load.

只有当智能体必须自己抵达这个技能，或另一个技能必须抵达它时，才选择模型调用。如果它只会被手动触发，就把它做成用户调用，不付任何上下文负载。

Shared reference that two user-invoked skills both need can live in neither: with no descriptions, neither can fire the other. Push it to a plain file outside the skill system: external reference any skill can point at.

两个用户调用技能都需要的共享参考，放在哪一个里都不行：两者都没有 description，谁也触发不了谁。把它推到技能体系之外的一个普通文件里：成为任何技能都能指向的外部参考。

Splitting by invocation 按调用方式拆分

The invocation cut of splitting (the sequence cut lives in `SKILL.md`): split off a model-invoked skill when you have a distinct leading word that should trigger it on its own (a trigger word you actually use in your prompts), or another skill must reach it. You pay context load for the new always-loaded description, so that independent reach has to be worth it.

这是拆分中按调用方式的那一刀（按顺序的那一刀在 `SKILL.md` 中）：当你有一个独立的引导词应当单独触发它（一个你确实会在提示词里使用的触发词），或另一个技能必须抵达它时，就拆出一个模型调用技能。你要为新增的始终加载的 description 付出上下文负载，所以这种独立的可达性必须值得。

Router skills 路由技能

When user-invoked skills multiply past what you can remember, that piled-up cognitive load is cured by a **router skill**: one user-invoked skill that names the others and when to reach for each, so the human has one skill to remember instead of many. It can only hint, never fire them: user-invoked skills have no description, so nothing but the human can reach them.

当用户调用技能多到你记不住时，这种堆积起来的认知负载可以用**路由技能**来解决：一个用户调用技能，列出其他技能以及各自该何时取用，这样人只需记住一个技能，而不是许多个。它只能提示，从不能触发它们：用户调用技能没有 description，所以除了人，什么都抵达不了它们。

REFERENCE · 参考

Reference and License
参考与许可证

Skill index, fixed source links, and MIT license.

技能索引、固定源码链接与 MIT 许可证。

This appendix provides a compact index, immutable source information, and the license under which the original skills are distributed.

本附录提供简明索引、不可变源码信息，以及原始技能的分发许可证。

Skill Index 技能索引

Engineering Engineering（工程）

- `ask-matt` - routes a situation to the right skill or flow.

  `ask-matt`--把当前情况路由到正确的技能或流程。
- `diagnosing-bugs` - enforces a tight, red-capable diagnosis loop.

  `diagnosing-bugs`--强制建立紧凑、能变红的诊断循环。
- `grill-with-docs` - sharpens plans while maintaining the glossary and ADRs.

  `grill-with-docs`--锐化计划，同时维护术语表和 ADR。
- `triage` - moves issues and external PRs through triage states.

  `triage`--推动 issue 和外部 PR 走完分诊状态。
- `improve-codebase-architecture` - finds and visualizes deepening opportunities.

  `improve-codebase-architecture`--寻找并可视化加深机会。
- `setup-matt-pocock-skills` - configures tracker, labels, and domain docs.

  `setup-matt-pocock-skills`--配置 tracker、label 和领域文档。
- `tdd` - runs behavior-first red-green development at agreed seams.

  `tdd`--在约定的接缝处进行行为优先的红-绿开发。
- `to-spec` - synthesizes the current conversation into a published spec.

  `to-spec`--把当前对话综合成已发布的规范。
- `to-tickets` - decomposes work into blocked tracer-bullet tickets.

  `to-tickets`--把工作分解为带阻塞关系的曳光弹票据。

- `wayfinder` - charts huge, uncertain work as decision tickets.

  `wayfinder`--把巨大而不确定的工作绘制为决策票据。
- `implement` - builds a spec or ticket set, then reviews and commits.

  `implement`--实现规范或票据集，然后审查并提交。
- `implement-spec` - builds a whole spec as a parallel task graph on one integration branch.

  `implement-spec`--把整份规范作为并行任务图，在一个集成分支上完成实现。
- `prototype` - answers one design question with throwaway code.

  `prototype`--用一次性代码回答一个设计问题。
- `research` - investigates a question against high-trust primary sources.

  `research`--依据高可信度的一手来源调研一个问题。
- `domain-modeling` - sharpens ubiquitous language and records durable decisions.

  `domain-modeling`--锐化通用语言并记录持久决策。
- `codebase-design` - supplies the vocabulary for deep modules and seams.

  `codebase-design`--提供深模块与接缝的设计词汇。
- `code-review` - reviews Standards and Spec as separate axes.

  `code-review`--把 Standards 与 Spec 作为两个独立维度审查。
- `pr` - shapes a PR body around the smallest visual and before/after evidence.

  `pr`--围绕最小的可视化和前后对比证据组织 PR 正文。
- `retro` - turns a session's mistakes into environment changes.

  `retro`--把一次会话中的失误转化为对环境的改进。
- `wizard` - generates an interactive Bash guide for human-only steps.

  `wizard`--为只有人类能完成的步骤生成交互式 Bash 向导。

Productivity Productivity（效率）

- `grill-me` - starts a stateless, relentless interview.

  `grill-me`--启动无状态、穷追不舍的访谈。
- `grilling` - provides the reusable design-tree interview primitive.

  `grilling`--提供可复用的设计树访谈原语。
- `handoff` - writes a portable continuation document for another agent.

  `handoff`--为另一个智能体编写可移植的交接文档。

- `teach` - maintains a stateful, multi-session teaching workspace.

  `teach`--维护有状态、跨多个会话的教学工作区。

- `to-questionnaire` - turns a knowledge gap into a recipient-ready questionnaire.

  `to-questionnaire`--把知识缺口转成接收者可直接填写的问卷。

- `wait-what` - re-pitches a message that did not land.

  `wait-what`--重新表达没有讲明白的消息。

- `writing-for-agents` - sets the rules for skills, AGENTS.md, and CLAUDE.md.

  `writing-for-agents`--为技能、AGENTS.md 和 CLAUDE.md 制定写作规则。

Fixed Source Links 固定源码链接

Every chapter and reference section links to `https://github.com/mattpocock/skills/blob/b0618bc436ad893b3c5e84e55fba86586d34a404/...`. These links point at the pinned commit rather than a branch, so they keep showing exactly the text translated here even after upstream changes. To compare with the latest version, replace the commit with `main` in the URL.

每个章节和参考小节都链接到 `https://github.com/mattpocock/skills/blob/b0618bc436ad893b3c5e84e55fba86586d34a404/...`。这些链接指向固定 commit 而非分支，因此即使上游发生变化，它们仍会精确显示本书所翻译的文本。如需与最新版本对比，把 URL 中的 commit 替换为 `main` 即可。

License 许可证

The original skills are distributed under the MIT License. The English text below is the binding license; the Chinese translation is for reference only.

原始技能以 MIT 许可证分发。下方英文文本为具有约束力的许可证，中文翻译仅供参考。

MIT License

MIT 许可证

Copyright (c) 2026 Matt Pocock

版权所有 (c) 2026 Matt Pocock

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

特此免费授予任何获得本软件及相关文档文件（“软件”）副本的人不受限制地处理本软件的权利，包括但不限于使用、复制、修改、合并、发布、分发、再许可和/或销售本软件副本的权利，并允许向其提供本软件的人这样做，但须符合以下条件：

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

上述版权声明和本许可声明应包含在本软件的所有副本或主要部分中。

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

本软件按“原样”提供，不附带任何明示或暗示的担保，包括但不限于对适销性、特定用途适用性和不侵权的担保。在任何情况下，作者或版权持有人均不对任何索赔、损害或其他责任负责，无论是在合同诉讼、侵权诉讼或其他诉讼中，由本软件或本软件的使用或其他交易引起、由此产生或与之相关。